

A Study of Microgrid Power Scheduling Based on Mixed-Integer Programming and Historical Scenario Optimization

Abstract

Microgrid procurement links uncertain demand and photovoltaic (PV) generation to battery losses, discharge limits, and the cost of revising purchases. We develop schedules for the four tasks under explicit information and settlement rules. Source records are aligned by interval and forecast issue time; power is converted to energy, structural date labels are filled, and hourly PV forecasts are integrated without smoothing observations.

For Problem 1, a deterministic mixed-integer linear program jointly schedules purchases and battery flows. HiGHS returns a daily cost of 35,126.95 CNY, 26.898% below the no-storage benchmark, with a reported zero optimality gap for that supplied instance.

For Problems 2-4, we use historical daily forecast errors to evaluate a shared purchase vector and a causal discharge-reserve vector. A threshold issued before execution can retain energy for later expensive deficits; setting it to 1,200 kWh recovers maximum deficit discharge. Four L-BFGS-B searches generate candidates for each decision, which are then explicitly rescored. Twenty-two continuous February-December 2025 paths compare the reserve and greedy controllers with the same search counts and per-search limits.

For Problem 2, the reserve policy costs 13,845,452.99 CNY and saves 69,508.07 CNY against its matched greedy control. Emergency expenditure nevertheless rises by 22,702.84 CNY. For Problem 3, the selected corrected-PV rolling reserve policy costs 13,574,461.81 CNY. Within the reserve family, state feedback with paid revisions saves CNY 208,115.60; adding later raw PV releases saves CNY 167,121.93. The selected corrected-PV reserve policy costs an additional CNY 6,206.07 relative to its matched greedy control, so additional storage flexibility does not assure lower realized cost.

For Problem 4, planning prices are separated from actual billed prices. The midnight branch selects the causal-OLS midnight reserve policy, costing 14,618,518.98 CNY; the rolling branch selects the raw-PV/causal-OLS rolling reserve policy, costing 14,313,637.10 CNY. Both use actual variable-price settlement and the full fivefold emergency tariff.

Independent reconstruction of source data, physical and fee replay, candidate checks, and isolated reproduction support the consistency of the implementation. The five workbooks contain the resulting schedules. Because we selected policies on a repeatedly inspected historical year and used finite nonsmooth searches, these results cannot establish future savings or global optimality.

Keywords: microgrid procurement, causal storage reserve, historical scenarios, paid rolling revisions, mixed-integer linear programming

1. Problem statement and modeling rationale

1.1. Decisions, costs, and required outputs

The grid-connected microgrid meets residential demand using photovoltaic (PV) generation, purchases from the external grid, and battery storage. The battery can absorb PV surplus or low-price purchases for later use. Its losses and power limits make minimum purchased energy and minimum expenditure different objectives. Uncertain load and PV output can leave a deficit after the normal commitment has been fixed, requiring emergency purchases at five times the current tariff. The four questions in the competition statement [1] vary the information and contractual flexibility available to this system.

The supplied battery has a rated capacity of 12,000 kWh, maximum charging/discharging power of 5,000 kW, an operating energy range of 1,200-10,800 kWh, and a stated charging/discharging efficiency of 90%. Its stored energy at 00:00 on 1 January 2025 is 6,000 kWh. Load and PV observations are in kW, prices are in CNY/kWh, and the requested purchase and battery-flow outputs are in kWh. Section 2 distinguishes these given conditions from the measurement and efficiency interpretations needed for calculation.

Figure 1 illustrates the microgrid components and energy flows shared by the four tasks. The emergency-purchase branch applies to Q2-Q4.

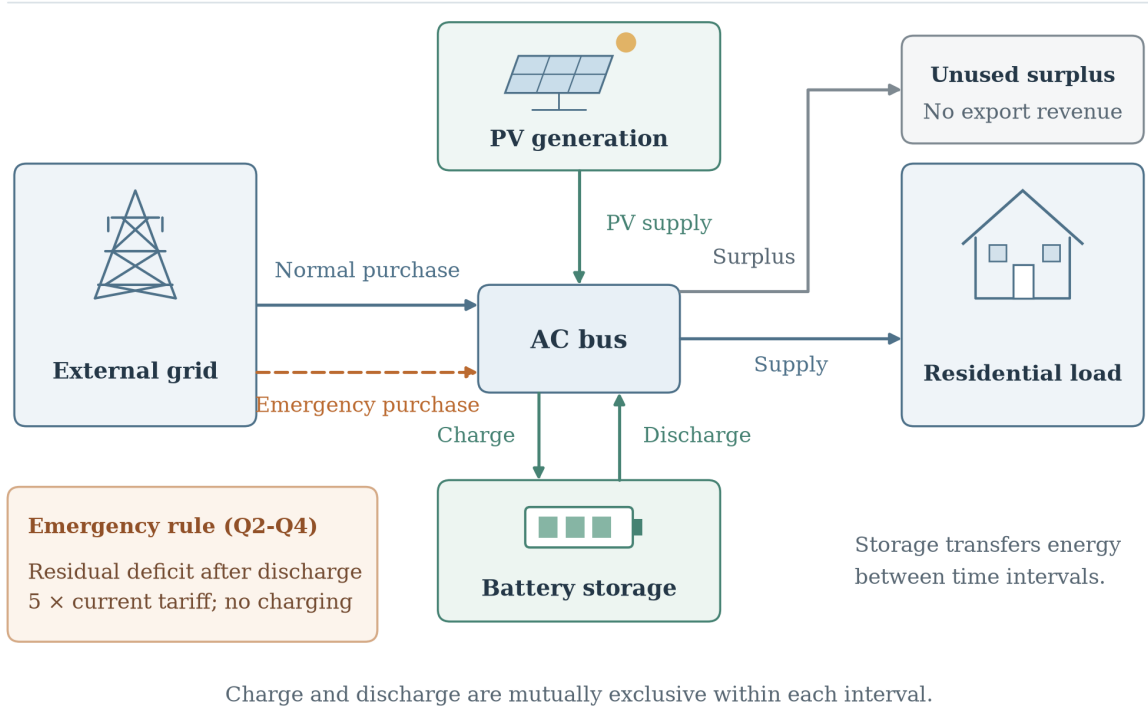


Figure 1: Microgrid configuration and energy flows. In Q2-Q4, emergency purchases cover demand left after discharge under the preissued storage reserve rule; they cannot charge storage. Unused surplus earns no export revenue. Arrows represent modeled energy flows, not detailed electrical wiring.

Q1: deterministic daily scheduling. Attachment 1 supplies a daily load, tariff, and PV forecast profile. A purchase plan is made at midnight with equal beginning and ending stored energy. The task requires the specified interval purchases, daily purchase amount and fee, four-hour charging/discharging totals, and boundary energy in `result1.xlsx`.

Q2: a fixed commitment under uncertain load and PV. With the repeated Attachment 1 tariff and Attachment 2 observations, the entire normal-purchase vector is fixed at midnight. During actual operation, emergency purchases cover any remaining supply deficit at five times the trading-time tariff; the normal commitment remains payable. The annual plan and realized storage and emergency-purchase records form `result2.xlsx`.

Q3: paid revision using released PV forecasts. Attachment 3 adds 24-hour hourly PV forecasts released at 00:00, 06:00, 12:00, and 18:00. Future purchases may be revised using later releases. The statement assigns a

50%-of-tariff breach charge to reductions and a 1.5-times-tariff price to added quantities. The model must account for these charges and emergency purchases, produce `result3.xlsx`, and assess whether later forecasts justify revising the commitment.

Q4: variable-price settlement. Attachment 4 replaces the repeated tariff with time-varying actual prices. The Q2 and Q3 tasks are repeated as `result4-2.xlsx` and `result4-3.xlsx`. We interpret this extension as preserving each branch's PV access and revision permissions. Planning-price alternatives are compared under the same actual-price settlement within each branch.

The five workbooks contain complete schedules. Q1 covers one supplied day; the annual branches cover 1 February-31 December 2025, or 334 days and 48,096 ten-minute intervals each. Appendix D gives every required result for 20 March, 21 June, 23 September, and 21 December, including all emergency episodes. Its captions identify the original statement requirements; paper table numbers are editorial.

1.2. Why these model components belong together

Battery operation couples successive intervals: charging now changes the energy available to meet a later deficit. In Q1, the supplied profiles permit a deterministic mixed-integer linear program (MILP), with linear energy accounting and binary operating modes that exclude simultaneous charging and discharging. Related battery energy-management applications use this formulation [2].

For Q2-Q4, normal purchases and the timing of battery discharge must be assessed together. Fixing purchases does not make immediate maximum discharge economical: using storage for a low-price deficit can leave a later high-price deficit uncovered. We therefore choose a purchase vector and a time-varying reserve threshold, then evaluate the same pair on complete historical daily net-load errors. The threshold limits discharge using current energy; it does not give a scenario advance knowledge of its future. Stochastic receding-horizon microgrid control provides a precedent for combining uncertainty with updated information [3], but does not specify this problem's revision bill. An explicit final-contract cost extends the paired decision to permitted update times. Each update starts from actual stored energy, freezes executed purchases and thresholds, and can retain the complete current pair.

Table 1 links each model component to the limitation it addresses and the evidence used to assess it. We evaluate forecast changes by their effect on operating cost, using the same physical and settlement rules for each comparison.

Table 1: Task structure, model choice, and planned verification.

Task structure	Limitation requiring attention	Component and purpose	Evidence in this paper
Battery energy couples successive intervals; charge and discharge cannot coincide	Purchasing each interval's net deficit ignores storage transfer	Linear balance and energy recursion with binary operating modes	Q1 bounds, no-storage comparison, efficiency and power-side cases
Purchases are frozen before uncertain supply and demand arrive	A zero predicted deficit does not imply a low realized bill; immediate discharge may consume energy valuable later	Shared purchases and reserve thresholds evaluated on complete historical daily errors	Q2 baselines and matched greedy/reserve comparisons; physical and cost checks
Forecasts arrive in stages and changing a commitment costs money	New information matters only through an allowed action; free procurement revision omits fees	Remaining-day purchase and reserve updates from actual energy with final-contract settlement	Q3 no-update, energy-feedback, and new-PV comparisons within each controller
Actual prices fluctuate and need not be known in advance	A more elaborate price input need not improve the resulting decision	Fixed versus causal price inputs under identical actual-price settlement	Separate Q4-2 and Q4-3 price comparisons

Figure 2 maps the information, decisions, and verification interfaces. Q1 solves its own deterministic MILP. Its physical constraints provide a common foundation, while its optimized battery trajectory is specific to the supplied day. Q2-Q4 share the historical-scenario objective and causal reserve execution, with the previously selected Q2 history window and cost weights inherited by the rolling and variable-price branches. A nominal MILP supplies starting values, not a trajectory that later policies must reproduce. Each annual policy propagates its own actual battery state from the common initialization.

PV bias correction is an additional candidate, retained only by historical cash ranking within the selected controller family. It does not change the common physical rules or the task-specific information permissions.

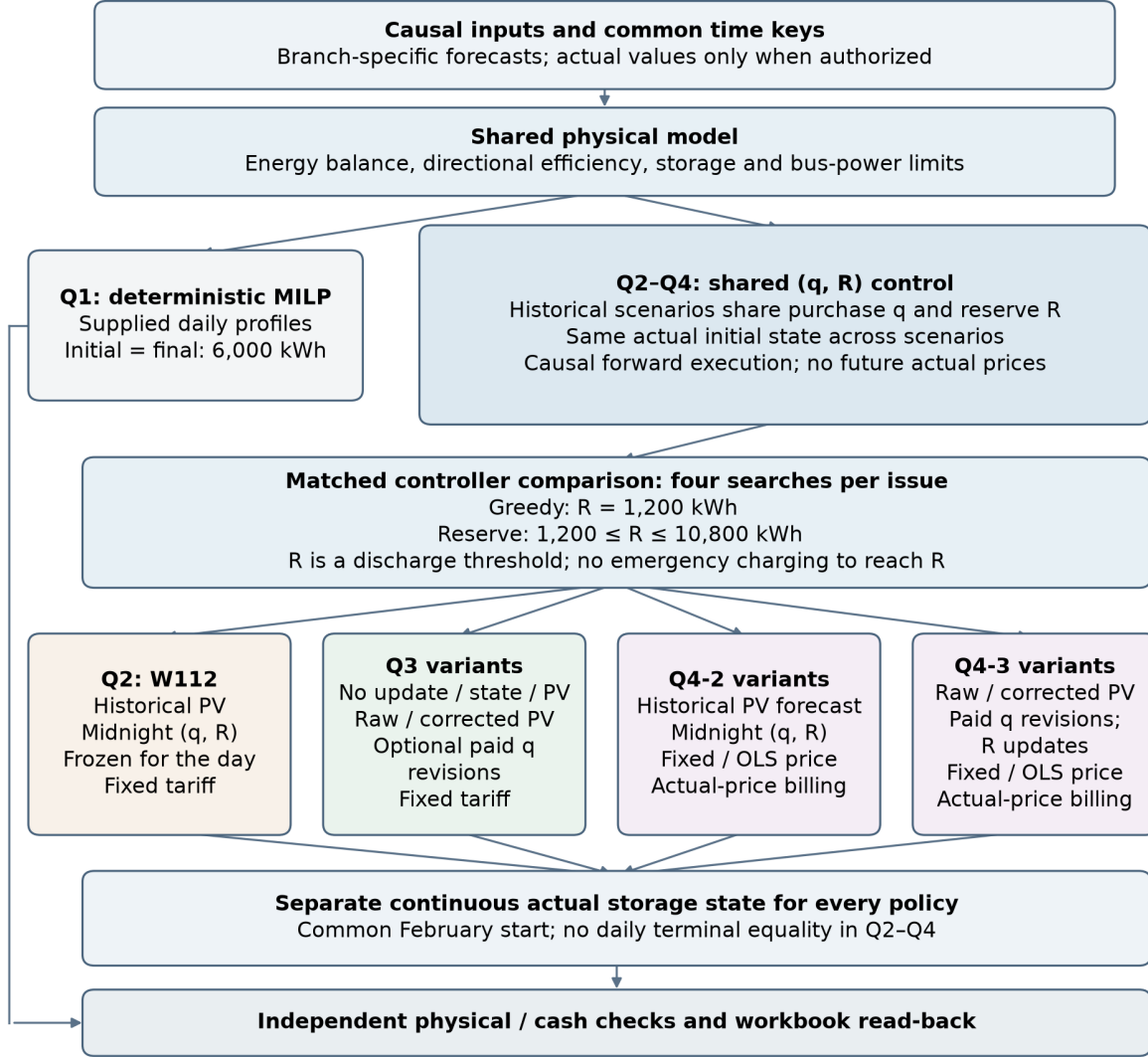


Figure 2: Common data and physical constraints feed an independent Q1 MILP and a shared historical-scenario purchase/reserve model for Q2-Q4. Historical-PV midnight policies and released-PV rolling policies have different information permissions. Q1 starts and ends at 6,000 kWh; annual policies carry their own actual state continuously without a daily terminal equality. Planning prices and billed prices remain distinct.

2. Data, working interpretations, and notation

2.1. Data interfaces and information cutoffs

Attachment 1 contains 144 observations for the supplied Q1 day and the repeated reference tariff. Attachments 2 and 4 each contain 365 daily ten-minute profiles, giving 52,560 aligned actual intervals. Attachment 3 contains four releases per day with 24 hourly forecast targets per release: 1,460 releases and 35,040 release-target records. Attachment 5 supplies the workbook structures. Source workbooks remain unchanged. Derived tables retain source rows, dates, interval boundaries, and forecast issue times. We join records by explicit date-time keys and keep issue time separate from target time, so each scheduling input can be traced to its source and its availability at the decision.

Power observations in kW are multiplied by $\Delta t = 1/6$ h to obtain kWh. Tariffs already have units CNY/kWh and are not multiplied by the interval length. The retained data audit found no missing, non-numeric, or non-finite numerical entries and no negative load, PV, or price values in the input regions. All 23,540 valid zero observations in actual PV remain present. Large adjacent changes are retained and traced to source cells for review; a jump alone is insufficient evidence of a recording error. No observations are smoothed, shifted, or re-estimated for this paper.

Attachment 3 contains 1,095 blank date labels on repeated within-day forecast-release rows. These are structural labels, not missing PV forecasts. Their earlier forward filling follows the original row order and retains a transformation record without changing forecast-power values. Under the interpolation convention in Section 2.2, the first release at 00:00 on 1 January lacks an available left endpoint for its first hour. Its six ten-minute derived intervals are left unavailable. The remaining 1,459 releases each provide 144 derived intervals, giving 210,234 converted records in total. This first-release gap does not enter the required February-December output horizon. Original ten-minute labels are interpreted as interval ends and readings as interval-average power. Hourly forecast point values are interpolated linearly; the first forecast hour uses an earlier issued value at its left endpoint. These measurement conventions define the conversions, rather than being corrections to forecast values.

Figure 3 summarizes the supplied year before discussing the decision models. Daily demand and PV energy integrate all 144 ten-minute intervals, including zero-generation observations; the daily mean actual tariff is an unweighted arithmetic mean of those same intervals, not a procurement-weighted price. Calendar-month groups distinguish seasonal position from within-month daily variation. These are retrospective descriptions of all 365 supplied days, including January, rather than the 334-day policy-evaluation sample or information supplied to a midnight decision. Density estimates summarize distributions without replacing or smoothing any observed time trajectory.

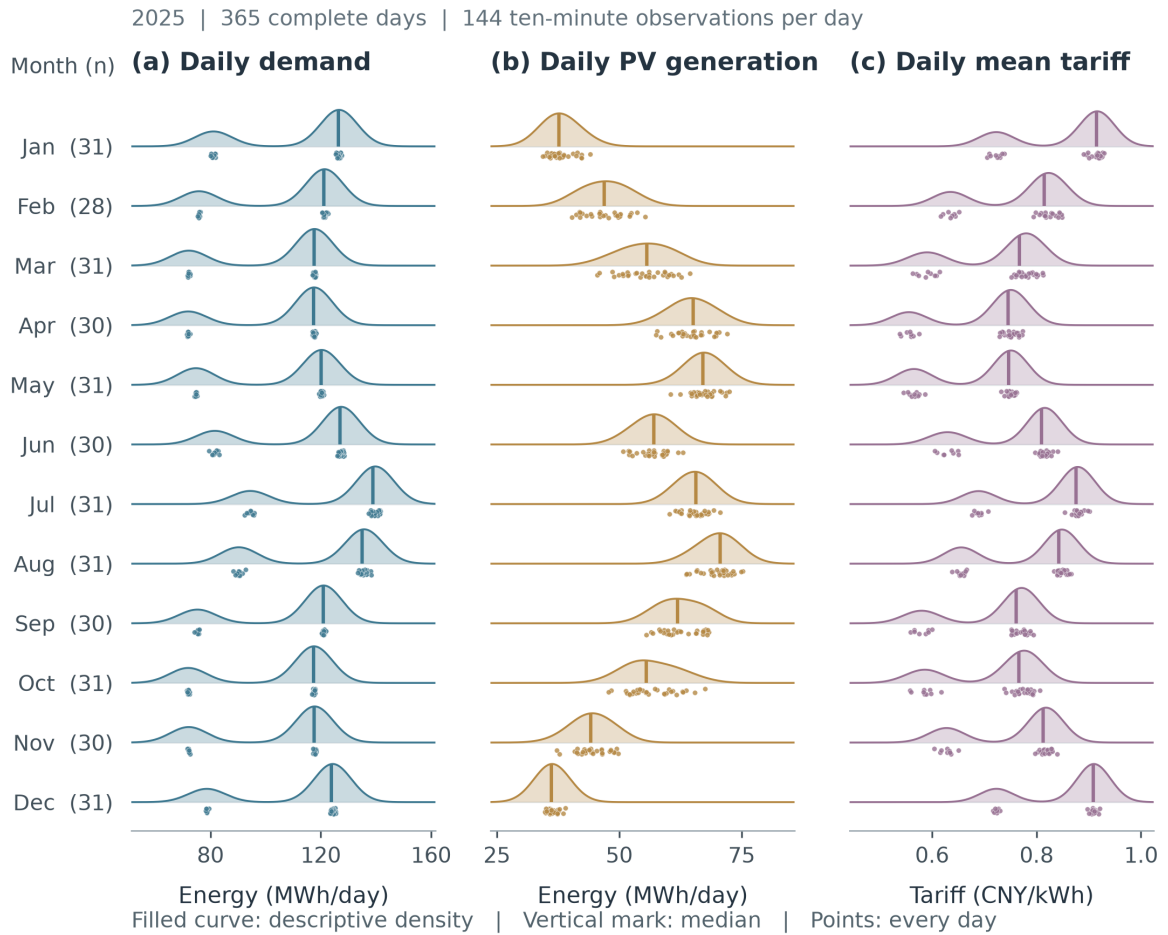


Figure 3: Monthly distributions across all 365 supplied days of 2025. Daily demand and PV energy use all ten-minute observations (MWh/day); mean actual tariff is the daily arithmetic mean (CNY/kWh). Original daily observations and sample counts accompany each density curve; vertical marks are monthly medians. A Scott bandwidth computed from all 365 daily values is held fixed across months within each column, with a shared density-height scale. Months stay in calendar order, and zero-PV intervals remain in daily totals. Panel scales differ by physical quantity.

Table 2 distinguishes admissible planning inputs from subsequent actual execution data. In the annual tasks, load forecasts are fixed at midnight. A forecast for a historical date is reconstructed with that date's own information cutoff, rather than with coefficients fitted later [4].

Table 2: Available information and adopted purchase/reserve update schedule.

Branch	Midnight load/PV input	06:00, 12:00, 18:00	Planning price	Actual settlement
Q1	Supplied deterministic daily profiles	No plan revision	Attachment 1	Attachment 1
Q2	Historical-data load and PV forecasts	Midnight purchases and reserve thresholds remain fixed	Attachment 1	Attachment 1
Q3	Midnight load forecast; current Attachment 3 PV release	Actual energy and permitted PV release; future purchases and thresholds only	Attachment 1	Attachment 1 plus paid procurement revisions
Q4-2	Q2 historical load/PV information; Attachment 3 excluded under the working interpretation in Table 3	Midnight purchases and reserve thresholds remain fixed	Fixed reference or causal price forecast	Actual Attachment 4 prices
Q4-3	Same release permissions as Q3	Actual energy, permitted PV release, and completed price history; future purchases and thresholds only	Fixed reference or causal price forecast	Actual Attachment 4 prices plus paid procurement revisions

2.2. Working interpretations and their consequences

The problem specifies energy bounds, rated power, and a 90% efficiency, while leaving some measurement and settlement details open. Table 3 separates the supplied requirements from the working interpretations adopted in our calculations. Interval observations follow the alignment and data-quality procedures in Section 2.1.

Table 3: Physical and accounting conventions used in the calculations.

Item	Adopted convention and source	Where it matters
Battery limits	Rated capacity 12,000 kWh; operating energy 1,200-10,800 kWh, given in the statement	All state constraints
Efficiency and power side	Charging and discharging efficiencies each 0.9; 5,000 kW measured at the AC bus, a working interpretation	Energy recursion and power bounds; Q1 alternatives in Section 4.2
Observation and balancing	End-of-interval observations and automatic balancing within the interval are available	Realized deficit coverage at ten-minute resolution
Initial and terminal energy	Q1 starts and ends at 6,000 kWh; annual policies share 7,268.4231640740745 kWh after a common January seasonal-baseline warm-up	Annual paths branch on 1 February and continue without daily/monthly resets
Contract settlement	The final effective quantity is settled once relative to the midnight commitment under rule A	Paid revisions in Q3/Q4-3
Storage execution	Author-selected surplus charging and discharge limited by a preissued reserve threshold; the 1,200 kWh threshold recovers greedy discharge	Shared scenario and actual recursion (3); threshold policy class, not unrestricted optimal control
Unused energy and emergencies	Unused committed purchases remain chargeable; no export revenue; emergency energy covers the residual deficit only	Actual cash bill and surplus disposal
Q4-2 PV information	Preserve Q2's historical-PV information set when recalculating it under variable prices; an author interpretation, not an explicit branch-specific ban on Attachment 3	Q4-2 input construction and matched planning-price comparisons
Variable-price information	Future realized prices are unavailable to planning unless already observed	Causal price inputs and actual-price rebilling

A common seasonal-baseline warm-up runs from 1 to 31 January, starting at the statement's 6,000 kWh. Its 1 January cold start uses zero normal planned purchases; subsequent January purchases follow that baseline. The retained 4,464-interval record ends at 7,268.4231640740745 kWh, which becomes the shared 1 February initial state (Appendix A). The compared annual policies branch from this state; it is not the outcome of running each

final policy separately through January. Thereafter, each policy carries its own actual energy across every midnight and month boundary. Q1's equality of initial and terminal energy is required by its task. In Q2-Q4, a nominal zero-deficit trajectory and terminal energy of 1,200 kWh constrain the MILP initializer only. They do not constrain the final purchase/reserve pair, and actual storage is never reset to that level.

The efficiency and power-side conventions specify where losses occur in the energy recursion. Applying 90% in each direction gives a different usable transfer from a symmetric 90% round-trip interpretation; locating the power limit inside the battery also changes the bus-flow bounds. Section 4.2 compares these interpretations for Q1. The no-export convention is needed because the statement gives no selling tariff. Q1 permits only PV curtailment, whereas Q2-Q4 may also dispose of unused committed purchases that remain chargeable. Emergency purchases supply only the deficit left by the adopted discharge rule and cannot charge the battery for arbitrage.

The commitment and settlement conventions connect a causal decision to its realized bill. At each decision time, the model can use completed actual observations, released forecasts, current actual stored energy, and admissible price information. The original purchase commitment stays fixed in Q2 and Q4-2; we also fix the day's reserve thresholds at midnight as a controller-design choice. Q3 and Q4-3 may update future purchases and thresholds at their permitted decision times. Rule A settles each interval's final effective purchase quantity once against its midnight commitment; earlier versions are not added as separate purchases. No separate reserve-change fee is introduced, because the contract charges concern grid quantities and no storage switching cost is supplied. Section 3 gives the exact refund-and-breach-charge treatment. Future actual prices are withheld from planning, while actual Attachment 4 prices determine Q4 settlement.

Within a forecast hour, let a and b denote consecutive forecast target times, with power values $\widehat{P}_a$ and $\widehat{P}_b$. For any ten-minute interval I_t within that hour, the generation input is

$$\widehat{G}_t = \int_{I_t} \left[\widehat{P}_a + \frac{\tau - a}{b - a} (\widehat{P}_b - \widehat{P}_a) \right] d\tau. \quad (1)$$

Time in (1) is measured in hours, giving kWh. Only issued values are used. A release contains 24 hours of forecasts, but the current optimization uses only intervals up to the current day's 24:00. The implications of this remaining-day horizon are discussed in Section 9.

A single internal energy state represents the battery. We omit self-discharge, degradation charges, ramping limits, network constraints, communications cost, and sub-interval converter dynamics because the task supplies no parameters for them. The model retains capacity, efficiency, power limits, and energy transfer between intervals. It describes interval-level scheduling; it has not been validated as a physical controller operating on a seconds scale. Historical cost calculations require no assumption of normal, independent, or stationary errors, and the empirical scenario average does not establish those properties.

2.3. Symbols and units

At release stage $k \in \{0, 1, 2, 3\}$, the clock time is $6k$ hours and the first unexecuted ten-minute interval is $\tau_k = 36k$. Thus $\mathcal{T}_k = \{\tau_k, \dots, 143\}$. Q2 and Q4-2 use only $k = 0$. A scenario at stage k starts from the actual boundary state $E_{d,\tau_k}^{\text{act}}$; k is a release index, whereas the subscript of E_t always denotes a physical interval boundary. Previously executed quantities for $t < \tau_k$ are fixed. Suppress d, k only within one decision.

Table 4 collects quantities shared by the models. Stored energy E is measured inside the battery, whereas charging and discharging flows are measured at the bus.

Table 4: Main notation.

Symbol	Meaning	Unit
d, t, k, s	Operating day; ten-minute interval $t = 0, \dots, 143$; decision stage; completed historical scenario day	Index
$\tau_k, \mathcal{T}_k$	First unexecuted boundary $\tau_k = 36k$; remaining intervals $\{\tau_k, \dots, 143\}$	Index, set
$H_{d,k}$	Eligible completed historical days within the preceding W calendar days	Set
L_t, G_t	Community load energy and PV generation energy in interval t	kWh
$\widehat{L}_{d,t,00}$	Load-energy forecast for day d , interval t , fixed at midnight	kWh
$\widehat{G}_{d,t,k}$	PV-energy forecast for day d , interval t , available at stage k	kWh
q_t^0	Original normal-purchase commitment made at midnight for interval t	kWh
q_t^k	Normal-purchase commitment for interval t after stage k	kWh
$\bar{q}_t$	Final effective normal-purchase commitment executed in interval t	kWh

Table 4: Main notation. (continued)

Symbol	Meaning	Unit
$R_t^0, R_t^k, \bar{R}_t$	Discharge reserve threshold issued at midnight, updated at stage k , and finally applied in interval t	kWh
c_t	Energy charged from the microgrid bus into the battery in interval t	kWh
b_t	Energy discharged from the battery to the microgrid bus in interval t	kWh
u_t	Emergency purchase covering the remaining supply deficit in interval t	kWh
w_t	Unused surplus disposed of without sale revenue in interval t	kWh
E_t	Stored energy at the beginning of interval t	kWh
$p_{t k}^{\text{plan}}, p_t^{\text{act}}$	Price used for planning; price used for actual settlement	CNY/kWh
P	Bus-side charge/discharge energy limit per ten-minute interval, 5000/6	kWh
η_c, η_d	Charging and discharging efficiencies, respectively	Dimensionless
$r_{t,k}^s, n_{d,t,k}^s$	Historical net forecast error; scenario net load	kWh
W	Length of the calendar window used to select completed historical days	day
κ	Emergency-cost weight in the empirical planning objective	Dimensionless
μ	Fixed terminal-inventory value proxy in the planning objective	CNY/kWh
ν	Common terminal-energy valuation used only in the historical inventory audit	CNY/kWh
F_A	Final rule-A contract cost relative to the midnight commitment	CNY
J_k	Remaining-horizon empirical planning objective at stage k	CNY

The positive part is $(x)_+ = \max(x, 0)$. Day and stage subscripts are suppressed when the context fixes them. Unadorned q and R denote the candidate purchase and reserve vectors. Both are shared across scenarios; scenario-specific energy states and emergency quantities arise only by forward execution of that common policy.

3. Common execution model and joint purchase/reserve objective

3.1. Physical execution and historical scenarios

Let q_t be the effective normal purchase in interval t . At the AC bus, normal purchases, PV, discharge b_t , and emergency purchases u_t meet load and charging c_t ; residual surplus w_t is disposed of without revenue. The common balance and battery recursion are

$$q_t + G_t + b_t + u_t = L_t + c_t + w_t, \quad E_{t+1} = E_t + \eta_c c_t - b_t / \eta_d. \quad (2)$$

Stored energy remains in [1200, 10800] kWh, and $0 \leq c_t, b_t \leq P = 5000/6$ kWh. Q1 jointly optimizes purchases and battery actions and sets $u_t = 0$ and $0 \leq w_t \leq G_t$. Its particular optimizer output must not be confused with the following rule for uncertain realized operation.

For Q2-Q4, let $v_t = q_t + G_t - L_t$ be the realized pre-storage surplus. Along with purchases, choose a discharge reserve threshold $R_t \in [1200, 10800]$ kWh before executing the interval. Surplus charging remains limited by current surplus, power, and capacity. During a deficit, only energy above the threshold may be discharged:

$$\begin{aligned} c_t &= \min\{(v_t)_+, P, (10800 - E_t)/\eta_c\}, \\ b_t &= \min\{(-v_t)_+, P, \eta_d(E_t - R_t)_+\}, \\ u_t &= ((-v_t)_+ - b_t)_+, \quad w_t = (v_t - c_t)_+. \end{aligned} \quad (3)$$

The threshold is a discharge rule, not a new state constraint $E_t \geq R_t$. If current energy is below it, discharge is zero; emergency purchases do not refill the battery to the threshold. Setting every $R_t = 1200$ recovers the previous greedy executor. A larger threshold can preserve energy for later deficits while increasing the current emergency purchase, so its economic value must be measured over the full path. The sign of v_t prevents simultaneous charging and discharging. Surplus may include unused paid purchases as well as PV, with no sale credit. Equations (2)-(3) enforce the physical energy and power limits for any finite $q \geq 0$ and $1200 \leq R \leq 10800$, starting from a physically admissible state.

The joint objective must anticipate how errors deplete or refill storage over successive intervals. Let $H_{d,k}$ contain eligible completed days in the preceding W calendar days with their contemporaneous forecast records. For historical day s , define

$$r_{t,k}^s = (L_t^s - G_t^s) - (\widehat{L}_{s,t,00} - \widehat{G}_{s,t,k}), \quad n_{d,t,k}^s = \widehat{L}_{d,t,00} - \widehat{G}_{d,t,k} + r_{t,k}^s. \quad (4)$$

Q2 and Q4-2 use historical-data PV forecasts. Q3 and Q4-3 match the authorized PV release stage; an energy-feedback-only comparison still uses the midnight PV version. All residuals are from completed historical dates. The available pool is shorter than W near the beginning of the evaluation period; future days are never used to fill the window.

Each scenario retains its observed within-day error order. We simulate the same remaining pair (q, R) against every n^s using (2)-(3), starting from the policy's current actual energy. Each next state depends on that scenario's current net load and preceding state. Scenarios share purchases and reserve thresholds, and execution uses no later PV release or future actual price. Allowing a separate dispatch with knowledge of each scenario's future would produce an anticipatory benchmark. The empirical sample-average construction [5] lets us evaluate a time-varying threshold policy with common decisions. The windows overlap, and the search is finite and nonsmooth. Section 8 discusses the scope of the empirical comparisons, including the absence of a shuffled-scenario test.

3.2. Complete empirical program and settlement

For known scenario net loads from (4), the complete remaining-day program is given below. The free decisions q, R are common to all scenarios; each scenario has its own forward-recursive storage and emergency quantities. Previously executed intervals are fixed.

$$\begin{aligned} \min_{q,R} \quad & J_k(q, R) = \sum_{t \in \mathcal{T}_k} C_{t,k}(q_t) + \frac{\kappa}{|H_{d,k}|} \sum_{s \in H_{d,k}} \sum_{t \in \mathcal{T}_k} p_{t|k}^{\text{plan}} u_t^s \\ & - \frac{\mu}{|H_{d,k}|} \sum_{s \in H_{d,k}} (E_{144}^s - E_{d,\tau_k}^{\text{act}}) \\ \text{subject to} \quad & v_t^s = q_t - n_t^s, \quad E_{\tau_k}^s = E_{d,\tau_k}^{\text{act}}, \\ & c_t^s = \min\{(v_t^s)_+, P, (10800 - E_t^s)/\eta_c\}, \\ & b_t^s = \min\{(-v_t^s)_+, P, \eta_d(E_t^s - R_t)_+\}, \\ & u_t^s = ((-v_t^s)_+ - b_t^s)_+, \quad w_t^s = (v_t^s - c_t^s)_+, \\ & q_t + b_t^s + u_t^s = n_t^s + c_t^s + w_t^s, \\ & E_{t+1}^s = E_t^s + \eta_c c_t^s - b_t^s/\eta_d, \\ & q_t \geq 0, \quad 1200 \leq R_t \leq 10800, \quad 0 \leq c_t^s, b_t^s \leq P, \quad u_t^s, w_t^s \geq 0, \\ & 1200 \leq E_j^s \leq 10800 \quad (j = \tau_k, \dots, 144), \\ & s \in H_{d,k}, \quad t \in \mathcal{T}_k, \quad P = 5000/6, \quad \eta_c = \eta_d = 0.9. \end{aligned} \quad (5)$$

Only q, R are free policy decisions; the displayed execution quantities are determined by recursion. There is no terminal-energy equality. Greedy controls add $R_t = 1200$; the reserve family permits the displayed interval. Algorithm 1 only approximates this nonsmooth program and does not certify its global minimum.

At midnight, $C_{t,0}(q_t) = p_{t|0}^{\text{plan}} q_t$. The emergency weight $\kappa = 5$ makes the middle term equal to the mean full fivefold emergency bill. Other weights are historical Q2 sensitivity candidates, not alternative settlement prices. The inherited $\mu = 0.38232$ CNY/kWh values retained energy in the planning objective; it is neither cash revenue nor an identified Bellman value function. The actual bill never subtracts this term.

For an intraday revision, the normal term is the final rule-A contract cost relative to the midnight commitment:

$$F_A(\bar{q}_t; q_t^0, p_t) = p_t q_t^0 + 1.5 p_t (\bar{q}_t - q_t^0)_+ - 0.5 p_t (q_t^0 - \bar{q}_t)_+. \quad (6)$$

Thus $C_{t,k}(q_t) = F_A(q_t; q_t^0, p_{t|k}^{\text{plan}})$ at a paid update. The negative term implements the adopted net reduction adjustment. Reducing a one-kWh midnight commitment to zero leaves a half-price charge; increasing it to two kWh yields $2.5p_t$. These charges implement the working settlement interpretation in Table 3.

Only the final effective quantity for each interval is billed, once. Earlier versions are not cumulatively added, and executed intervals are frozen. The realized bills are therefore

$$\begin{aligned} B_{Q2/Q42} &= \sum_t p_t^{\text{act}}(q_t^0 + 5u_t), \\ B_{Q3/Q43} &= \sum_t \left[F_A(\bar{q}_t; q_t^0, p_t^{\text{act}}) + 5p_t^{\text{act}}u_t \right]. \end{aligned} \quad (7)$$

The emergency multiplier is the entire price $5p$, not an extra $5p$ on top of the normal price. Q4 uses the actual Attachment 4 price in both components of (7), regardless of the planning price in (5). For Q1-Q3 the applicable actual tariff is the repeated Attachment 1 tariff.

3.3. Nominal initialization and candidate search

A deterministic MILP supplies a nominal starting trajectory. The minimum operators in (3), the reserve boundary, and the contract kink at $q = q^0$ make the shared objective (5) nonsmooth. Limited-memory bound-constrained search [6] generates candidates that are explicitly retained and rescored; its stopping status is not an optimality certificate. The SciPy interface supplies the bounds and termination controls [7]. The comparison gives both the reserve and greedy controllers four searches per decision, so adding two search starts alone is not counted as evidence for improved storage control.

The initializer uses a separate 28-day residual pool. For each clock hour, δ_t is the linear-interpolated empirical quantile of all available residuals within that hour, with levels 0.85 for 09:00-11:00, 0.65 for 11:00-18:00, 0.95 for 18:00-22:00, and 0.80 otherwise. Let $\tilde{L}_t = (\tilde{L}_{d,t,00} + \delta_t)_+$ and $\tilde{G}_t = \tilde{G}_{d,t,k} + (-\tilde{L}_{d,t,00} - \delta_t)_+$. These preserve the risk-adjusted net load, including negative net values. With tildes denoting nominal rather than executed variables, solve

$$\begin{aligned} \min \quad & \sum_{t \in \mathcal{T}_k} \tilde{C}_{t,k}(\bar{q}_t) \\ \text{subject to} \quad & \bar{q}_t + \tilde{G}_t + \tilde{b}_t = \tilde{L}_t + \tilde{c}_t + \tilde{w}_t, \\ & \tilde{E}_{t+1} = \tilde{E}_t + \eta_c \tilde{c}_t - \tilde{b}_t / \eta_d, \\ & \tilde{E}_{\tau_k} = E_{d,\tau_k}^{\text{act}}, \quad \tilde{E}_{144} = 1200, \\ & 1200 \leq \tilde{E}_j \leq 10800 \quad (j = \tau_k, \dots, 144), \\ & 0 \leq \tilde{c}_t \leq Pz_t, \quad 0 \leq \tilde{b}_t \leq P(1 - z_t), \quad z_t \in \{0, 1\}, \\ & \bar{q}_t \geq 0, \quad \tilde{w}_t \geq 0, \quad \tilde{w}_t \leq \tilde{G}_t \text{ if } k = 0, \quad t \in \mathcal{T}_k. \end{aligned} \quad (8)$$

At midnight, $\tilde{C}_{t,0} = p_{t|0}^{\text{plan}} \bar{q}_t$. At a paid update, $\tilde{C}_{t,k} = F_A(\bar{q}_t; q_t^0, p_{t|k}^{\text{plan}})$: the MILP introduces a nonnegative fee variable y_t and minimizes $\sum_t y_t$ subject to $y_t \geq 1.5p_t \bar{q}_t - 0.5p_t q_t^0$ and $y_t \geq 0.5p_t \bar{q}_t + 0.5p_t q_t^0$, using planning prices here. Intraday surplus has no PV-only upper bound because the inherited remaining-horizon implementation permits disposal of paid grid energy. The initializer contains no emergency purchase. Its zero-deficit balance, binary nominal dispatch and terminal target apply only to its starting trajectory. The final empirical program does not inherit those restrictions. Algorithm 1 uses $\bar{q}$ and, for one reserve start, clipped nominal end-of-interval energies; fallback behavior remains unchanged.

Algorithm 1. Joint purchase/reserve decision at an authorized stage.

1. Read the policy's actual energy and available information. Freeze both purchases and reserve thresholds for executed intervals.
2. Form release-matched historical scenarios using (4). Every candidate starts from the same current actual energy.
3. Construct a risk-profile MILP purchase start and a positive point-net-load start. Run two purchase-only L-BFGS-B searches with $R = 1200$, retaining each start, best evaluated point, and returned point: six greedy candidates.
4. Start two additional searches from the risk-profile purchase vector and the best of those six purchase vectors. In the reserve arm, their reserve starts are the MILP's nominal end-of-interval energies clipped to $[1200, 10800]$, and the constant 1,200 vector, respectively. Use the constant vector if the initializer has no energy trajectory. In the matched greedy arm, both additional searches keep $R = 1200$.

5. Retain each additional start, best evaluated pair, and returned pair. Rescore all twelve candidates with (5). At an intraday update, append the complete unchanged current pair (q, R) as candidate thirteen. Select the first score within 10^{-8} CNY of the minimum in this fixed order.
6. Apply the selected future purchases and thresholds, execute (3) until the next allowed decision, and carry actual energy forward. Settle final effective purchase quantities once using (7).

The nominal energy trajectory supplies only a starting threshold vector; final purchases, reserve thresholds, and realized day-end energy need not reproduce it.

The adopted parameters remain $W = 112$, $\kappa = 5$, and $\mu = 0.38232$, inherited from the earlier Q2 study without retuning. Every search uses `maxiter=180`, `maxfun=1600`, `maxls=30`, `maxcor=10`, `ftol=1e-10`, and `gtol=1e-5`. Procurement coordinates are scaled as $q/1000$, reserve coordinates as $(R - 1200)/1000$, and the objective as $J/10000$. Joint searches use derivatives with respect to both vectors; rule A changes only the procurement part of the gradient. The compiled forward and backward recursion evaluates the same model. At nonsmooth ties it uses recorded branch derivatives, including the intermediate contract slope p at $q = q^0$; these choices do not imply differentiability. Appendix A identifies code and configuration records.

Keeping the six original greedy candidates means the selected pair cannot have a worse current empirical score than their minimum, within the stated selection tolerance. This does not guarantee a lower realized bill, or a better score than the separately searched matched greedy controller. At an update the safeguard retains both current vectors, because changing only the threshold can change subsequent discharge. A nonfinite or infeasible optimizer return is recorded and replaced by an available finite incumbent; absence of a finite candidate stops the run. All solver statuses and retained arrays are preserved for the checks in Section 8.

4. Q1: Deterministic daily storage scheduling

4.1. Deterministic formulation and results

The supplied Q1 day fixes load, PV, and price, allowing joint optimization without a forecast-error model. We use linear energy accounting and a binary charging mode. Let $z_t = 1$ allow charging and $z_t = 0$ allow discharging. The deterministic MILP is

$$\begin{aligned}
\min \quad & \sum_{t=0}^{143} p_t q_t, \\
\text{subject to} \quad & q_t + G_t + b_t = L_t + c_t + w_t, \quad E_{t+1} = E_t + \eta_c c_t - b_t / \eta_d, \\
& E_0 = E_{144} = 6000, \quad 1200 \leq E_j \leq 10800 \quad (j = 0, \dots, 144), \\
& 0 \leq c_t \leq P z_t, \quad 0 \leq b_t \leq P(1 - z_t), \\
& q_t \geq 0, \quad u_t = 0, \quad 0 \leq w_t \leq G_t, \quad z_t \in \{0, 1\}, \quad t = 0, \dots, 143.
\end{aligned} \tag{9}$$

There are 865 variables, including 144 binary modes, and 578 linear constraints in the saved formulation, in addition to variable bounds. The physical bound P supplies the mode limit without an arbitrary large constant. HiGHS [8] solves this MILP using its integer-programming machinery. The preserved run uses Python 3.12.14 and HiGHS 1.14.0, one thread, seed 0, a 120-second limit, relative gap 10^{-9} , absolute gap 10^{-7} CNY, and primal, dual, and integrality tolerances 10^{-8} .

The returned cost is **35,126.95 CNY** (unrounded 35,126.948589289634), with **59,482.698998 kWh** purchased. The saved lower bound is 35,126.94858928963 CNY and the relative gap is zero; the same-constraint LP relaxation also reaches the same objective. The solver processed one branch-and-bound node. This establishes the optimal objective within numerical tolerance for the supplied deterministic instance and working interpretations.

Without storage, purchase is $(L_t - G_t)_+$ and excess PV is discarded. Table 5 separates the benefit of recovering PV from battery losses and from moving purchases between prices.

Table 5: Q1 daily storage comparison; bus-side energy quantities.

Configuration	Cost (CNY)	Purchase (kWh)	Charge (kWh)	Discharge (kWh)	Curtailed PV (kWh)
No storage	48,052.05	61,789.935400	0.000000	0.000000	6,247.962967
Storage, each-way efficiency 0.9	35,126.95	59,482.698998	20,740.666132	16,799.939567	0.000000

Storage saves **12,925.10 CNY**, or **26.898%** of the no-storage bill. It removes 6,247.962967 kWh of PV curtailment but incurs 3,940.726565 kWh of battery losses, leaving a net reduction of 2,307.236402 kWh in external purchases. The remaining cost benefit reflects the timing of purchases. The equal initial and final energy prevents the comparison from treating depletion of the initial battery as free energy.

Figure 4 maps the supplied Q1 net load, tariff, normal purchases, signed battery exchange, and stored energy onto a common 24-hour clock. Its 144 sectors retain the original ten-minute intervals, while the linear step views allow direct reading of purchases and all 145 stored-energy boundaries. Each ring has its own physical scale. The storage ring records the end of each interval; the separate 00:00 and 24:00 boundaries both equal 6,000 kWh for this deterministic Q1 instance.

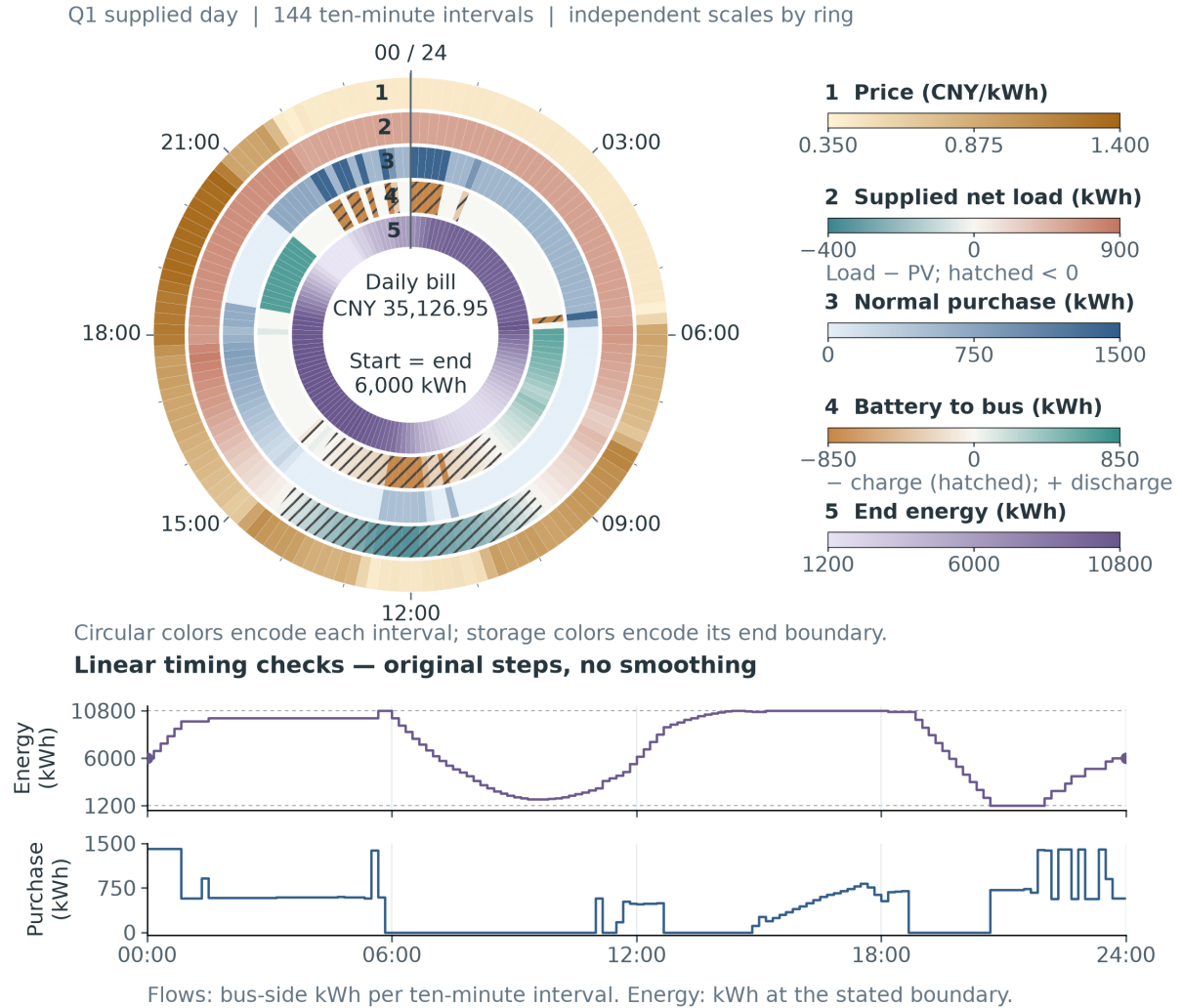


Figure 4: The saved Q1 schedule as a 24-hour circular matrix with linear step views. Each sector is one original ten-minute interval. Net load is load minus PV; battery exchange is bus-side discharge minus charge, positive for discharge. Independent scales distinguish price, interval energy, and stored energy. The storage ring uses interval-end energy; the linear trace retains all 145 boundaries from 00:00 to 24:00. Hatching marks negative values in the net-load and battery-flow rings.

The returned energy trajectory reaches both the 1,200 and 10,800 kWh operating bounds while returning to 6,000 kWh at midnight. There are 45 charging intervals and 37 discharging intervals; 19 charging intervals attain the 5,000 kW bus limit. These counts describe this solution, not a unique operating pattern. A separately saved near-optimal alternative changes some purchases or states by up to 333.333333 kWh while changing cost by only 10^{-7} CNY, showing that specific action times should not be interpreted as unique.

Tables 19-20 in Appendix D.1 give the prescribed Q1 outputs from `result1.xlsx`; Table 18 maps each workbook to the statement requirements.

4.2. Physical interpretation sensitivity and checks

Two interpretations of the stated efficiency and two locations of the power rating yield the four existing deterministic cases in Table 6. In the battery-internal power case, bus flows are bounded by $c_t \leq P/\eta_c$ and $b_t \leq P\eta_d$; in the bus case each is bounded by P .

Table 6: Q1 efficiency-by-power-side sensitivity.

Efficiency convention	Power rating applies at	Cost (CNY)
0.9 in each direction	AC bus	35,126.95
0.9 in each direction	Battery interior	35,101.57
0.9 round trip, symmetric directions	AC bus	33,801.50
0.9 round trip, symmetric directions	Battery interior	33,790.32

The four MILPs and their same-builder LP relaxations agree in objective; 48 separate algebra checks verify the physical equations. The original baseline audit independently re-reads the saved schedule and source prices, with maximum balance residual 1.14×10^{-13} kWh and state-recursion residual 8.81×10^{-13} kWh, below the 10^{-6} kWh threshold. It verifies fixed endpoints, mode exclusion, power limits, PV curtailment, and cost reconstruction. The LP comparison is useful numerical evidence, but is not an independently derived physical model. The sensitivity cases address the supplied day's interpretation; no annual robustness conclusion follows from their differences.

A schedule optimized for forecast inputs can still leave expensive deficits when actual load and PV differ and normal purchases are fixed. Q2 therefore evaluates purchases by simulating execution under historical errors.

5. Problem 2: History-Based Direct Procurement

5.1. Forecast inputs and historical scenarios

Problem 2 requires a nonnegative vector of 144 purchases to be committed at midnight. The controller also chooses 144 discharge thresholds at midnight and holds them fixed for the day. Actual load, PV, and stored energy then determine discharge and emergency purchases through (3), while normal commitments remain payable. Under-purchasing can expose the microgrid to the fivefold emergency tariff; over-purchasing can leave paid energy unused. A reserve may avoid spending stored energy on an early low-price deficit, but it may also buy emergency energy unnecessarily. Purchases and this storage response must therefore be evaluated together.

We retain the causal forecasting structure selected during January as the common input to the procurement comparisons. This structure, denoted B0, combines a load regression with one-day and seven-day lags, weekday indicators and two daily harmonics, with a PV model using three daily harmonics and a bounded first-order residual adjustment. Load parameters use the preceding 28 eligible training days and PV parameters the preceding seven days; the residual coefficient is clipped to $[-0.99, 0.99]$. The January 15-31 shadow comparison selected this structure using realized cost with an auxiliary terminal-inventory adjustment. Its structure was frozen on February 1, although parameters were subsequently re-estimated from available history. The seasonal comparator B1 uses the previous week's load and previous day's PV at the same time of day.

That January choice did not retain its economic advantage over February-December. B0 incurred CNY 16,333,683.39, including CNY 4,345,851.83 of emergency cost, whereas B1 incurred CNY 15,378,207.35. B0 was CNY 955,476.04 more expensive. This is evidence about the complete forecast-and-purchase policies; it does not establish a ranking of their prediction errors. Keeping B0 fixed allows the subsequent procurement comparisons to use the same forecast input, without retrospectively replacing it in response to a new annual ranking. A stronger comparator first adds hourly historical net-error quantiles to the point forecast and then solves a deterministic procurement model. Its best retained profile uses quantiles 0.85 during 09:00-11:00, 0.65 during 11:00-18:00, 0.95 during 18:00-22:00 and 0.80 otherwise, estimated from the preceding 28 days. We call it the *quantile-procurement baseline*. Its total cost is CNY 13,994,282.26 and emergency cost CNY 644,292.17. These are frozen historical references using their original execution and search procedures. Their nominal dispatch can differ from the greedy rule actually executed. The current paired experiment separately tests whether optimizing a discharge threshold helps under a matched search budget.

The Q2 branch of (5) uses the complete 144-interval B0 error vectors available at midnight, one shared (q^0, R^0) pair, and current actual energy. It fixes both vectors for the day and has no final-energy equality. Algorithm 1 gives each controller twelve midnight candidates under matched search limits. Eligibility, within-day ordering, fee weights, and the distinction between nominal initialization and actual execution follow Section 3.

5.2. Matched costs, selection, and historical variability

The previous Q2 study selected $W = 112$, $\kappa = 5$, and $\mu = 0.38232$; its twelve-policy matrix remains historical evidence in Appendix B. Those values and the forecast model are fixed for this new experiment. The greedy and reserve arms each use four searches with identical per-search limits. Differences reflect the resulting controller/search procedures on separate continuous state paths, not a claim that the wider feasible policy class was globally optimized. The frozen D112 reference used two searches, so comparisons against it also include the changed search budget. Table 7 reports the matched Q2 costs and the two frozen references.

Table 7: Q2 matched controllers and frozen historical references, February-December 2025 (CNY).

Policy / controller	Contract cost	Emergency cost	Total cost
Frozen quantile baseline	13,349,990.09	644,292.17	13,994,282.26
Frozen D112 (two searches)	13,075,237.03	838,655.45	13,913,892.48
Matched greedy (four searches)	13,075,508.77	839,452.30	13,914,961.06
Reserve (four searches)	12,983,297.86	862,155.13	13,845,452.99

The reserve controller saves CNY 69,508.07 (0.500%) against matched greedy. Ordinary commitments fall by CNY 92,210.91, while emergency fees rise by CNY 22,702.84. It purchases 20,865,128.06 kWh through normal commitments and 186,092.42 kWh through emergency purchases. Lower normal commitments therefore offset higher emergency expenditure. The one-million-CNY emergency reference in the earlier research served as a screening preference.

Q2's prescribed-date results and emergency records are in Appendix D.2 (Tables 21-24) and `result2.xlsx`.

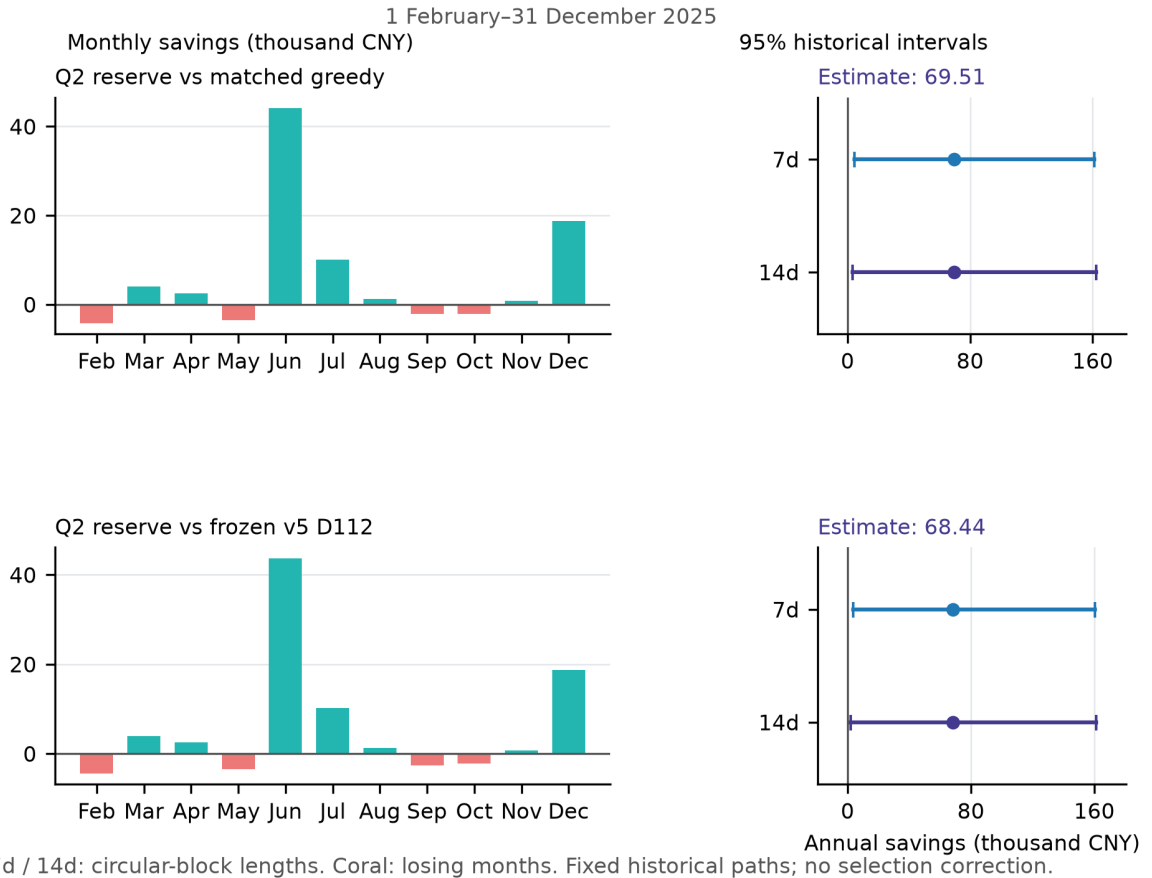


Figure 5: Q2 reserve-policy monthly savings and 7/14-day descriptive circular-block intervals against matched greedy and frozen D112 references. Negative months and any intervals crossing zero remain visible; the two references answer different comparison questions.

Figure 5 shows the seasonal variation behind the annual saving: against matched greedy, Q2 wins on 221 days and loses on 113. Its 7-day interval is [4,484.52, 160,892.55] CNY; the 14-day interval is [3,279.77, 162,427.06] CNY. These resample fixed historical cost differences and do not generate new weather or battery paths. Section 8 reports the same convention across branches.

The maximum 112-day window contains 30-112 completed historical days: it is shorter during the first 82 evaluation days. Earlier changes to window length, inventory weight, emergency weight, and search limits are retained in Table 14 under their original greedy workflow. They motivated inherited settings but do not establish sensitivity or optimal parameter values for the new reserve controller. No annual retuning was performed in this experiment.

For scale, the preserved full-information reference has value CNY 12,227,565.30 under the common initial energy and equipment limits, free terminal energy within its bounds, and ordinary-priced total external supply. Its access to the whole future and relaxed supply price make it an idealized lower reference, not an implementable midnight policy. The current Q2 gap is CNY 1,617,887.69; it is not promised attainable saving or an estimated value of perfect information.

6. Problem 3: Paid Rolling Procurement with Updated PV Forecasts

6.1. Rolling decisions and information comparisons

Problem 3 adds forecasts released at 00:00, 06:00, 12:00 and 18:00 and permits paid procurement revisions after midnight. We retain Q2's shared purchase/reserve objective, execution rule, and fixed parameters, using the current Attachment 3 forecast and the final-contract cost from Section 3. The load forecast is formed at midnight and remains fixed throughout the day. A PV release is interpolated onto the ten-minute grid and truncated at the current day's midnight boundary. Although each release extends 24 hours ahead, this inherited optimization horizon ends at 24:00 of the present day.

At release k , only (q_t^k, R_t^k) for $t \in \mathcal{T}_k$ may change. Both executed vectors are frozen, and every scenario starts at actual energy $E_{d,\tau_k}^{\text{act}}$, not a previously forecast state. Historical errors match the PV release stage used at the current decision. Thus a noon update uses historical noon-issued PV errors, while an energy-feedback-only control retains the remaining portion of historical midnight errors. Reserve feedback does not relax these information cutoffs.

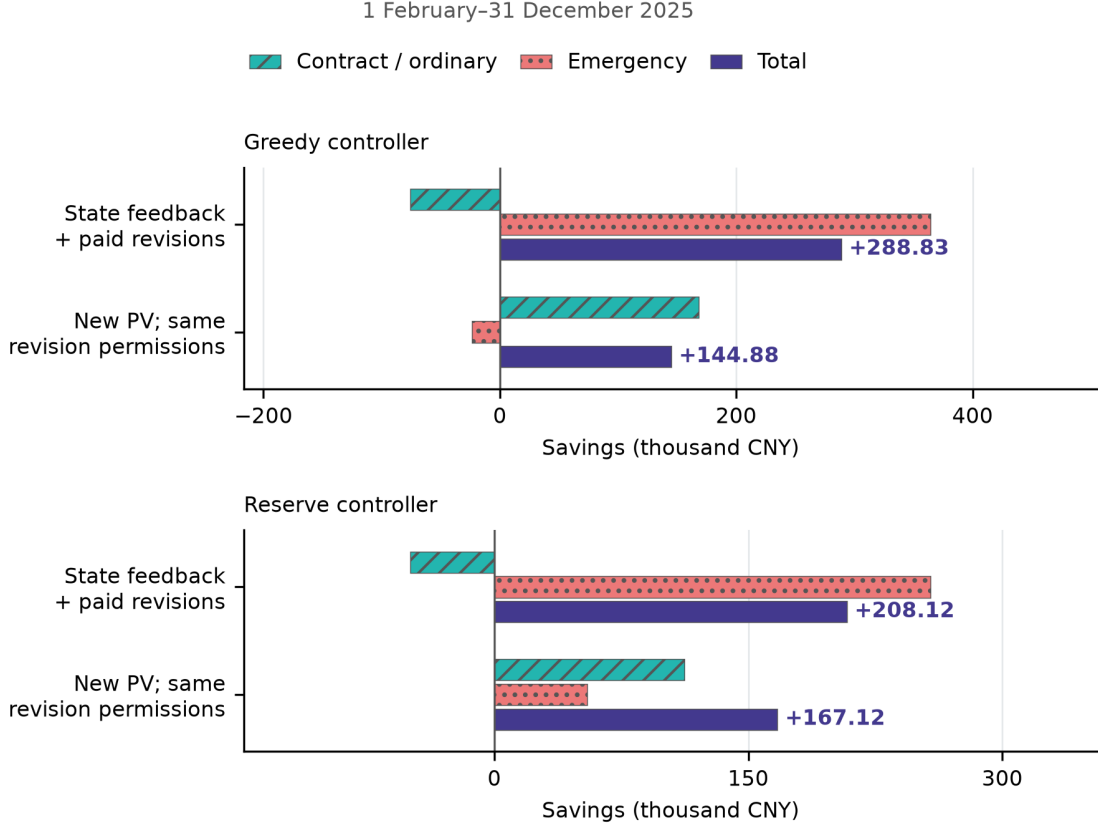
The contract term at midnight is $p_t q_t^0$. At later releases it becomes $F_A(q_t^k; q_t^0, p_t)$, always referenced to the original midnight quantity. Under this working settlement rule, an additional kWh above the original commitment costs $1.5p_t$, while a reduction below it reduces the original bill by only $0.5p_t$ per kWh. The candidate is evaluated with this remaining contract cost, scenario emergency cost and inventory proxy. Previously executed costs are sunk, and successive optimization objectives are never added to form the bill. Actual settlement applies F_A once to the final effective quantity $\bar{q}_t$, plus the full $5p_t u_t$ emergency charge.

These changes yield a rolling purchase/reserve policy. A remaining-horizon MILP uses the applicable contract objective to supply a risk-profile purchase start and nominal energy values for one reserve start. Its optimized dispatch is not the final execution policy. Final decisions impose no daily terminal-energy equality, and actual energy follows (3) continuously across days and months.

At midnight, four searches generate twelve candidate pairs and the selected pair becomes (q^0, R^0) . At 06:00, 12:00 and 18:00, the current pair of future purchase and reserve vectors is retained as candidate thirteen. Retaining it avoids forcing a search result with a worse current empirical score, but cannot assure a lower later realized bill. An opportunity may change purchases, thresholds, both, or neither; unchanged purchases alone do not mean the controller took no action.

Within each controller family, the no-update policy retains the midnight PV and both decisions all day. The state-feedback policy may revise future purchases and thresholds from actual energy but retains the midnight PV release. The raw-PV policy also uses subsequent forecast releases. All begin at the same February initial energy; later states develop independently as consequences of the policies. Contract cost includes the final adjustment against the original commitment and excludes emergencies.

Figure 6 separates the savings associated with state feedback and later PV releases. In the reserve family, state feedback with paid revisions saves CNY 208,115.60. Its contract saving is CNY -49,627.38 and emergency saving CNY 257,742.98. Adding later raw PV releases saves CNY 167,121.93, with contract and emergency savings of CNY 112,167.65 and 54,954.28. The combined raw-PV change saves CNY 375,237.53. Negative components denote higher expenditure.



Each increment is within one controller arm; every policy maintains its own continuous actual state.

Figure 6: Q3 information comparisons within each storage-controller family. Signed contract and emergency savings distinguish state feedback with revisions from the additional effect of later raw PV releases. Each comparison uses 334 matched dates.

The first difference combines observed-state feedback and revision permission. The second isolates the policy effect of later PV releases under the same permissions, including subsequent state changes. Neither estimates a universal price for information or the contribution of one particular release. Table 15 retains greedy controls so these effects are not inferred by comparing different storage rules.

6.2. Correction and the selected policy's negative result

Persistent forecast bias motivates testing a correction, but its value must be judged through the purchasing decision. We retain the existing 28-day correction as a candidate. The correction is fitted and applied to the interpolated ten-minute energy forecasts (kWh). For each release hour, it estimates the mean actual-minus-raw-PV error from eligible positive raw forecasts whose target intervals are within the previous 28 days and have ended by the current day's midnight. It uses only targets belonging to the historical release day. The same-day observations between midnight and a later release do not enter this correction fit. A positive raw forecast receives the estimated bias and is clipped at zero; an originally zero forecast remains zero.

The deployment rule activates the correction on April 1; February and March use raw PV. Historical scenario errors are reconstructed using each historical day's own deployable forecast and bias parameters. We do not apply today's correction retrospectively to older predictions. A window spanning April 1 therefore contains the original and corrected forecast versions actually prescribed on their respective dates. This preserves the deployment record while leaving a documented nonstationarity in the residual pool.

Table 8: Q3 PV correction in each matched controller, 334 days (CNY).

Controller	Raw PV	Correction from April 1	Saving from correction
Greedy	13,559,330.38	13,568,255.74	-8,925.36
Reserve	13,574,997.43	13,574,461.81	535.62

Q3 correction saves CNY 535.62 in the reserve arm and costs an additional CNY 8,925.36 in the greedy arm. This reversal ties the correction's value to the full purchasing and storage procedure. The earlier nominal-MILP correction study uses a different purchasing method (Appendix B).

The reserve correction wins on 121 days, loses on 154, and ties on 59. Its 7-day interval is [-25,199.40, 28,640.66] CNY and its 14-day interval [-26,491.00, 29,134.66] CNY. Both cross zero. The corrected choice follows unrounded historical cash ranking, without establishing a stable correction benefit; these are correction comparisons, distinct from the controller comparisons in Section 8.

The submission retains reserve-family selection as its declared study scope, with historical cash cost ranking the PV variants within that family. Greedy policies remain economic controls in the full comparison. Under this restriction, the submitted Q3 schedule is not the least-cost policy across both families.

The reserve-family selection is `q3_w28_reserve`, the corrected-PV rolling reserve policy. Original commitments cost CNY 13,276,739.67; upward and downward terms are CNY 208,355.49 and -250,200.37. Together, these terms give a contract bill of CNY 13,234,894.79. Adding CNY 339,567.02 of emergencies gives CNY 13,574,461.81. The final effective quantities are settled once, rather than summing all issued versions.

Against the same input configuration with matched greedy control, this policy costs an additional CNY 6,206.07. Thus the Q3 comparison provides no realized cash advantage for the added reserve control. Possible contributors include finite search, differences between historical scenarios and realized conditions, and their effects on later storage states; their separate contributions were not isolated. Table 15 reports all eight Q3 outcomes.

The selected Q3 schedule is detailed in Appendix D.3, Tables 25-28, and `result3.xlsx`, including its original and revised purchases.

7. Problem 4: Planning Prices and Realized Variable-Price Settlement

7.1. Price information without advance knowledge of actual tariffs

Attachment 4 records realized prices but does not state that a complete future price path is available when purchases are decided. We therefore compare two implementable planning inputs: the known fixed tariff from Attachment 1 and a causal price forecast. In both cases, the actual bill uses Attachment 4 prices for the target intervals. Using a fixed reference in the optimizer defines a purchasing rule under uncertain prices; settlement still uses the actual tariff.

The causal forecast uses ordinary least squares with a trailing 28-calendar-day window. Its predictors are an intercept, two daily Fourier harmonics, one-day and seven-day price lags, and six weekday indicators. Writing ϕ_t for the fraction of the day elapsed at interval t , the planning price is

$$\begin{aligned}
\tilde{p}_{d,t|k} = & \beta_0 + \sum_{j=1}^2 \left[a_j \sin(2\pi j \phi_t) + b_j \cos(2\pi j \phi_t) \right] \\
& + \beta_1 p_{d-1,t}^{\text{act}} + \beta_7 p_{d-7,t}^{\text{act}} + \sum_{r=1}^6 \gamma_r \mathbf{1}\{\text{weekday}(d) = r\}, \\
p_{d,t|k}^{\text{plan}} = & \max\{10^{-6}, \tilde{p}_{d,t|k}\}.
\end{aligned} \tag{10}$$

Coefficients are re-estimated at each allowed decision time using only completed price observations. Equivalently, with the 13-column design matrix X and observed training-price vector y at the current issue, the fit minimizes $\|X\theta - y\|_2^2$ over $\theta \in \mathbb{R}^{13}$; it adds no procurement constraint. Training rows must also have their lagged predictors available. Since the prediction horizon ends at the current day's 24:00, its previous-day and previous-week lags are already observed. At an intraday decision, completed current-day prices may enter the training window, while uncompleted and future target prices cannot. The floor of 10^{-6} CNY/kWh is a fixed numerical positivity rule, not a minimum estimated from the evaluation period. This regression supplies decision inputs; correlated ten-minute observations are not treated as independent samples for coefficient-significance claims.

We interpret recalculating Problem 2 under variable prices as retaining its historical-PV information set. The statement lists Attachments 2-4 for Problem 4 without an explicit branch-specific ban on Attachment 3; the Q4-2 boundary in Table 3 is therefore a working interpretation. Under it, Q4-2 uses B0 load and historical-PV forecasts, excludes Attachment 3, and fixes both its 144 purchases and 144 reserve thresholds at midnight. Q4-3 uses the available Attachment 3 releases and actual energy feedback, with future purchase/reserve updates at 06:00, 12:00 and 18:00. Both inherit $W = 112$, $\kappa = 5$, $\mu = 0.38232$, the four-search protocol, and physical parameters without retuning. The planning price enters both ordinary contract cost and scenario emergency cost in (5), so it can alter procurement and the timing of energy retention. Changing only the ordinary-price term while keeping emergency weights at fixed tariffs would not implement this comparison.

During execution, p^{act} determines cash settlement for normal purchases, contract adjustments, and emergencies. It does not replace a preissued threshold using information that was unavailable when that threshold was chosen. The inherited inventory coefficient remains a planning proxy and is never deducted from the bill. Planning prices influence the selected (q, R) pair; realized prices determine its cash cost.

7.2. Price comparisons and selected policies

Table 15 compares fixed-reference and causal-OLS planning prices within each controller. Actual settlement, information access, physical limits, initialization, and search settings are held fixed. This comparison does not add Attachment 3 to the midnight branch.

OLS saves CNY 3,079.27 in the reserve family and costs an additional CNY 6,786.91 in the matched greedy family. The selected reserve policy is `q42_ols_reserve`. Its total CNY 14,618,518.98 comprises CNY 13,704,872.50 of ordinary fees and CNY 913,646.48 of emergencies, all at actual variable prices. The selected policy saves CNY 68,043.65 relative to its same-input greedy control.

For OLS versus fixed planning prices within the reserve family, the 7-day interval is [-31,696.16, 32,765.14] CNY and the 14-day interval [-32,148.32, 34,023.98] CNY. Both cross zero. They describe the price-input ranking and are not replaced by the reserve-versus-greedy intervals in Table 10.

For each controller, Q2 and the fixed-planning-price Q4-2 policy produce identical purchases, thresholds, and physical paths. Their cash bills differ because Q4-2 uses Attachment 4 actual prices. The independent comparison checks this identity for all 48,096 intervals and both arms; a different physical path here would indicate an unintended planning change.

Appendix D.4, Tables 29-32, and `result4-2.xlsx` contain the prescribed-date outputs for the selected Q4-2 policy.

Figure 7 compares the price and PV-correction choices. With raw PV, causal OLS saves CNY 28,892.88 in the reserve arm. Under OLS, PV correction costs an additional CNY 4,984.88; under fixed planning prices it saves CNY 230.32. The selected reserve policy is `q43_raw_ols_reserve`, the raw-PV/causal-OLS rolling reserve policy. It saves CNY 7,194.43 against the matched greedy configuration. Table 15 preserves all eight outcomes so price effects are not inferred from differently corrected PV inputs or controllers.

The raw-PV OLS-versus-fixed comparison has positive descriptive intervals: [14,140.65, 44,563.84] CNY with 7-day blocks and [11,797.92, 46,451.05] CNY with 14-day blocks. In contrast, both block lengths cross zero for each reserve-family PV-correction comparison, under fixed and OLS planning prices. These intervals distinguish the observed price effect from the smaller correction rankings; Section 8 defines their resampling scope.

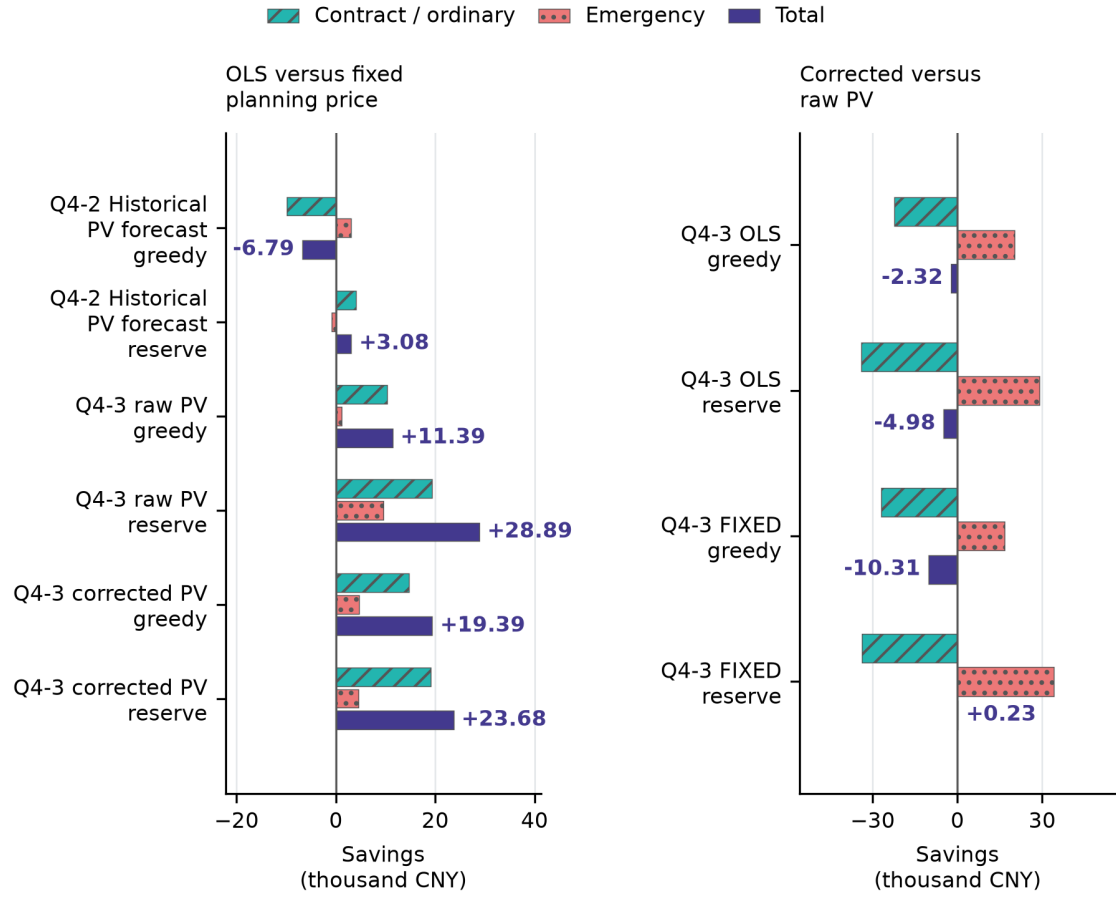
The corresponding Q4-3 outputs, including paid revisions and emergency events, are in Appendix D.5, Tables 33-36, and `result4-3.xlsx`.

Figure 8 uses the prespecified diagnostic date, 14 July 2025, to show how released planning prices and procurement versions lead to the executed energy and reserve trace. Each version starts at its release and leaves completed intervals fixed. The actual-price trace provides retrospective settlement context.

Both Q4 branches carry their own actual state continuously. Q4-2 has midnight historical-PV decisions; Q4-3 has released PV, paid revisions, and state feedback. Their absolute bill difference therefore combines several changes and is not a pure estimate of PV information value. No new Q4 no-update or state-only controls were added. The supported price comparisons are the within-controller, within-PV pairs in Table 15.

The four selected reserve policies are also compared with their frozen v5 predecessors in Section 8. Those references retain their original two-search budget and historical selection, so they show the effect of replacing the previous selections. The four-search greedy arm provides the more closely matched controller comparison.

1 February–31 December 2025



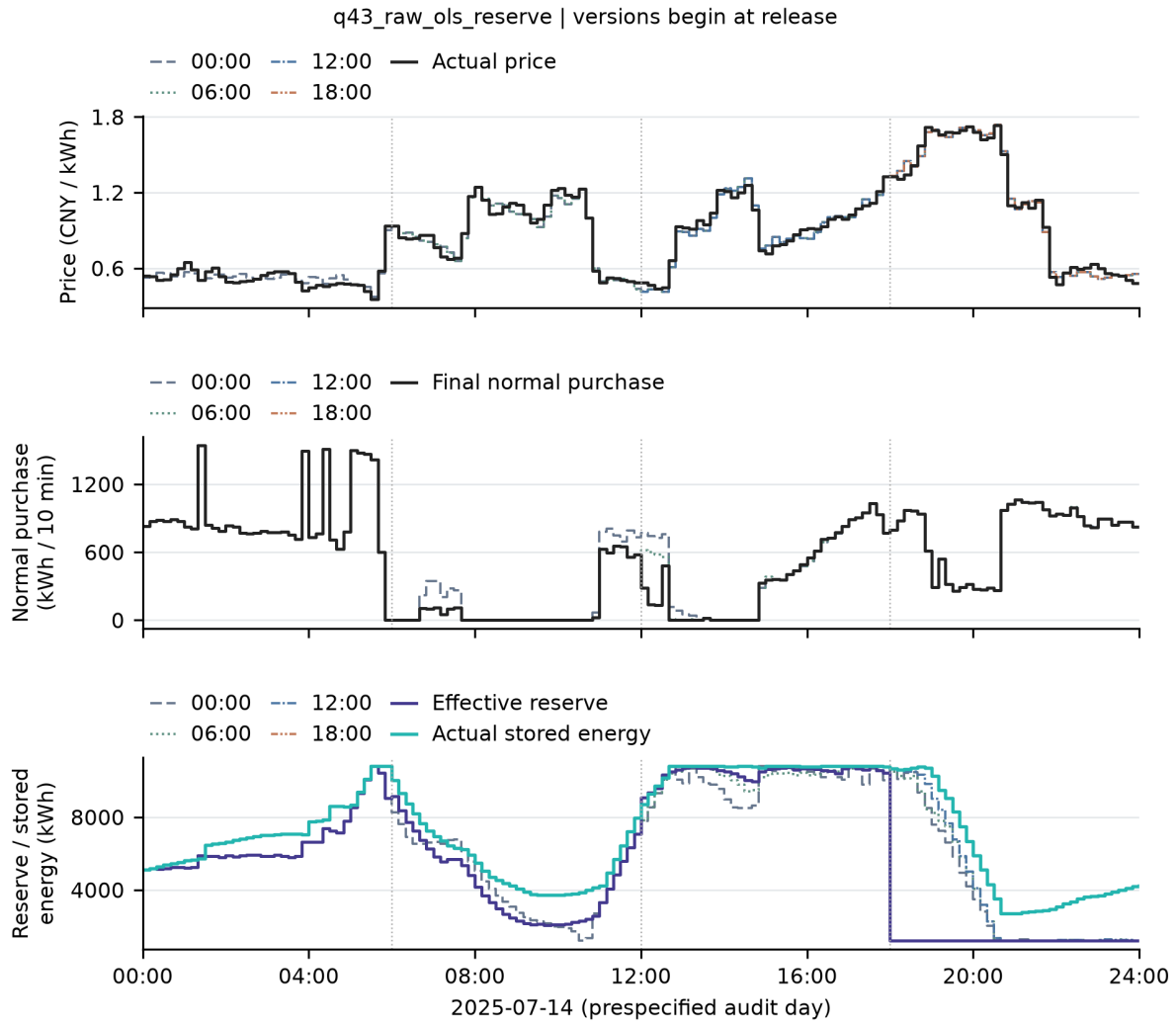
All Q4 bills use actual variable prices. Fixed prices are planning inputs. Savings = reference – candidate.

Figure 7: Q4 planning-price and PV-correction comparisons under matched storage controllers. Q4-2 retains historical PV; Q4-3 uses the permitted forecast releases. Every cost is billed at actual variable prices, regardless of its planning-price input.

8. Correctness, sensitivity, and historical uncertainty

8.1. Implementation, information cutoffs, and reproduction

Independent verification rebuilt the source inputs and refitted historical forecasts at their own information cutoffs, using implementations separate from the production forecast, execution, billing, and objective functions. It replayed all twenty-two annual paths and checked candidate scores and saved workbooks. Table 9 reports the numerical discrepancies and tolerances; Appendix A records the coverage and reproduction details.



Original 10-minute intervals through 24:00. Actual prices are used for retrospective billing only.

Figure 8: Selected Q4-3 policy on 14 July 2025: released planning prices, purchase versions, and storage energy with executed reserve thresholds. Future versions begin only at 00:00, 06:00, 12:00, or 18:00. Actual prices are retrospective settlement data, and final normal purchases exclude emergencies.

Table 9: Recorded v6 verification evidence and tolerances.

Object	Scope of comparison	Maximum absolute discrepancy	Threshold
Physical and forecast energy	Raw source conversion, continuous execution, nominal initializer, and candidate storage paths	2.4920×10^{-9} kWh	10^{-6} kWh
Cash and candidate scores	Full contract, fivefold emergencies, and scenario scores	1.8626×10^{-9} CNY	10^{-5} CNY
Saved workbooks: energy	All five actual XLSX files, specified dates, and physical time keys	8.7311×10^{-11} kWh	10^{-6} kWh
Saved workbooks: fees	Actual saved quantities rebilled from original prices	1.1548×10^{-7} CNY	10^{-5} CNY
Joint derivatives away from kinks	720 purchase/reserve coordinates at horizons 36, 72, 108, and 144; step 0.001 kWh	5.0546×10^{-8} CNY/kWh	10^{-5} CNY/kWh

The workbook checks read the saved XLSX files directly and reconciled each variable-price emergency event with its independently checked ten-minute ledger allocation. Table 9 reports the energy and fee discrepancies for these saved quantities.

The motivating two-interval test fixes zero normal purchases, two 100 kWh deficits, initial energy 1,311.11111 kWh, and tariffs 0.4248/1.162 CNY/kWh. Greedy discharge costs CNY 581 in emergencies; retaining the battery for the second deficit costs CNY 212.4, with both paths ending at 1,200 kWh. The code reproduces that admissible counterexample. This refutes compulsory-discharge optimality, but the CNY 368.6 difference is not an estimate of annual loss. Boundary cases, scalar/vector recursion comparisons, finite differences away from kinks, and a separate core review check the new mechanism. At a kink the implemented branch derivative is not asserted to be a unique classical derivative.

Independent score accumulation gave a different first qualifying index in 2 numerically tied decisions. Each saved choice obeyed the original 10^{-8} CNY first-index rule on its saved scores. Recomputed scores stayed within the cash tolerance, and accepted index differences corresponded to purchase and reserve vectors agreeing within 10^{-6} kWh. The verification record lists the score gaps.

Twenty-nine information tests used six preselected decision fixtures on 15 May: eighteen interventions on unavailable information, nine positive controls using permitted inputs, and two PV-access guards. Changing future demand/PV, unreleased PV forecasts, or future actual prices left the available inputs, full purchase/reserve candidates, scores, and selected pairs unchanged. Permitted PV, completed price history, and actual-energy changes produced the expected responses. The tests disabled price caches and kept the B0 archive fixed; its historical reconstruction was checked separately. The Q4-2 guard enforced the working interpretation in Table 3.

An isolated reproduction of the selected reserve policies rebuilt the kernel and reproduced the saved trajectories and candidate/input arrays exactly. It used the same host and dependency versions; Appendix A specifies the comparisons and exclusions.

Of 85,504 local searches across twenty-two annual policies, 46,365 ended with non-success stopping records. The logs contain 0 solver exceptions, 0 initializer fallbacks, and 0 invalid-return fallbacks. Selection retained a starting candidate 12 times and the complete keep-current pair 3,047 times. Every finite retained pair was rescored before selection and executed through the hard physical rule. Independent rebilling of the saved ledgers, using separately validated prices, also recomputed all twenty-two summaries and twenty-nine named paired comparisons.

Figure 9 compares midnight forecasts with the latest release permitted by the 00:00/06:00/12:00/18:00 protocol on the selected raw-PV/OLS Q4-3 reserve path. The PV panels share 26,880 intervals with positive actual generation. This subset is chosen retrospectively for evaluation and excludes 21,216 zero-generation intervals, including 1,280 positive forecasts in each version. It leaves forecast-time inputs unchanged. PV MAE falls from 60.77 to 31.83 kWh per ten-minute interval on the subset and from 34.44 to 18.27 kWh over all 48,096 intervals. Price MAE changes from 0.04270 to 0.04240 CNY/kWh. These error summaries cover one retained historical path; Table 15 gives the matched policy comparisons used to assess operating costs.

8.2. Paired savings, losing periods, and terminal inventory

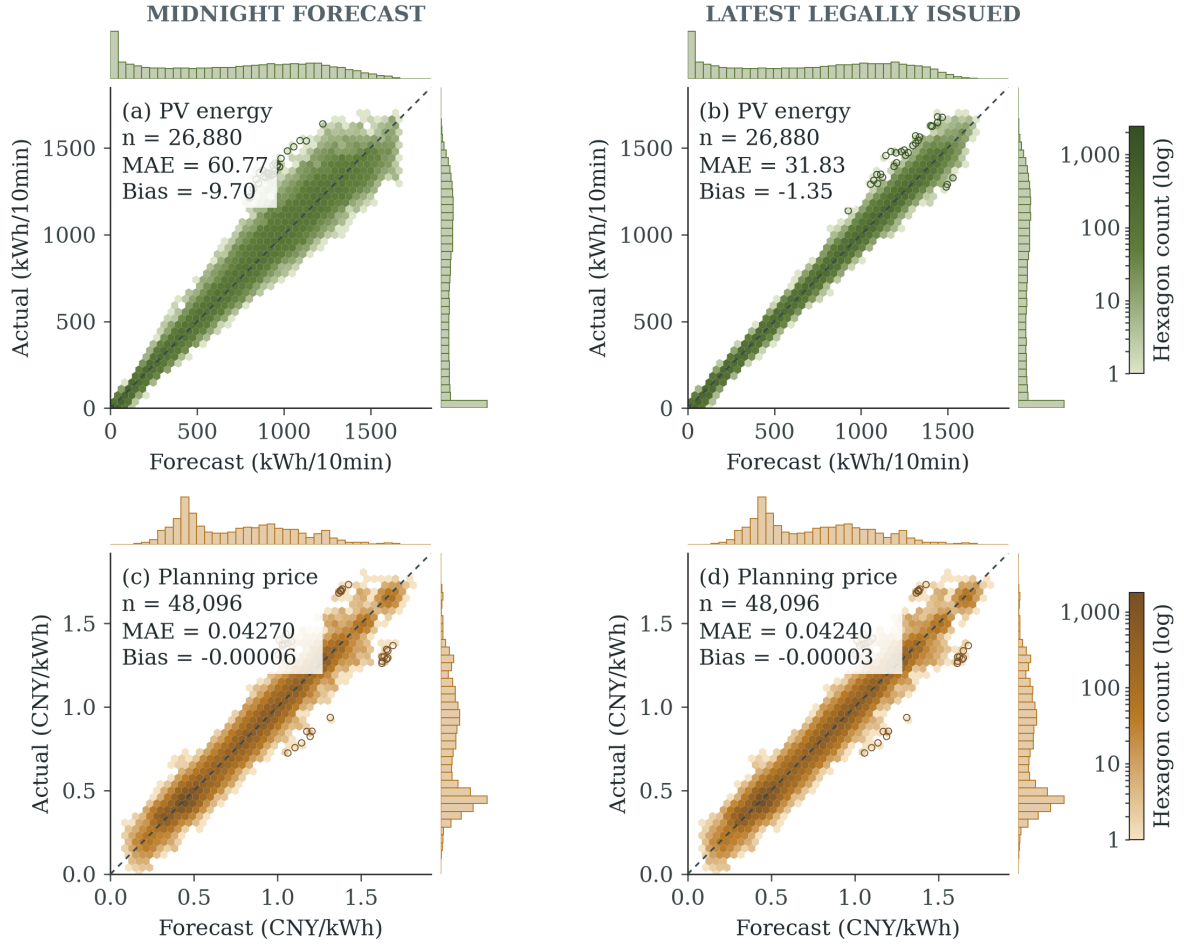
For candidate a and reference b , daily saving is $S_d = B_d^b - B_d^a$. Positive values favor the candidate; monthly and annual savings sum these same matched daily differences. The denominator of a saving percentage is the named reference's total cash bill. Each selected reserve policy is paired below with the same input configuration in the four-search greedy arm.

Figure 10 shows why winning on many days need not reduce the annual bill: the selected Q3 reserve policy saves money on 188 of 334 days but costs CNY 6,206.07 more over the full period. The plotted observations retain both tails; Table 10 gives the annual block intervals.

Table 10: Selected reserve policies versus same-configuration greedy controls, 334 days; CNY. Descriptive 95% intervals use 2,000 circular-block resamples.

Branch	Matched greedy reference	Cash saving	7-day interval	14-day interval
Q2	q2_greedy	69,508.07	[4,484.52, 160,892.55]	[3,279.77, 162,427.06]
Q3	q3_w28_greedy	-6,206.07	[-26,367.00, 14,137.80]	[-29,281.12, 16,720.13]
Q4-2	q42_ols_greedy	68,043.65	[1,453.28, 155,787.03]	[-3,135.99, 158,176.86]
Q4-3	q43_raw_ols_greedy	7,194.43	[-14,347.68, 30,167.31]	[-18,423.91, 33,626.59]

Both Q2 block intervals are positive. Q4-2's 14-day interval crosses zero, and both intervals cross zero for Q3 and Q4-3. Every selected reserve policy has losing months against its matched greedy control; favorable annual totals do not imply uniform benefit.



Q4-3 raw-PV + OLS reserve policy | 1 Feb-31 Dec 2025 | Bias = forecast – actual
 PV: actual > 0 only; 21,216 zero intervals omitted. Price: all 48,096 intervals.
 Dashed: 1:1. Circles: 30 largest errors per panel. Actuals used only for evaluation.

Figure 9: Forecast-actual distributions for the selected raw-PV/OLS Q4-3 reserve policy, February-December 2025. PV panels use the same actual-positive subset (26,880 intervals); price panels use all 48,096 intervals. Forecast versions are aligned by date, interval, and permitted issue time. Hexagons show counts on a shared logarithmic scale within each row; marginals show counts, dashed lines show equality, and circles mark the 30 largest absolute errors per panel. Bias is forecast minus actual. Actual values are retrospective evaluation outcomes.

Figure 11 extends the monthly comparison to all eleven matched configurations. Color is the monthly cash saving as a percentage of that configuration's greedy bill, so it is not an absolute-cost ranking. Rows are ordered by average-linkage Euclidean clustering of their standardized monthly percentage profiles; the displayed colors remain unstandardized and months remain chronological. The side tracks show annual cash saving and additional emergency energy, making the cost and emergency-use tradeoff visible. These are comparisons of the complete procurement-and-storage procedures, not isolated effects of changing a reserve threshold while holding procurement fixed.

Table 11 uses the same four pairs to show endpoint energy and an auxiliary inventory adjustment:

$$S_{\text{inventory}} = \sum_d S_d + \nu(E_{\text{end}}^a - E_{\text{end}}^b). \quad (11)$$

The diagnostic rate is $\nu = \eta_d \bar{p} = 0.9 \times 0.7661972222 \dots = 0.6895775$ CNY/kWh, where $\bar{p}$ is the arithmetic mean of the 144 Attachment 1 tariff entries. It values one internal kWh by its discharge efficiency at the mean fixed tariff and was inherited as a common endpoint diagnostic. It is separate from the planning coefficient $\mu = 0.38232$, is not fitted to the selected v6 cost savings, and is neither a sale tariff nor cash income. An endpoint adjustment does not reoptimize trajectories or remove losses earlier in the year.

Daily wins can hide an annual loss

Reserve vs matched greedy / 1 February–31 December 2025 / 334 days per policy

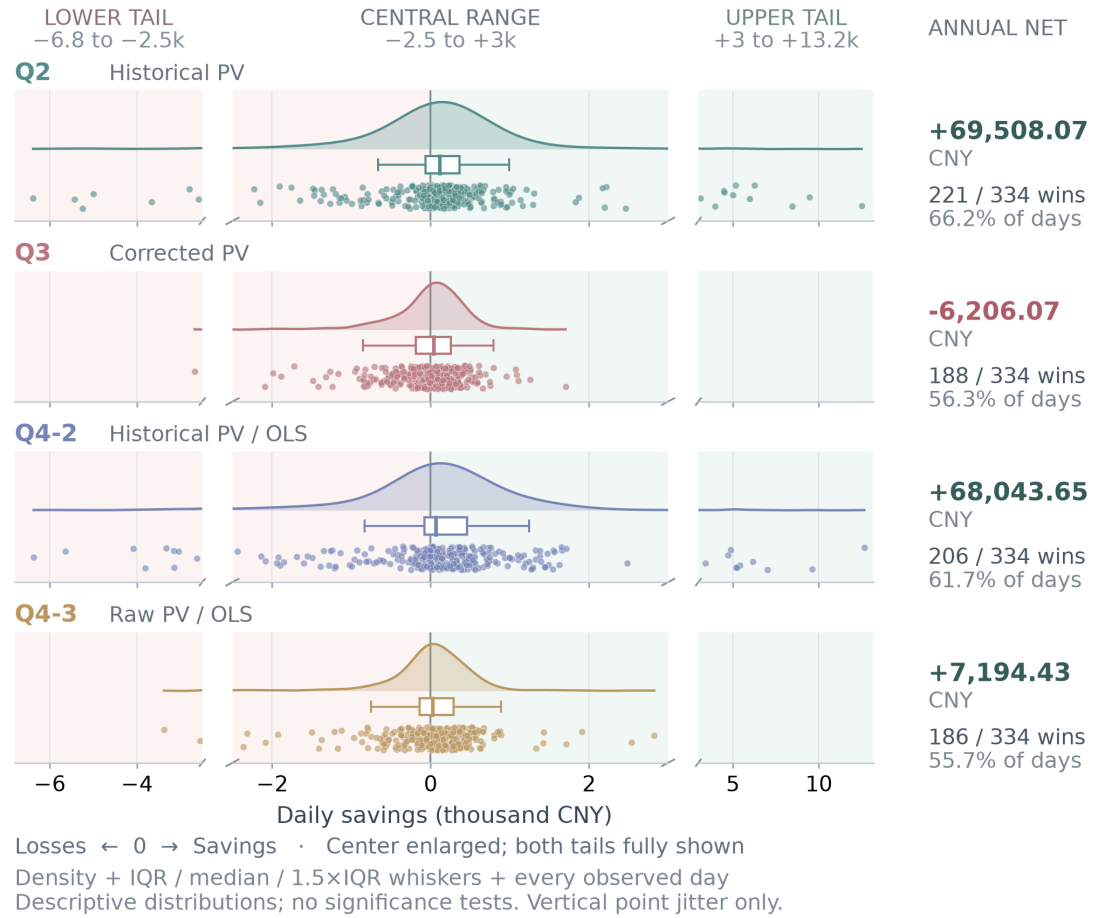


Figure 10: Daily cash savings relative to each selected policy's same-configuration four-search greedy control, February–December 2025. Each row contains all 334 days. Positive values favor reserve. Three adjoining horizontal ranges retain both tails while enlarging the center; their scales differ. Half-violins use Scott-bandwidth density estimates normalized to each row's peak and show shape only; plotted areas and heights are not probability comparisons. Boxes show the median and interquartile range, with 1.5-IQR whiskers; dots are original daily values with vertical jitter. Annual net savings and winning-day counts use the full rows.

Table 11: Terminal-inventory audit for the same matched comparisons; energy in kWh and savings in CNY.

Branch	Reserve final E	Greedy final E	Cash saving	Inventory-adjusted
Q2	3,179.368744	3,146.715222	69,508.07	69,530.59
Q3	2,438.521203	2,353.164464	-6,206.07	-6,147.21
Q4-2	2,989.756760	3,280.866789	68,043.65	67,842.90
Q4-3	2,514.150453	2,397.432892	7,194.43	7,274.91

Against the frozen two-search v5 selections, the new selected reserve policies have signed cash savings of CNY 68,439.49 (Q2), -12,628.36 (Q3), 60,020.48 (Q4-2), and 8,221.27 (Q4-3). Negative saving denotes increased expenditure. These combine controller, search-budget, and any input-selection changes, so they are not substituted for the matched controller comparisons.

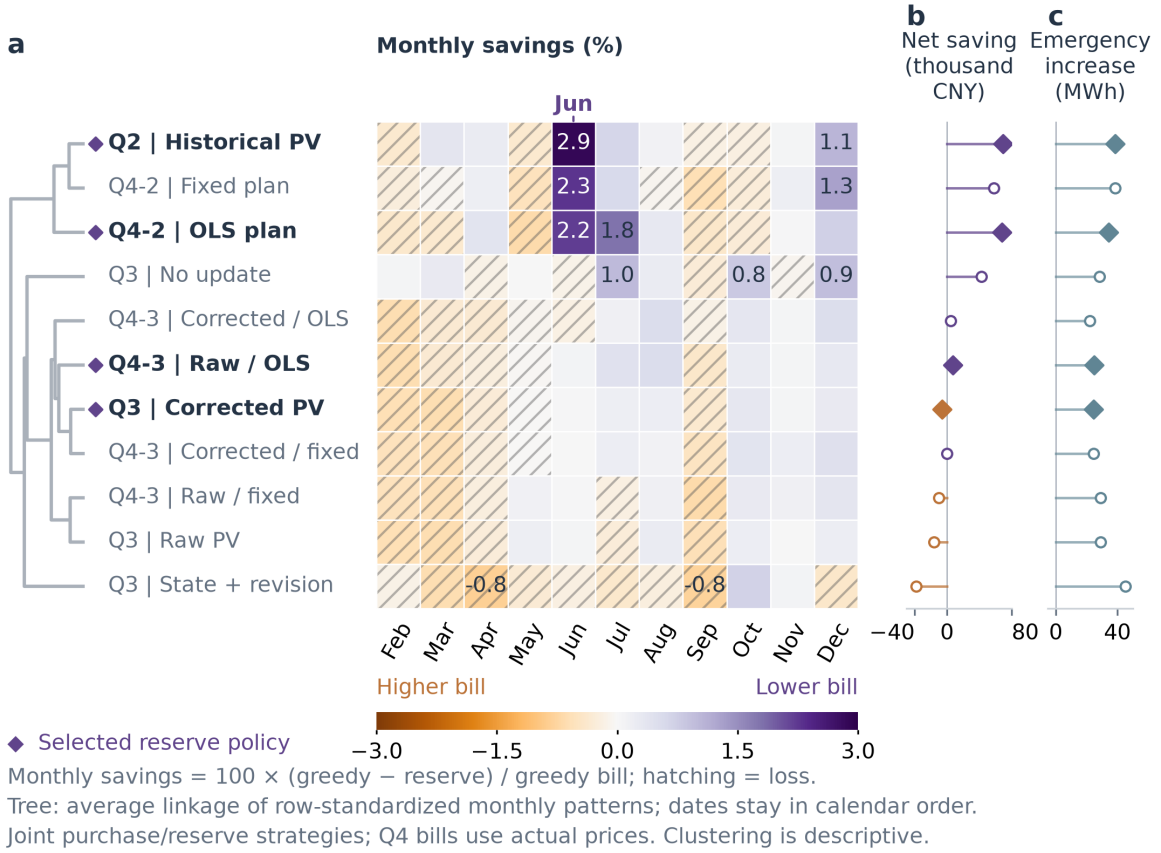


Figure 11: Seasonal savings profiles across all eleven reserve/greedy pairs, February-December 2025. Heatmap values are 100 times the monthly greedy-minus-reserve cash difference divided by the matched greedy monthly bill. Standardization affects the descriptive tree and row order only. Side tracks show annual net saving (thousand CNY) and reserve-minus-greedy emergency energy (MWh); diamonds mark the four selected reserve policies. Q4 fixed/OLS labels identify planning prices, while bills use actual prices. Hatching marks negative monthly savings.

Circular block resampling retains neighboring daily differences and wraps at the endpoints [9]. Both block lengths use 2,000 resamples with seed 20260915. The procedure resamples fixed historical cost differences: it neither reruns the battery on new weather nor refits forecasts, and it leaves repeated 2025 selection uncorrected. Its intervals therefore provide no selection-adjusted inference for unseen years or causal effects. The density displays and descriptive clustering summarize observed variation without significance tests. Scenario ordering was preserved; a shuffled-scenario comparison was not performed.

Q1's efficiency-by-power-side cases address the supplied physical ambiguity. The earlier Q2 history-window, cost-weight, and optimizer-budget cases concern the original greedy workflow. The current twenty-two-policy matrix fixes those inherited parameters and compares storage control, information permissions, planning prices, and PV correction. Earlier 275-day reserve/update-gate and correction cases remain explicitly historical in Appendix B. None establishes robustness of the new controller to different capacities, degradation, sustained forecast bias, or unseen years.

9. Evaluation and final operating policies

The model chooses when to retain battery energy together with normal procurement. The same reserve rule evaluates historical scenarios and executes actual intervals, preserving physical limits and the full fivefold emergency price. Paid procurement updates retain the whole current purchase/reserve pair and freeze executed quan-

ties. Planning prices affect decisions; actual prices determine cash settlement. Together, these rules allow the controller to retain energy for later deficits instead of requiring immediate discharge.

Table 12: Five final policies: cash fees in CNY and final stored energy in kWh. Q1 is one day; annual branches cover 334 days.

Branch	Decision / PV input	Planning price	Total fee	Emergency fee	Final energy
Q1	Midnight; deterministic	Fixed	35,126.95	0.00	6,000.000000
Q2	Midnight; historical PV; reserve	Fixed reference	13,845,452.99	862,155.13	3,179.368744
Q3	00/06/12/18; corrected PV; reserve	Fixed reference	13,574,461.81	339,567.02	2,438.521203
Q4-2	Midnight; historical PV; reserve	Causal OLS	14,618,518.98	913,646.48	2,989.756760
Q4-3	00/06/12/18; raw PV; reserve	Causal OLS	14,313,637.10	389,707.68	2,514.150453

Table 12 reports the lowest unrounded historical cash cost within each reserve branch, with emergency fee and declared order breaking exact ties. Q2 saves CNY 69,508.07; Q3 costs an additional CNY 6,206.07; Q4-2 saves CNY 68,043.65; Q4-3 saves CNY 7,194.43. The full matrix and the losing months are retained. The five workbooks provide these selected reserve-family schedules, with the unchanged deterministic Q1 schedule.

Interval labels, efficiency and power measurement, interpolation, balancing, and rule-A settlement follow working conventions; official clarification or metering specifications would be needed before recalculating under different conventions. Q4-2's exclusion of Attachment 3 is also an author interpretation. Surplus charging and preissued reserve thresholds restrict the control law, and finite nonsmooth search supplies no global certificate. Richer causal controllers and independently bounded solutions should therefore be compared on the same information set. The fixed terminal proxy omits degradation and network dynamics; estimating these costs and testing a longer horizon require measured battery-aging and network data. Repeated selection on one historical year limits evidence for future savings. Untouched periods, prespecified comparisons, and matched tests of history windows, weights, capacity, and sustained forecast bias remain necessary. None of these additional tests has been performed for the current reserve controller.

For transfer to another microgrid, replace the aligned demand, PV and tariff inputs; set local capacity, bus-side power, efficiency and initial energy; and rewrite information cutoffs and contract charges to match the local market. Refit forecasts using only available local history and revalidate physical balances and settlement before evaluating economics. This is a transfer procedure, not evidence that the present numerical rankings persist.

AI tool usage declaration

The team used AI tools to help interpret the problem and its ambiguities, discuss model formulation and implementation, generate and debug code, execute computational experiments, check and analyze results, produce figures, organize and draft the manuscript, edit the language, and check references. The team selected the initial models and reported reviewing the first MILP experiment and each subsequent task, including manual changes to an initially conservative Q2 implementation. Final adoption and required manual verification remain the team's responsibility. Detailed use is reported in the supplementary Details of AI Tool Usage PDF.

References

- [1] CUMCM Organizing Committee. *2026 CUMCM Problem C: Microgrid and External-Grid Power Regulation Strategies*, with Attachments 1-5. Supplied competition statement and data, 2026.
- [2] Sigalo, M. B., Pillai, A. C., Das, S., and Abusara, M. An Energy Management System for the Control of Battery Storage in a Grid-Connected Microgrid Using Mixed Integer Linear Programming. *Energies*, 14(19), 6212, 2021. doi : 10.3390/en14196212.
- [3] Parisio, A., Rikos, E., and Glielmo, L. Stochastic model predictive control for economic/environmental operation management of microgrids: An experimental case study. *Journal of Process Control*, 43, 24-37, 2016. doi : 10.1016/j.jprocont.2016.04.008.
- [4] Hyndman, R. J., and Athanasopoulos, G. *Forecasting: Principles and Practice*, 3rd ed. OTexts, 2021. Section 5.10, Time series cross-validation, accessed 11 September 2026. Chapter.
- [5] Kleywegt, A. J., Shapiro, A., and Homem-de-Mello, T. The Sample Average Approximation Method for Stochastic Discrete Optimization. *SIAM Journal on Optimization*, 12(2), 479-502, 2002. doi : 10.1137/S1052623499363220.
- [6] Byrd, R. H., Lu, P., Nocedal, J., and Zhu, C. A Limited Memory Algorithm for Bound Constrained Optimization. *SIAM Journal on Scientific Computing*, 16(5), 1190-1208, 1995. doi : 10.1137/0916069.
- [7] SciPy developers. *minimize(method='L-BFGS-B')*. Official API documentation, accessed 11 September 2026. Method documentation.
- [8] HiGHS developers. *HiGHS: Linear Optimization Software*. Official project and documentation, accessed 11 September 2026. HiGHS.
- [9] Sheppard, K. *arch: Time-series Bootstraps*. Official software documentation, accessed 11 September 2026. Circular and other time-series bootstraps.

Appendix A. Implementation and reproducibility specification

Appendix E contains the complete scientific source listings and the exact support-file inventory. In the accompanying support archive, README.md gives the environment, original-input placement, lightweight checks, and optional reproduction commands; MANIFEST.csv records packaged identities. The scientific entry points are scripts/solve_q1.py and scripts/storage_control_v6/run.py. The manuscript builder only typesets completed evidence. The twenty-two annual experiments and their isolated reproduction were performed before this editorial revision.

Paths in Table 13 are relative to the scientific support directory unless explicitly labeled as original project provenance. Original contest attachments are not duplicated in the archive: raw/required_files.json identifies them, and raw/README.md specifies neutral input filenames. SOURCE_PROVENANCE.json distinguishes original sources, portable copies and saved evidence; source_changes.diff records path and logging adaptations. The historical studies remain named references and are not represented as newly executed results.

Table 13: Scientific program, saved-result, and provenance interfaces.

Model or evidence	Relative entry	Role
Raw inputs and conversion	raw/README.md, raw/required_files.json; scripts/prepare_data.py, scripts/prepare_inputs.py; configs/data_processing.json	Required original-input identities, key alignment and forecast conversion
Q1 schedule and diagnostics	scripts/solve_q1.py; saved/q1/	Complete deterministic model, saved baseline, certificate and named physical cases
Forecast family and common January state	scripts/run_q2.py; scripts/storage_control_v6/compat.py; saved/warmup/	Original forecasting and initialization logic; actual common boundary record
Current annual model	scripts/storage_control_v6/: inputs.py, policy.py, control.py, kernel.py, kernel.c, run.py, compat.py	Causal inputs, shared candidates, reserve execution and settlement
Current configurations and analysis	configs/storage_control_v6/experiments.json; scripts/storage_control_v6/analyze.py; saved/comparisons/	All controller configurations, restricted selection and named comparisons
Selected issued and executed decisions	saved/selected/<policy>/: execution_plans.npz, plan_versions.npz, ledger_four_dates.csv, plan_versions_four_dates.csv	Lossless annual purchase/reserve arrays and complete prescribed-date records
All current outcomes	saved/all_policies/	Every current daily summary, configuration and total, including adverse alternatives
Frozen comparison references	saved/historical_references/; results/q2_direct_v4/runs/D112/; results/unified_direct_v5/runs/	Original reference summaries and date/cash/state projections for analysis
Historical development provenance	Original project: results/q2_direct_v4/analysis_all/comparison_all.csv, results/innovation/evaluation/, results/robustness/	Retained sources of Tables 14, 16 and 17; separate historical methods and periods
Independent saved-result checks	scripts/check_saved.py, scripts/smoke_test.py	Workbook readback, physical/cash replay and bounded execution checks; no annual optimization in lightweight commands
Final workbooks	workbooks/: result1.xlsx, result2.xlsx, result3.xlsx, result4-2.xlsx, result4-3.xlsx, policy_mapping.json	Unchanged English results and original template/policy mapping
Environment and identities	requirements.txt, MANIFEST.csv, SOURCE_PROVENANCE.json, source_changes.diff	Dependencies, actual archive inventory and traceable source adaptations

The candidate order is profile start/best/return, point start/best/return, additional profile start/best/return, and additional best-greedy start/best/return. Intraday, keep_current is last and retains both future vectors. The first candidate within 10^{-8} CNY of the rescored minimum is selected. The nominal initializer constrains only its own

trajectory; its end-of-interval energy values initialize one reserve search. Section 3.3 gives all search limits and scaling. There is no random scenario subsampling.

The independent checker in Section 8 rebuilds raw inputs, historical forecast parameters, and scenario scores from saved candidates using implementations separate from the production forecast, execution, billing, and objective functions. The isolated reproduction reruns the production code in a separate source/data tree and recompiles the kernel. Both studies predate this editorial revision. The bundled `check_saved.py` supports saved-result checks, with default and optional full-input replay modes documented in the support README.

Verification coverage comprised the original input workbooks, 364 historical B0 forecast days, 1,460 raw PV releases, and 1,336 causal price releases. Across all annual paths, the checks covered 1,058,112 actual intervals, 21,376 decisions, 270,540 retained candidate pairs, and 27,544,050 candidate-scenario forward paths. A separate saved-result check replayed the 22 realized trajectories and reread the five workbooks.

Workbook verification completed 5,317,739 checks and 2,654,345 numerical comparisons, including 288,720 purchase cells, 1,008 time mappings, and 50 specified-day CSV/Markdown pairs.

A separate reproduction reran all four selected reserve policies for the full 334 days from an isolated copy of the source and inputs, with a newly compiled kernel. Ledger, daily-summary, and purchase/reserve-version CSVs were byte-identical, and every numerical candidate/input array agreed exactly. Comparisons excluded solver wall-clock metadata and used the same host and dependency versions. A three-day legacy-mode check also reproduced the earlier Q2 and raw-PV Q3 paths exactly, distinguishing the matched four-search controls from the frozen two-search references.

The frozen experiment environment is Python 3.12.14, NumPy 2.3.5, SciPy 1.18.1, and HiGHS 1.14.0 on macOS 26.6.2, arm64. OpenBLAS and OpenMP use one thread. The deterministic initializer uses seed 0; the separate block-resampling seed is 20260915. The support README pins these dependencies and requires a C99 compiler. Recorded annual reproduction was on the same host and dependencies; bounded package checks do not establish universal portability. Original model, input, workbook and paper identities remain frozen. The retained baseline configurations include historical status and policy fields. These are source metadata, not the current submission status or task permissions; the current twenty-two-policy experiment is defined by `configs/storage_control_v6/experiments.json`.

Appendix B. Full comparisons and retained negative findings

B.1 Frozen Q2 development matrix

Table 14 preserves the twelve-policy Q2 study that selected the inherited history window and cost weights before the reserve experiment. All rows use the original greedy workflow and search counts; D112 is the frozen v5 reference, not the current result2.xlsx policy. These outcomes are not reinterpreted as reserve-controller sensitivities.

Table 14: Complete Q2 historical comparison, 1 February-31 December 2025.

Policy ID	Ordinary (CNY)	Emergency (CNY)	Total (CNY)	Final E (kWh)
B0	11,987,831.57	4,345,851.83	16,333,683.39	2,723.620153
B1	12,263,657.58	3,114,549.77	15,378,207.35	6,434.175697
P_floor	13,361,564.95	661,273.65	14,022,838.60	1,978.550864
X_strong_morning85	13,349,990.09	644,292.17	13,994,282.26	2,441.729981
D56	13,192,899.64	769,064.99	13,961,964.63	2,050.715674
D28	13,192,732.28	767,523.60	13,960,255.88	2,107.322361
D112	13,075,237.03	838,655.45	13,913,892.48	3,139.663254
D56_risk625	13,305,891.73	638,539.01	13,944,430.74	2,060.384331
D56_mu069	13,294,245.37	734,215.87	14,028,461.25	10,727.473563
D56_mu0	13,152,123.53	920,040.32	14,072,163.85	1,569.804649
D112_refined	13,075,951.33	839,116.05	13,915,067.38	3,154.980179
D56_risk625_refined	13,305,422.99	643,672.76	13,949,095.75	2,054.975640

B.2 Complete current matched matrix

Table 15 contains all twenty-two policies from the recorded matched experiment: eleven input/permission configurations under each controller. Both arms have four searches per issue. Every row covers 334 continuous days from the same initial energy; the four selected reserve rows correspond to Table 12. Non-success counts refer to numerical stopping flags, not physical infeasibility.

Table 15: All 22 matched v6 policies, 1 February-31 December 2025; CNY. Suffixes identify the controller.

Policy ID	Contract	Emergency	Total	Non-success starts
q2_greedy	13,075,508.77	839,452.30	13,914,961.06	952
q2_reserve	12,983,297.86	862,155.13	13,845,452.99	1,071
q3_no_update_greedy	13,352,912.18	640,123.32	13,993,035.50	930
q3_no_update_reserve	13,264,208.54	686,026.42	13,950,234.96	1,043
q3_soc_greedy	13,428,635.70	275,573.74	13,704,209.45	2,539
q3_soc_reserve	13,313,835.92	428,283.44	13,742,119.36	3,002
q3_raw_greedy	13,260,330.46	298,999.92	13,559,330.38	2,571
q3_raw_reserve	13,201,668.27	373,329.17	13,574,997.43	2,969
q3_w28_greedy	13,286,743.84	281,511.90	13,568,255.74	2,497
q3_w28_reserve	13,234,894.79	339,567.02	13,574,461.81	2,935
q42_ols_greedy	13,817,869.87	868,692.76	14,686,562.63	1,018
q42_ols_reserve	13,704,872.50	913,646.48	14,618,518.98	1,108
q42_fixed_greedy	13,808,029.62	871,746.10	14,679,775.73	952
q42_fixed_reserve	13,708,867.43	912,730.83	14,621,598.25	1,071
q43_raw_ols_greedy	13,994,684.40	326,147.12	14,320,831.53	2,394
q43_raw_ols_reserve	13,923,929.42	389,707.68	14,313,637.10	2,964
q43_w28_ols_greedy	14,017,133.20	306,013.76	14,323,146.95	2,423
q43_w28_ols_reserve	13,957,964.46	360,657.52	14,318,621.98	2,954
q43_raw_fixed_greedy	14,004,935.62	327,290.75	14,332,226.37	2,571
q43_raw_fixed_reserve	13,943,230.62	399,299.36	14,342,529.98	2,969
q43_w28_fixed_greedy	14,031,875.36	310,663.35	14,342,538.71	2,497
q43_w28_fixed_reserve	13,977,052.49	365,247.17	14,342,299.66	2,935

B.3 Earlier reserve and update-gate experiments

Table 16 reports an archived study that used March for selection and April-December (275 days) for subsequent evaluation after the year had already been inspected. It belongs to the earlier nominal purchasing workflow, with its own within-group initial states. Its reserve reference is a different device from the present shared discharge-threshold vector. Cross-group costs are not one-factor comparisons.

Table 16: Earlier candidate outcomes, 1 April-31 December 2025; saving = within-group reference minus candidate.

Policy ID	Group reference	Total (CNY)	Saving (CNY)	Emergency (kWh)
pv_raw	pv_raw	11,438,135.24	0.00	115,781.783236
pv_bias	pv_raw	11,391,988.76	46,146.49	107,231.288066
risk_none	risk_none	13,750,367.53	0.00	676,738.389457
risk_fixed	risk_none	13,750,362.50	5.03	676,735.970580
risk_g05	risk_none	13,750,362.50	5.03	676,735.970580
risk_g10	risk_none	13,750,365.17	2.36	676,735.970580
gate_paid	gate_paid	11,438,052.17	0.00	115,748.103000
gate_q50	gate_paid	11,440,332.16	-2,279.99	119,018.242572
gate_q75	gate_paid	11,573,630.89	-135,578.73	145,217.448236
gate_frozen	gate_paid	13,239,531.48	-1,801,479.31	507,500.070395
gate_info_frozen	gate_paid	13,239,531.48	-1,801,479.31	507,500.070395

The old reserve reference saved only about 2-5 CNY and both revision gates increased cost. In that older executor, receiving a new forecast while freezing effective purchases left execution unchanged. That statement does not apply to the current controller when a permitted update changes thresholds while purchases stay fixed.

B.4 Earlier correction sensitivity

Table 17 reports six preserved cases using the original April-December period and nominal-MILP workflow. February and March used raw PV in the corresponding February-December deployment; the displayed comparison starts in April. Pairs are matched within each case, while different cases may change efficiency, terminal restrictions, or price input.

Table 17: Earlier correction sensitivity, 1 April-31 December 2025; CNY.

Raw / corrected pair	Raw cost	Corrected cost	Cash saving	Inventory-adjusted
fixed_raw / fixed_w14	11,438,135.24	11,401,468.18	36,667.07	36,613.71
fixed_raw / fixed_w28	11,438,135.24	11,391,988.76	46,146.49	46,106.35
fixed_raw / fixed_w56	11,438,135.24	11,389,342.27	48,792.97	48,752.49
efficiency_raw / efficiency_w28	11,055,054.93	11,004,126.31	50,928.63	50,857.90
soft_raw / soft_w28	11,438,052.17	11,391,863.74	46,188.42	46,192.08
variable_raw / variable_w28	12,142,444.11	12,100,661.01	41,783.10	41,783.13

All six archived correction differences were favorable in their own workflow. Current correction results are instead given in Tables 8 and 15; the historical table cannot certify their robustness. No history window, cost weight, or physical parameter was retuned during the present reserve comparison.

Appendix C. Workbook, table, and audit mapping

The support archive contains the five current workbooks in `workbooks/`. They are byte-identical copies of the English files under the original project directory `outputs/storage_control_v6_english_20260912/`; original production sources remain under `outputs/storage_control_v6/`. The package manifests identify these roles separately. English workbook copies preserve the numerical results and physical time intervals of the verified production sources, with translated labels recorded by the accompanying mapping.

Table 18: Final workbooks and original statement requirements.

Workbook	Actual policy identifier	Saved record directory	Required response
result1.xlsx	baseline	saved/q1/baseline/	Q1 original Tables 1-2
result2.xlsx	q2_reserve	saved/selected/q2_reserve/	Q2 original Tables 1-3, four dates
result3.xlsx	q3_w28_reserve	saved/selected/q3_w28_reserve/	Q3 original Tables 1-3, four dates
result4-2.xlsx	q42_ols_reserve	saved/selected/q42_ols_reserve/	Q4-2 original Tables 1-3, four dates
result4-3.xlsx	q43_raw_ols_reserve	saved/selected/q43_raw_ols_reserve/	Q4-3 original Tables 1-3, four dates

Worksheet names and displayed field labels are translated into English; the accompanying mapping records their correspondence to the original templates. The reviewed time mapping covers 00:00-00:10 through 23:50-24:00, with values assigned by physical date and interval keys. Plan-sheet daily fees mean midnight commitment fees, excluding emergencies. Daily fees on the Final Effective Purchases sheet mean final rule-A contract fees; they are not added to midnight fees again. Total cash adds emergencies once. Full precision is retained in the saved values.

The saved-record paths in Table 18 are relative to the support archive. Annual directories include `ledger_four_dates.csv` and `plan_versions_four_dates.csv`; Appendix D reports their complete required aggregates and emergency-event lists. Q1 uses the saved baseline schedule and states.

Official templates have no reserve-threshold column. Annual issued purchase/reserve versions are therefore retained losslessly in `plan_versions.npz`, and the midnight/final purchase and executed threshold arrays in `execution_plans.npz`. Together with original actual inputs, these reconstruct the selected energy paths without adding columns to the required worksheets. The original workspace archives also retain every discarded candidate and scenario input; those larger candidate archives are not part of the compact support submission.

The scientific archive includes a hashed file inventory. The separate LaTeX archive contains the typesetting sources and the complete program listings in Appendix E. Section 8 describes the scientific verification behind these materials.

Appendix D. Complete prescribed outputs

All quantities below come from the unchanged selected ledgers and the five verified workbooks. Energy is in kWh and fees in CNY. Six decimals are display precision; calculations use unrounded values. The interval labels are physical time keys. For the fixed Q2 and Q4-2 policies, normal purchases satisfy $q^0 = \bar{q}$ and both revision adjustments are zero. Their duplicate purchase and fee rows are combined without removing any required result. Q3 and Q4-3 retain separate original and final quantities because paid revisions can change them.

Emergency events join consecutive intervals above 10^{-7} kWh and split at midnight. A None row records an empty event list for a required date. The full annual ledger, rather than these four dates, determines the annual bill.

D.1 Q1 prescribed results

Tables 19-20 report baseline in `result1.xlsx`. The supplied deterministic day has no emergency purchase or contract revision.

Table 19: Specified purchases, daily purchase and fee.

Interval / scope	Quantity	single day
10:00-10:10	Purchase (kWh)	0.000000
12:00-12:10	Purchase (kWh)	480.412450
14:00-14:10	Purchase (kWh)	0.000000
16:00-16:10	Purchase (kWh)	445.431650
18:00-18:10	Purchase (kWh)	531.894050
20:00-20:10	Purchase (kWh)	0.000000
Daily total	Daily purchase (kWh)	59,482.698998
Daily total	Daily fee (yuan)	35,126.948589

Table 20: Six four-hour storage blocks and start/end stored energy.

Period / instant	Quantity	single day
00:00-04:00	Charge (kWh)	4,500.000000
00:00-04:00	Discharge (kWh)	0.000000
04:00-08:00	Charge (kWh)	833.333333
04:00-08:00	Discharge (kWh)	6,365.841200
08:00-12:00	Charge (kWh)	4,787.964288
08:00-12:00	Discharge (kWh)	1,702.996983
12:00-16:00	Charge (kWh)	5,286.035177
12:00-16:00	Discharge (kWh)	91.101383
16:00-20:00	Charge (kWh)	0.000000
16:00-20:00	Discharge (kWh)	5,780.131883
20:00-24:00	Charge (kWh)	5,333.333333
20:00-24:00	Discharge (kWh)	2,859.868117
00:00	Stored energy (kWh)	6,000.000000
24:00	Stored energy (kWh)	6,000.000000

D.2 Q2 prescribed results

Tables 21-24 report `q2_reserve` in `result2.xlsx`. All four required dates are retained: 20 March, 21 June, 23 September, and 21 December 2025.

Table 21: Q2 specified purchases and daily energy totals; selected reserve policy.

Interval / scope	Quantity	2025-03-20	2025-06-21	2025-09-23	2025-12-21
10:00-10:10	Normal purchase $q^0 = \bar{q}$ (kWh)	0.000000	0.000000	0.000000	1.683020
10:00-10:10	Emergency purchase u (kWh)	0.000000	0.000000	0.000000	0.000000
12:00-12:10	Normal purchase $q^0 = \bar{q}$ (kWh)	619.541702	0.000000	405.028155	956.192518
12:00-12:10	Emergency purchase u (kWh)	0.000000	0.000000	0.000000	0.000000
14:00-14:10	Normal purchase $q^0 = \bar{q}$ (kWh)	0.000000	0.000000	0.000000	0.000000
14:00-14:10	Emergency purchase u (kWh)	0.000000	0.000000	0.000000	0.000000
16:00-16:10	Normal purchase $q^0 = \bar{q}$ (kWh)	496.476970	81.745910	543.987338	840.916582
16:00-16:10	Emergency purchase u (kWh)	0.000000	0.000000	45.445312	0.000000
18:00-18:10	Normal purchase $q^0 = \bar{q}$ (kWh)	564.901285	294.506723	805.488168	706.902519
18:00-18:10	Emergency purchase u (kWh)	0.000000	0.000000	0.000000	0.000000
20:00-20:10	Normal purchase $q^0 = \bar{q}$ (kWh)	0.016603	0.000000	43.021379	0.200558
20:00-20:10	Emergency purchase u (kWh)	0.000000	0.000000	0.000000	0.000000
Daily total	Normal purchase $q^0 = \bar{q}$ (kWh)	67,472.389438	35,218.191521	68,656.407422	98,274.716724
Daily total	Emergency purchase u (kWh)	19.105566	0.000000	334.879790	2.681759

Table 22: Q2 daily fees (CNY), independently recomputed from final quantities.

Interval / scope	Fee component	2025-03-20	2025-06-21	2025-09-23	2025-12-21
Daily total	Increase / decrease adjustments	0.000000	0.000000	0.000000	0.000000
Daily total	Original = final contract fee	41,290.566164	20,719.403264	43,176.524232	64,318.885716
Daily total	Emergency fee	69.386342	0.000000	1,672.805435	15.576998
Daily total	Total fee	41,359.952506	20,719.403264	44,849.329666	64,334.462715

Table 23: Q2 six four-hour storage blocks and boundary energies.

Period / instant	Quantity	2025-03-20	2025-06-21	2025-09-23	2025-12-21
00:00-04:00	Charge (kWh)	8,322.922140	553.255790	8,830.130051	7,982.771293
00:00-04:00	Discharge (kWh)	0.000000	203.092360	0.000000	105.229592
04:00-08:00	Charge (kWh)	1,290.600179	1,077.508954	1,133.115655	1,551.603092
04:00-08:00	Discharge (kWh)	4,959.549293	1,887.179989	5,746.826239	1,540.807619
08:00-12:00	Charge (kWh)	6,120.603454	9,410.152143	4,957.573919	5,658.596986
08:00-12:00	Discharge (kWh)	2,777.377217	0.000000	1,896.615717	6,472.931247
12:00-16:00	Charge (kWh)	3,922.817958	0.000000	4,383.909676	6,014.338555
12:00-16:00	Discharge (kWh)	354.984715	0.000000	705.241463	2,148.813207
16:00-20:00	Charge (kWh)	245.330538	133.151000	53.887397	27.800567
16:00-20:00	Discharge (kWh)	5,553.162454	4,671.984546	5,120.680579	4,764.391505
20:00-24:00	Charge (kWh)	610.437987	1,905.570843	1,410.071489	1,459.694101
20:00-24:00	Discharge (kWh)	3,170.288767	3,049.221347	3,071.381732	2,807.609610
00:00	Stored energy (kWh)	2,003.592163	3,185.699634	2,145.760190	3,122.067361
24:00	Stored energy (kWh)	1,781.297142	4,055.731999	2,458.973192	3,725.410630

Table 24: Q2 complete emergency-event listing for the four specified dates.

Date	Event status	Interval	Emergency energy (kWh)
2025-03-20	Emergency purchase	05:40-05:50	14.695696
2025-03-20	Emergency purchase	18:10-18:20	4.409870
2025-06-21	None	-	0.000000
2025-09-23	Emergency purchase	16:00-17:00	110.317614
2025-09-23	Emergency purchase	17:30-17:40	47.633424
2025-09-23	Emergency purchase	18:20-18:40	81.570408
2025-09-23	Emergency purchase	20:50-21:10	23.482097

Table 24: Q2 complete emergency-event listing for the four specified dates. (continued)

Date	Event status	Interval	Emergency energy (kWh)
2025-09-23	Emergency purchase	21:20-21:50	71.876247
2025-12-21	Emergency purchase	07:50-08:00	2.681759

D.3 Q3 prescribed results

Tables 25-28 report q3_w28_reserve in result3.xlsx. All four required dates are retained: 20 March, 21 June, 23 September, and 21 December 2025.

Table 25: Q3 specified purchases and daily energy totals; selected reserve policy.

Interval / scope	Quantity	2025-03-20	2025-06-21	2025-09-23	2025-12-21
10:00-10:10	Midnight commitment q^0 (kWh)	0.000000	0.000000	0.000000	0.006922
10:00-10:10	Final effective purchase $\bar{q}$ (kWh)	0.000000	0.000000	0.000000	0.000000
10:00-10:10	Emergency purchase u (kWh)	0.000000	0.000000	0.000000	0.000000
12:00-12:10	Midnight commitment q^0 (kWh)	559.595452	0.000000	420.738655	1,022.998393
12:00-12:10	Final effective purchase $\bar{q}$ (kWh)	465.460501	0.000000	102.396315	835.913644
12:00-12:10	Emergency purchase u (kWh)	0.000000	0.000000	0.000000	0.000000
14:00-14:10	Midnight commitment q^0 (kWh)	0.000000	0.000000	0.000000	0.000000
14:00-14:10	Final effective purchase $\bar{q}$ (kWh)	0.000000	0.000000	0.000000	0.000000
14:00-14:10	Emergency purchase u (kWh)	0.000000	0.000000	0.000000	0.000000
16:00-16:10	Midnight commitment q^0 (kWh)	740.446814	267.878812	597.822027	853.355529
16:00-16:10	Final effective purchase $\bar{q}$ (kWh)	588.326565	267.878812	545.829487	853.346854
16:00-16:10	Emergency purchase u (kWh)	0.000000	0.000000	2.112447	0.000000
18:00-18:10	Midnight commitment q^0 (kWh)	690.567085	525.677320	776.517087	683.033450
18:00-18:10	Final effective purchase $\bar{q}$ (kWh)	690.489352	525.677320	723.326761	640.356996
18:00-18:10	Emergency purchase u (kWh)	0.000000	0.000000	0.000000	0.000000
20:00-20:10	Midnight commitment q^0 (kWh)	1.504363	0.000000	43.209865	0.438859
20:00-20:10	Final effective purchase $\bar{q}$ (kWh)	0.000000	0.000000	43.178087	0.379689
20:00-20:10	Emergency purchase u (kWh)	0.000000	0.000000	0.000000	0.000000
Daily total	Midnight commitment q^0 (kWh)	69,506.344688	34,602.450543	71,905.290979	98,844.356354
Daily total	Final effective purchase $\bar{q}$ (kWh)	65,574.553306	34,602.450543	68,927.281966	96,853.790938
Daily total	Emergency purchase u (kWh)	405.591370	0.000000	2.112447	0.000000

Table 26: Q3 daily fees (CNY), independently recomputed from final quantities.

Interval / scope	Fee component	2025-03-20	2025-06-21	2025-09-23	2025-12-21
Daily total	Original commitment fee	43,254.042196	20,773.174362	46,190.080842	64,380.914597
Daily total	Increase fee	0.712650	0.000000	7.929151	0.000395
Daily total	Decrease adjustment	-1,319.123299	0.000000	-905.932800	-765.204642
Daily total	Final contract fee	41,935.631548	20,773.174362	45,292.077193	63,615.710350
Daily total	Emergency fee	2,032.274871	0.000000	8.736026	0.000000
Daily total	Total fee	43,967.906419	20,773.174362	45,300.813219	63,615.710350

Table 27: Q3 six four-hour storage blocks and boundary energies.

Period / instant	Quantity	2025-03-20	2025-06-21	2025-09-23	2025-12-21
00:00-04:00	Charge (kWh)	7,651.882012	622.341893	7,830.808863	8,816.531849
00:00-04:00	Discharge (kWh)	0.000000	123.533993	29.699427	80.845309
04:00-08:00	Charge (kWh)	1,542.581259	424.013391	937.883122	1,701.924213
04:00-08:00	Discharge (kWh)	5,991.042078	2,237.512624	5,470.575538	1,708.528551
08:00-12:00	Charge (kWh)	4,165.124146	9,799.866873	5,142.905450	5,685.292613
08:00-12:00	Discharge (kWh)	2,500.150113	0.000000	1,896.615717	6,503.996700
12:00-16:00	Charge (kWh)	6,337.915081	0.000000	4,121.587803	5,936.931482

Table 27: Q3 six four-hour storage blocks and boundary energies. (continued)

Period / instant	Quantity	2025-03-20	2025-06-21	2025-09-23	2025-12-21
12:00-16:00	Discharge (kWh)	126.358517	27.005000	460.930104	2,071.774124
16:00-20:00	Charge (kWh)	305.779626	203.460647	138.271987	72.297301
16:00-20:00	Discharge (kWh)	5,137.139025	4,390.695375	4,458.901938	5,186.107264
20:00-24:00	Charge (kWh)	584.795417	1,859.126345	1,481.697910	1,179.995060
20:00-24:00	Discharge (kWh)	3,135.969354	2,943.954339	3,252.315630	2,835.770964
00:00	Stored energy (kWh)	2,359.641046	3,661.785188	3,237.372839	2,390.215285
24:00	Stored energy (kWh)	2,121.511848	4,476.711944	3,626.280957	3,013.865095

Table 28: Q3 complete emergency-event listing for the four specified dates.

Date	Event status	Interval	Emergency energy (kWh)
2025-03-20	Emergency purchase	09:10-10:00	277.227103
2025-03-20	Emergency purchase	14:10-14:40	128.364267
2025-06-21	None	-	0.000000
2025-09-23	Emergency purchase	16:00-16:10	2.112447
2025-12-21	None	-	0.000000

D.4 Q4-2 prescribed results

Tables 29-32 report `q42_ols_reserve` in `result4-2.xlsx`. All four required dates are retained: 20 March, 21 June, 23 September, and 21 December 2025.

Table 29: Q4-2 specified purchases and daily energy totals; selected reserve policy.

Interval / scope	Quantity	2025-03-20	2025-06-21	2025-09-23	2025-12-21
10:00-10:10	Normal purchase $q^0 = \bar{q}$ (kWh)	0.000000	0.000000	0.000000	0.000000
10:00-10:10	Emergency purchase u (kWh)	0.000000	0.000000	0.000000	0.000000
12:00-12:10	Normal purchase $q^0 = \bar{q}$ (kWh)	612.356989	0.000000	408.833631	939.331894
12:00-12:10	Emergency purchase u (kWh)	0.000000	0.000000	0.000000	0.000000
14:00-14:10	Normal purchase $q^0 = \bar{q}$ (kWh)	0.000000	0.000000	0.000000	0.000000
14:00-14:10	Emergency purchase u (kWh)	0.000000	0.000000	0.000000	0.000000
16:00-16:10	Normal purchase $q^0 = \bar{q}$ (kWh)	522.812758	85.111219	538.552472	849.050539
16:00-16:10	Emergency purchase u (kWh)	0.000000	0.000000	25.673195	0.000000
18:00-18:10	Normal purchase $q^0 = \bar{q}$ (kWh)	557.888819	253.910664	674.755908	735.419364
18:00-18:10	Emergency purchase u (kWh)	0.000000	0.000000	0.000000	0.000000
20:00-20:10	Normal purchase $q^0 = \bar{q}$ (kWh)	547.850652	0.000000	42.294725	0.037150
20:00-20:10	Emergency purchase u (kWh)	0.000000	0.000000	0.000000	0.000000
Daily total	Normal purchase $q^0 = \bar{q}$ (kWh)	67,081.511578	35,188.284492	68,411.094612	95,601.259539
Daily total	Emergency purchase u (kWh)	20.980414	0.000000	147.341009	0.000000

Table 30: Q4-2 daily fees (CNY), independently recomputed from final quantities.

Interval / scope	Fee component	2025-03-20	2025-06-21	2025-09-23	2025-12-21
Daily total	Increase / decrease adjustments	0.000000	0.000000	0.000000	0.000000
Daily total	Original = final contract fee	42,506.757999	17,977.043138	45,211.408117	74,342.617796
Daily total	Emergency fee	60.174357	0.000000	710.709323	0.000000
Daily total	Total fee	42,566.932356	17,977.043138	45,922.117441	74,342.617796

Table 31: Q4-2 six four-hour storage blocks and boundary energies.

Period / instant	Quantity	2025-03-20	2025-06-21	2025-09-23	2025-12-21
00:00-04:00	Charge (kWh)	6,952.620820	1,221.039164	9,255.730063	6,189.682586
00:00-04:00	Discharge (kWh)	99.390812	1,407.698468	152.905237	48.018548
04:00-08:00	Charge (kWh)	3,031.894949	1,323.025048	560.996989	129.373932
04:00-08:00	Discharge (kWh)	5,135.092555	1,765.273891	5,601.079156	893.806780
08:00-12:00	Charge (kWh)	5,491.701411	9,379.220527	4,758.806531	5,189.806286
08:00-12:00	Discharge (kWh)	2,777.377217	0.000000	1,896.615717	6,104.074137
12:00-16:00	Charge (kWh)	4,296.777844	0.000000	4,578.880928	6,109.879829
12:00-16:00	Discharge (kWh)	328.338506	0.000000	652.180336	2,220.109417
16:00-20:00	Charge (kWh)	151.500566	164.603582	0.000000	40.071189
16:00-20:00	Discharge (kWh)	6,635.647218	4,632.067342	5,461.172272	4,758.950441
20:00-24:00	Charge (kWh)	496.875033	2,009.760130	1,288.894199	1,804.503860
20:00-24:00	Discharge (kWh)	1,882.097485	3,106.550897	2,614.812274	3,344.167707
00:00	Stored energy (kWh)	2,189.397108	3,594.568578	2,282.989605	5,238.697054
24:00	Stored energy (kWh)	1,837.582011	4,158.462630	2,483.339675	3,456.652936

Table 32: Q4-2 complete emergency-event listing for the four specified dates.

Date	Event status	Interval	Emergency energy (kWh)
2025-03-20	Emergency purchase	05:20-05:30	1.221579
2025-03-20	Emergency purchase	05:40-05:50	19.758835
2025-06-21	None	-	0.000000
2025-09-23	Emergency purchase	15:00-15:10	7.441153
2025-09-23	Emergency purchase	16:00-16:50	128.465330
2025-09-23	Emergency purchase	17:10-17:20	11.434526
2025-12-21	None	-	0.000000

D.5 Q4-3 prescribed results

Tables 33-36 report `q43_raw_ols_reserve` in `result4-3.xlsx`. All four required dates are retained: 20 March, 21 June, 23 September, and 21 December 2025.

Table 33: Q4-3 specified purchases and daily energy totals; selected reserve policy.

Interval / scope	Quantity	2025-03-20	2025-06-21	2025-09-23	2025-12-21
10:00-10:10	Midnight commitment q^0 (kWh)	0.000000	0.000000	0.000000	0.000000
10:00-10:10	Final effective purchase $\bar{q}$ (kWh)	0.000000	0.000000	0.000000	0.000000
10:00-10:10	Emergency purchase u (kWh)	0.000000	0.000000	0.000000	0.000000
12:00-12:10	Midnight commitment q^0 (kWh)	573.679976	0.000000	440.362277	1,010.030952
12:00-12:10	Final effective purchase $\bar{q}$ (kWh)	473.770839	0.000000	149.968831	849.860385
12:00-12:10	Emergency purchase u (kWh)	0.000000	0.000000	0.000000	0.000000
14:00-14:10	Midnight commitment q^0 (kWh)	114.777886	0.000000	0.000000	0.000000
14:00-14:10	Final effective purchase $\bar{q}$ (kWh)	0.018211	0.000000	0.000000	0.000000
14:00-14:10	Emergency purchase u (kWh)	0.000000	0.000000	0.000000	0.000000
16:00-16:10	Midnight commitment q^0 (kWh)	718.544559	130.667725	630.916833	878.912606
16:00-16:10	Final effective purchase $\bar{q}$ (kWh)	555.832097	130.667577	567.312395	878.921542
16:00-16:10	Emergency purchase u (kWh)	6.429453	0.000000	21.811177	0.000000
18:00-18:10	Midnight commitment q^0 (kWh)	697.304657	314.191179	631.627442	747.048913
18:00-18:10	Final effective purchase $\bar{q}$ (kWh)	695.955883	314.188984	631.627507	691.709852
18:00-18:10	Emergency purchase u (kWh)	0.000000	0.000000	0.000000	0.000000
20:00-20:10	Midnight commitment q^0 (kWh)	509.049495	0.000000	44.992820	0.000000
20:00-20:10	Final effective purchase $\bar{q}$ (kWh)	489.003699	0.000000	44.992410	0.000000
20:00-20:10	Emergency purchase u (kWh)	0.000000	0.000000	0.000000	0.000000

Table 33: Q4-3 specified purchases and daily energy totals; selected reserve policy. (continued)

Interval / scope	Quantity	2025-03-20	2025-06-21	2025-09-23	2025-12-21
Daily total	Midnight commitment q^0 (kWh)	68,878.182059	34,624.799275	70,701.483930	95,487.866034
Daily total	Final effective purchase $\bar{q}$ (kWh)	65,815.578856	34,621.574698	68,093.153974	94,603.617970
Daily total	Emergency purchase u (kWh)	80.521279	0.000000	21.811177	0.000000

Table 34: Q4-3 daily fees (CNY), independently recomputed from final quantities.

Interval / scope	Fee component	2025-03-20	2025-06-21	2025-09-23	2025-12-21
Daily total	Original commitment fee	44,495.069291	18,136.015869	47,523.413623	73,806.817939
Daily total	Increase fee	0.513036	0.018309	8.938354	0.013674
Daily total	Decrease adjustment	-1,197.234520	-0.645221	-761.907137	-425.607374
Daily total	Final contract fee	43,298.347807	18,135.388956	46,770.444841	73,381.224239
Daily total	Emergency fee	416.482073	0.000000	101.694611	0.000000
Daily total	Total fee	43,714.829880	18,135.388956	46,872.139452	73,381.224239

Table 35: Q4-3 six four-hour storage blocks and boundary energies.

Period / instant	Quantity	2025-03-20	2025-06-21	2025-09-23	2025-12-21
00:00-04:00	Charge (kWh)	6,972.588796	890.944050	8,222.974419	6,974.234699
00:00-04:00	Discharge (kWh)	149.490758	1,679.811311	21.595547	37.603478
04:00-08:00	Charge (kWh)	2,411.755985	1,076.603419	211.067135	424.283461
04:00-08:00	Discharge (kWh)	5,728.020513	1,769.774621	5,619.749682	1,131.704666
08:00-12:00	Charge (kWh)	3,918.213892	9,684.825957	5,069.137275	4,998.096189
08:00-12:00	Discharge (kWh)	2,776.909839	0.000000	1,896.615717	5,950.168146
12:00-16:00	Charge (kWh)	6,478.306639	0.000000	4,516.597703	6,274.991366
12:00-16:00	Discharge (kWh)	126.358517	0.000000	434.337638	2,379.322893
16:00-20:00	Charge (kWh)	325.538364	124.231504	72.383770	0.000000
16:00-20:00	Discharge (kWh)	6,280.316300	4,269.435790	4,998.091007	5,030.583484
20:00-24:00	Charge (kWh)	547.438955	1,911.008150	1,360.696311	1,674.719363
20:00-24:00	Discharge (kWh)	1,982.933760	3,005.779581	2,666.370329	3,446.739271
00:00	Stored energy (kWh)	2,399.744521	4,145.737174	3,406.185412	4,549.776170
24:00	Stored energy (kWh)	2,050.392127	4,548.143054	3,539.578675	2,887.999921

Table 36: Q4-3 complete emergency-event listing for the four specified dates.

Date	Event status	Interval	Emergency energy (kWh)
2025-03-20	Emergency purchase	09:10-09:20	0.467377
2025-03-20	Emergency purchase	14:30-14:40	70.293283
2025-03-20	Emergency purchase	15:50-16:10	9.760619
2025-06-21	None	-	0.000000
2025-09-23	Emergency purchase	16:00-16:10	21.811177
2025-12-21	None	-	0.000000

Appendix E. Complete scientific programs

Table 37 lists every supporting file; the manifest gives its identity. The following source listings implement data preparation, Q1 scheduling, and the annual procurement and storage procedures. The accompanying support archive supplies the complete source files, configurations, environment requirements, input placement instructions, and manifests. These programs are separate from the LaTeX typesetting source.

Support inventory and execution scope

The supporting directory is separate from the manuscript PDF. Its exact relative file paths are listed below; the manifest supplies SHA-256 identities. Original contest attachments are obtained separately and placed as specified in the raw-input README. Scientific source listings are complete, rather than excerpts or typesetting programs. Saved historical references retain their original comparison roles.

Table 37: Complete supporting-file inventory.

File relative to support_materials/
AI_Tool_Usage_Declaration.md
DetailsofAIToolUsage.pdf
MANIFEST.csv
README.md
SOURCE_PROVENANCE.json
configs/data_processing.json
configs/model_baseline.json
configs/q2_baseline.json
configs/q4_baseline.json
configs/storage_control_v6/experiments.json
raw/README.md
raw/required_files.json
requirements.txt
results/q2_direct_v4/runs/D112/ledger.csv
results/unified_direct_v5/runs/q3_raw/ledger.csv
results/unified_direct_v5/runs/q42_fixed/ledger.csv
results/unified_direct_v5/runs/q43_raw_ols/ledger.csv
saved/all_policies/q2_greedy/config.json
saved/all_policies/q2_greedy/daily.csv
saved/all_policies/q2_greedy/summary.json
saved/all_policies/q2_reserve/config.json
saved/all_policies/q2_reserve/daily.csv
saved/all_policies/q2_reserve/summary.json
saved/all_policies/q3_no_update_greedy/config.json
saved/all_policies/q3_no_update_greedy/daily.csv
saved/all_policies/q3_no_update_greedy/summary.json
saved/all_policies/q3_no_update_reserve/config.json
saved/all_policies/q3_no_update_reserve/daily.csv
saved/all_policies/q3_no_update_reserve/summary.json
saved/all_policies/q3_raw_greedy/config.json
saved/all_policies/q3_raw_greedy/daily.csv
saved/all_policies/q3_raw_greedy/summary.json
saved/all_policies/q3_raw_reserve/config.json
saved/all_policies/q3_raw_reserve/daily.csv
saved/all_policies/q3_raw_reserve/summary.json
saved/all_policies/q3_soc_greedy/config.json
saved/all_policies/q3_soc_greedy/daily.csv
saved/all_policies/q3_soc_greedy/summary.json
saved/all_policies/q3_soc_reserve/config.json
saved/all_policies/q3_soc_reserve/daily.csv
saved/all_policies/q3_soc_reserve/summary.json
saved/all_policies/q3_w28_greedy/config.json
saved/all_policies/q3_w28_greedy/daily.csv
saved/all_policies/q3_w28_greedy/summary.json
saved/all_policies/q3_w28_reserve/config.json
saved/all_policies/q3_w28_reserve/daily.csv
saved/all_policies/q3_w28_reserve/summary.json
saved/all_policies/q42_fixed_greedy/config.json
saved/all_policies/q42_fixed_greedy/daily.csv
saved/all_policies/q42_fixed_greedy/summary.json
saved/all_policies/q42_fixed_reserve/config.json
saved/all_policies/q42_fixed_reserve/daily.csv

Support inventory (continued)

saved/all_policies/q42_fixed_reserve/summary.json
saved/all_policies/q42_ols_greedy/config.json
saved/all_policies/q42_ols_greedy/daily.csv
saved/all_policies/q42_ols_greedy/summary.json
saved/all_policies/q42_ols_reserve/config.json
saved/all_policies/q42_ols_reserve/daily.csv
saved/all_policies/q42_ols_reserve/summary.json
saved/all_policies/q43_raw_fixed_greedy/config.json
saved/all_policies/q43_raw_fixed_greedy/daily.csv
saved/all_policies/q43_raw_fixed_greedy/summary.json
saved/all_policies/q43_raw_fixed_reserve/config.json
saved/all_policies/q43_raw_fixed_reserve/daily.csv
saved/all_policies/q43_raw_fixed_reserve/summary.json
saved/all_policies/q43_raw_ols_greedy/config.json
saved/all_policies/q43_raw_ols_greedy/daily.csv
saved/all_policies/q43_raw_ols_greedy/summary.json
saved/all_policies/q43_raw_ols_reserve/config.json
saved/all_policies/q43_raw_ols_reserve/daily.csv
saved/all_policies/q43_raw_ols_reserve/summary.json
saved/all_policies/q43_w28_fixed_greedy/config.json
saved/all_policies/q43_w28_fixed_greedy/daily.csv
saved/all_policies/q43_w28_fixed_greedy/summary.json
saved/all_policies/q43_w28_fixed_reserve/config.json
saved/all_policies/q43_w28_fixed_reserve/daily.csv
saved/all_policies/q43_w28_fixed_reserve/summary.json
saved/all_policies/q43_w28_ols_greedy/config.json
saved/all_policies/q43_w28_ols_greedy/daily.csv
saved/all_policies/q43_w28_ols_greedy/summary.json
saved/all_policies/q43_w28_ols_reserve/config.json
saved/all_policies/q43_w28_ols_reserve/daily.csv
saved/all_policies/q43_w28_ols_reserve/summary.json
saved/comparisons/comparison_all.csv
saved/comparisons/monthly_all.csv
saved/comparisons/paired_daily.csv
saved/comparisons/paired_monthly.csv
saved/comparisons/paired_summary.csv
saved/comparisons/selection.json
saved/fixed_price.csv
saved/historical_references/frozen_v5_q2_summary.json
saved/historical_references/frozen_v5_q3_summary.json
saved/historical_references/frozen_v5_q42_summary.json
saved/historical_references/frozen_v5_q43_summary.json
saved/q1/alternate/config_snapshot.json
saved/q1/alternate/schedule.csv
saved/q1/alternate/solver_status.json
saved/q1/alternate/states.csv
saved/q1/alternate/summary.json
saved/q1/baseline/config_snapshot.json
saved/q1/baseline/schedule.csv
saved/q1/baseline/solver_status.json
saved/q1/baseline/states.csv
saved/q1/baseline/summary.json
saved/q1/baseline_lp/config_snapshot.json
saved/q1/baseline_lp/schedule.csv
saved/q1/baseline_lp/solver_status.json
saved/q1/baseline_lp/states.csv
saved/q1/baseline_lp/summary.json
saved/q1/comparison.csv
saved/q1/efficiency_sensitivity.json
saved/q1/input_day.csv
saved/q1/no_storage/config_snapshot.json
saved/q1/no_storage/schedule.csv
saved/q1/no_storage/solver_status.json
saved/q1/no_storage/states.csv
saved/q1/no_storage/summary.json
saved/q1/nonuniqueness.json
saved/q1/roundtrip_90/config_snapshot.json
saved/q1/roundtrip_90/schedule.csv
saved/q1/roundtrip_90/solver_status.json
saved/q1/roundtrip_90/states.csv

Support inventory (continued)

saved/q1/roundtrip_90/summary.json
saved/q1/roundtrip_90_lp/config_snapshot.json
saved/q1/roundtrip_90_lp/schedule.csv
saved/q1/roundtrip_90_lp/solver_status.json
saved/q1/roundtrip_90_lp/states.csv
saved/q1/roundtrip_90_lp/summary.json
saved/selected/q2_reserve/execution_plans.npz
saved/selected/q2_reserve/ledger_four_dates.csv
saved/selected/q2_reserve/plan_versions.npz
saved/selected/q2_reserve/plan_versions_four_dates.csv
saved/selected/q3_w28_reserve/execution_plans.npz
saved/selected/q3_w28_reserve/ledger_four_dates.csv
saved/selected/q3_w28_reserve/plan_versions.npz
saved/selected/q3_w28_reserve/plan_versions_four_dates.csv
saved/selected/q42_ols_reserve/execution_plans.npz
saved/selected/q42_ols_reserve/ledger_four_dates.csv
saved/selected/q42_ols_reserve/plan_versions.npz
saved/selected/q42_ols_reserve/plan_versions_four_dates.csv
saved/selected/q43_raw_ols_reserve/execution_plans.npz
saved/selected/q43_raw_ols_reserve/ledger_four_dates.csv
saved/selected/q43_raw_ols_reserve/plan_versions.npz
saved/selected/q43_raw_ols_reserve/plan_versions_four_dates.csv
saved/warmup/daily.csv
saved/warmup/forecast_selection.json
saved/warmup/ledger.csv
saved/warmup/summary.json
scripts/check_saved.py
scripts/prepare_data.py
scripts/prepare_inputs.py
scripts/run_q2.py
scripts/smoke_test.py
scripts/solve_q1.py
scripts/storage_control_v6/analyze.py
scripts/storage_control_v6/compat.py
scripts/storage_control_v6/control.py
scripts/storage_control_v6/inputs.py
scripts/storage_control_v6/kernel.c
scripts/storage_control_v6/kernel.py
scripts/storage_control_v6/policy.py
scripts/storage_control_v6/run.py
source_changes.diff
workbooks/policy_mapping.json
workbooks/result1.xlsx
workbooks/result2.xlsx
workbooks/result3.xlsx
workbooks/result4-2.xlsx
workbooks/result4-3.xlsx

Complete scientific programs

Program 1. scripts/prepare_data.py

```
1  """Deterministic parsing and explicitly assumed energy conversion; no modeling."""
2  import argparse
3  from datetime import datetime, time
4  import json
5  from pathlib import Path
6  import re
7  import numpy as np
8  import pandas as pd
9  import openpyxl
10 from openpyxl.utils import get_column_letter
11
12 WORK = Path(__file__).resolve().parents[1]
13 ROOT = WORK
14 CONFIG = json.loads((WORK / 'configs/data_processing.json').read_text())
15 SOURCE = ROOT / CONFIG['source_directory']
16 OUT = WORK / 'data/processed'
17
18
```



```

88 labels = [r[0] for r in rows[1:]]
89 minutes = slots(labels)
90 q = pd.DataFrame({'slot_id': np.arange(1, len(minutes)+1),
91                  'raw_time_label': [str(v) for v in labels],
92                  'start_minute': minutes-10, 'end_minute': minutes})
93 # Q1 has no source date. Do not attach a fictional full timestamp to this day.
94 for field, mins in [('start_time', minutes-10), ('end_time', minutes)]:
95     q[field] = [f'{m//60:02d}:{m%60:02d}:00' for m in mins]
96 q[['price_yuan_per_kwh', 'load_kw', 'pv_forecast_kw']] = numeric([r[1:] for r in rows[1:]])
97 q['load_kwh'] = q.load_kw/6
98 q['pv_forecast_kwh'] = q.pv_forecast_kw/6
99 q['net_load_forecast_kw'] = q.load_kw-q.pv_forecast_kw
100 q['net_load_forecast_kwh'] = q.load_kwh-q.pv_forecast_kwh
101 q['source_row'] = np.arange(2, len(q)+2)
102 fixed = q[['slot_id', 'raw_time_label', 'start_minute', 'end_minute',
103           'start_time', 'end_time', 'price_yuan_per_kwh', 'source_row']].copy()
104 write(q, 'q1_day')
105 write(fixed, 'fixed_price')
106 return q, fixed
107
108
109 def annual_long(filename, sheet, value_name, prefix):
110     rows = rows_of(filename, sheet)
111     labels = rows[0][1:]
112     minutes = slots(labels)
113     dates = pd.DatetimeIndex([naive_time(r[0]) for r in rows[1:]])
114     if not dates.equals(pd.date_range('2025-01-01', '2025-12-31')):
115         raise ValueError(f'Unexpected source dates: {filename}/{sheet}')
116     n = len(dates)
117     frame = pd.DataFrame({'date': np.repeat(dates.to_numpy(), 144),
118                          'slot_id': np.tile(np.arange(1,145), n),
119                          'value_name': numeric([r[1:] for r in rows[1:]].reshape(-1))})
120     frame['interval_end'] = frame.date + pd.to_timedelta(np.tile(minutes, n), unit='m')
121     frame['interval_start'] = frame.interval_end-pd.Timedelta(minutes=10)
122     frame[f'{prefix}_raw_time_label'] = np.tile([str(v) for v in labels], n)
123     frame[f'{prefix}_source_row'] = np.repeat(np.arange(2,n+2), 144)
124     frame[f'{prefix}_source_column'] = np.tile([get_column_letter(c) for c in range(2,146)], n)
125     return frame
126
127
128 def prepare_actual(fixed):
129     load = annual_long('attachment_2.xlsx', '\u5c0f\u533a\u8d1f\u8f7d', 'load_actual_kw', 'load')
130     pv = annual_long('attachment_2.xlsx', '\u5149\u4f0f\u53d1\u7535\u5b9e\u9645\u529f\u7387', 'pv_actual_kw', 'pv')
131     price = annual_long('attachment_4.xlsx', 'Sheet1', 'actual_price_yuan_per_kwh', 'price')
132     keys = ['date', 'slot_id', 'interval_start', 'interval_end']
133     actual = load.merge(pv, on=keys, how='outer', validate='one_to_one').merge(
134         price, on=keys, how='outer', validate='one_to_one')
135     actual = actual.merge(fixed[['slot_id', 'price_yuan_per_kwh']].rename(
136         columns={'price_yuan_per_kwh': 'fixed_price_yuan_per_kwh'}, on='slot_id', validate='many_to_one')
137     )
138     if len(actual) != len(load) or actual.isna().any().any():
139         raise ValueError('Unexpected merge expansion, loss or missing values')
140     actual = actual.sort_values('interval_start').reset_index(drop=True)
141     actual['load_actual_kwh'] = actual.load_actual_kw/6
142     actual['pv_actual_kwh'] = actual.pv_actual_kw/6
143     actual['net_load_actual_kw'] = actual.load_actual_kw-actual.pv_actual_kw
144     actual['net_load_actual_kwh'] = actual.load_actual_kwh-actual.pv_actual_kwh
145     actual['available_time'] = actual.interval_end
146     actual['date'] = actual.date.dt.strftime('%Y-%m-%d')
147     write(actual, 'actual_10min')
148     return actual
149
150 def prepare_hourly():
151     rows = rows_of('attachment_3.xlsx', 'Sheet1')
152     if list(rows[0][2:]) != [f'\u9884\u62a5{h}\u5c0f\u65f6' for h in range(1,25)]:
153         raise ValueError('Unexpected horizon headers')
154     raw_dates = pd.Series([r[0] for r in rows[1:]], dtype=object)
155     clean_dates = raw_dates.replace(r'^\s*$', np.nan, regex=True).ffill()

```

```

156 dates = pd.to_datetime(clean_dates, errors='raise')
157 minutes = [minute_of_day(r[1]) for r in rows[1:]]
158 issues = pd.DatetimeIndex(dates+pd.to_timedelta(minutes, unit='m'))
159 if not issues.equals(pd.date_range('2025-01-01', periods=1460, freq='6h')):
160     raise ValueError('Unexpected issue dates/order')
161 records = pd.DataFrame({'issue_time': np.repeat(issues.to_numpy(),24),
162                        'horizon_h': np.tile(np.arange(1,25),len(issues)),
163                        'pv_forecast_kw': numeric([r[2:] for r in rows[1:]].reshape(-1))})
164 records['target_time'] = records.issue_time+pd.to_timedelta(records.horizon_h,unit='h')
165 records['source_row'] = np.repeat(np.arange(2,len(issues)+2),24)
166 records['source_column'] = np.tile([get_column_letter(c) for c in range(3,27)],len(issues))
167 records['raw_date_label'] = np.repeat(raw_dates.fillna('').to_numpy(),24)
168 records['raw_issue_label'] = np.repeat([str(r[1]) for r in rows[1:]],24)
169 records['date_was_filled'] = np.repeat((raw_dates.isna() | raw_dates.astype(str).str.strip().eq('')
170 ).to_numpy(),24)
171 write(records, 'pv_forecast_hourly')
172 changes = [{'source_file': 'attachment_3.xlsx', 'source_sheet': 'Sheet1', 'source_cell': f'A{i+2}',
173            'original_value': json.dumps(raw_dates.iloc[i], ensure_ascii=False),
174            'new_value': clean_dates.iloc[i], 'reason': 'Forward-filled blank date labels within the
175            existing issue order; no power value changed'}
176            for i in range(len(raw_dates)) if records.date_was_filled.iloc[i*24]]
177 pd.DataFrame(changes).to_csv(WORK/'data/interim/transformation_log.csv', index=False)
178 return records
179
180 def convert_forecasts(hourly):
181     Point-power linear integral; first-hour anchor comes only from an older forecast.
182     frames, available_points = [], {}
183     for issue, group in hourly.sort_values(['issue_time', 'horizon_h']).groupby('issue_time', sort=True):
184         powers = group.pv_forecast_kw.to_numpy()
185         anchor = available_points.get(issue)
186         if anchor is None:
187             knots = np.arange(1,25)*60
188             p = powers
189             starts = np.arange(60,1440,10)
190         else:
191             knots = np.arange(25)*60
192             p = np.r_[anchor[1], powers]
193             starts = np.arange(0,1440,10)
194         left = np.interp(starts, knots, p)
195         right = np.interp(starts+10, knots, p)
196         out = pd.DataFrame({'issue_time': issue,
197                          'interval_start': issue+pd.to_timedelta(starts, unit='m'),
198                          'interval_end': issue+pd.to_timedelta(starts+10, unit='m'),
199                          'horizon_start_min': starts, 'horizon_end_min': starts+10,
200                          'pv_forecast_kwh': (left+right)/12,
201                          'conversion_method': CONFIG['forecast_conversion'],
202                          'endpoint_rule': np.where(starts<60, 'previous_issue_point', 'current_issue_points')})
203         out['endpoint_issue_time'] = pd.NaT
204         if anchor is not None:
205             out.loc[starts<60, 'endpoint_issue_time'] = anchor[0]
206         out['date'] = out.interval_start.dt.strftime('%Y-%m-%d')
207         out['slot_id'] = out.interval_start.dt.hour*6+out.interval_start.dt.minute//10+1
208         frames.append(out)
209         # Updating after conversion makes every eligible anchor strictly older.
210         for row in group.itertuples():
211             available_points[row.target_time] = (issue, row.pv_forecast_kw)
212     return pd.concat(frames, ignore_index=True)
213
214 def main():
215     parser = argparse.ArgumentParser()
216     parser.add_argument('--stage', choices=['q1', 'hourly', 'all'], default='all')
217     args = parser.parse_args()
218     if CONFIG['raw_time_label'] != 'interval_end_assumption' or CONFIG['power_interpretation'] != 'interval_average_assumption' or CONFIG['interval_minutes'] != 10:
219         raise ValueError('Changed assumptions require corresponding implementation and verification changes')
220     q, fixed = prepare_q1()
221     if args.stage == 'q1':

```

```

222         return
223     prepare_actual(fixed)
224     hourly = prepare_hourly()
225     if args.stage == 'hourly':
226         return
227     derived = convert_forecasts(hourly)
228     write(derived, 'pv_forecast_10min')
229     coverage = derived.groupby('issue_time').agg(interval_count=('interval_start', 'size'),
230         first_interval_start=('interval_start', 'min'), last_interval_end=('interval_end', 'max')).
231     reset_index()
232     coverage['missing_first_hour'] = coverage.interval_count.eq(138)
233     coverage.to_csv(WORK/'data/interim/forecast_coverage.csv', index=False, date_format='%Y-%m-%dT%H:%M:%S')
234
235 if __name__ == '__main__':
236     main()

```

Program 2. scripts/prepare_inputs.py

```

1  """Prepare raw attachments and historical forecast releases; no dispatch optimization."""
2  from pathlib import Path
3  import argparse
4  import hashlib
5  import json
6  import sys
7  import numpy as np
8  import pandas as pd
9  import prepare_data
10
11  WORK=Path(__file__).resolve().parents[1]
12  sys.path.insert(0, str(WORK/'scripts/storage_control_v6'))
13  from compat import forecast_day
14
15  def main():
16      parser=argparse.ArgumentParser(description=__doc__)
17      parser.add_argument('--without-price-cache', action='store_true')
18      args=parser.parse_args()
19      expected=json.loads((WORK/'raw/required_files.json').read_text())
20      for name, identity in expected.items():
21          source=WORK/'raw'/name
22          if not source.exists(): raise FileNotFoundError(f'Place the original attachment at raw/{name}; see raw/README.md')
23          if hashlib.sha256(source.read_bytes()).hexdigest()!=identity['sha256']:
24              raise ValueError(f'Attachment identity differs: {name}; do not silently change the published experiment input')
25      for rel in ['data/processed', 'data/interim', 'results/q2_direct_v4/forecasts', 'results/storage_control_v6/inputs', 'reports/storage_control_v6']:
26          (WORK/rel).mkdir(parents=True, exist_ok=True)
27      _, fixed=prepare_data.prepare_q1()
28      actual=prepare_data.prepare_actual(fixed)
29      hourly=prepare_data.prepare_hourly()
30      prepare_data.write(prepare_data.convert_forecasts(hourly), 'pv_forecast_10min')
31      # Build every historical release from its own completed history and parameters.
32      source=pd.read_csv(WORK/'data/processed/actual_10min.csv', float_precision='round_trip')
33      fields=['date', 'slot_id', 'interval_end', 'interval_start', 'load_actual_kwh', 'pv_actual_kwh', 'available_time']
34      source=source[fields]
35      ends=pd.to_datetime(source.interval_end)
36      cfg=json.loads((WORK/'configs/q2_baseline.json').read_text())
37      frames=[]; models=[]
38      for day in pd.date_range('2025-01-02', '2025-12-31'):
39          history=source.loc[ends<=day]
40          load, pv, meta=forecast_day(history, day, 'linear_harmonic', cfg)
41          frame=source.loc[source.date==str(day.date())].copy()
42          frame['issue_time']=str(day); frame['history_end']=meta['history_end']
43          frame['load_forecast_kwh']=load; frame['pv_forecast_kwh']=pv
44          frame['net_residual_kwh']=frame.load_actual_kwh-frame.pv_actual_kwh-load+pv
45          frame['residual_available_time']=str(day+pd.Timedelta(days=1))
46          frames.append(frame); models.append({'date': str(day.date()), **meta})

```

```

47 archive=pd.concat(frames,ignore_index=True)
48 archive.to_csv(WORK/'results/q2_direct_v4/forecasts/linear_harmonic.csv',index=False)
49 (WORK/'results/q2_direct_v4/forecasts/models.json').write_text(json.dumps(models,indent=2)+'\n')
50 if not args.without_price_cache:
51     from inputs import prepare
52     prepare()
53 print(json.dumps({'actual_intervals':len(actual),'historical_forecast_intervals':len(archive),
54                 'annual_dispatch_optimizations_run':0,'price_cache_prepared':not args.
55                 without_price_cache}))
56 if __name__=='__main__':main()

```

Program 3. scripts/solve_q1.py

```

1  """Q1 deterministic AC-bus dispatch. All outputs are internal assumption-based results."""
2  import argparse
3  import copy
4  import hashlib
5  import importlib.metadata
6  import json
7  import os
8  from pathlib import Path
9  import platform
10 import sys
11
12 WORK = Path(__file__).resolve().parents[1]
13 INTERIM = WORK / 'data/interim/q1'
14 INTERIM.mkdir(parents=True, exist_ok=True)
15 os.environ['TMPDIR'] = str(INTERIM)
16 os.environ['TMP'] = str(INTERIM)
17 os.environ['TEMP'] = str(INTERIM)
18 os.environ['MPLCONFIGDIR'] = str(INTERIM / 'cache/matplotlib')
19 sys.dont_write_bytecode = True
20
21 import highspy
22 import numpy as np
23 import pandas as pd
24
25
26 def save_json(path, value):
27     path.parent.mkdir(parents=True, exist_ok=True)
28     path.write_text(json.dumps(value, ensure_ascii=False, indent=2, allow_nan=False) + '\n')
29
30
31 def digest(path):
32     return hashlib.sha256(path.read_bytes()).hexdigest()
33
34
35 def solve_model(data, cfg, relaxed=False, tag=None, cost_cap=None):
36     """Return a dispatch plus solver evidence; LP relaxes only mode integrality."""
37     n = len(data)
38     b = cfg['battery']
39     dt = cfg['interval_minutes'] / 60
40     mc, md = b['max_charge_kw'] * dt, b['max_discharge_kw'] * dt
41     h = highspy.Highs()
42     h.setOptionValue('output_flag', bool(tag))
43     if tag:
44         h.setOptionValue('log_file', str(WORK / f'logs/q1/{tag}.log'))
45     for k, v in cfg['solver'].items():
46         if k != 'name':
47             if h.setOptionValue(k, v) != highspy.HighsStatus.kOk:
48                 raise ValueError(f'Invalid solver option: {k}')
49     p = data.price_yuan_per_kwh.to_numpy()
50     load = data.load_kwh.to_numpy()
51     pv = data.pv_forecast_kwh.to_numpy()
52     g = [h.addVariable(obj=float(p[t]) if cost_cap is None else 0., name=f'g_{t}') for t in range(n)]
53     c = [h.addVariable(ub=mc, obj=float(t + 1) if cost_cap is not None else 0., name=f'c_{t}') for t in
54         range(n)]
55     d = [h.addVariable(ub=md, name=f'd_{t}') for t in range(n)]
56     w = [h.addVariable(ub=float(pv[t]), name=f'w_{t}') for t in range(n)]

```

```

56 e = [h.addVariable(lb=b['min_energy_kwh'], ub=b['max_energy_kwh'], name=f'E_{t}') for t in range(n
57 +1)]
58 z = [h.addVariable(ub=1, type=highspy.HighsVarType.kContinuous if relaxed else highspy.HighsVarType
59 .kInteger,
60 name=f'z_{t}') for t in range(n)]
61 h.addConstr(e[0] == b['initial_energy_kwh'], name='initial')
62 h.addConstr(e[n] == b['initial_energy_kwh'], name='terminal')
63 for t in range(n):
64     h.addConstr(g[t] - w[t] + d[t] - c[t] == float(load[t]-pv[t]), name=f'balance_{t}')
65     h.addConstr(e[t+1] - e[t] - b['eta_charge'] * c[t] + (1 / b['eta_discharge']) * d[t] == 0.,
66     name=f'state_{t}')
67     h.addConstr(c[t] <= mc * z[t], name=f'charge_mode_{t}')
68     h.addConstr(d[t] + md * z[t] <= md, name=f'discharge_mode_{t}')
69 if cost_cap is not None:
70     h.addConstr(sum(float(p[t]) * g[t] for t in range(n)) <= cost_cap, name='primary_cost_cap')
71 if tag:
72     h.writeModel(str(WORK / f'results/q1/{tag}.lp'))
73 h.run()
74 status = h.modelStatusToString(h.getModelStatus())
75 if h.getModelStatus() != highspy.HighsModelStatus.kOptimal:
76     raise RuntimeError(f'{tag}: {status}; see solver log; no optimal result published')
77 sol, info = h.getSolution(), h.getInfo()
78 values = lambda vs: np.array([sol.col_value[int(v)] for v in vs])
79 ev = values(e)
80 x = {'grid_kwh': values(g), 'charge_kwh': values(c), 'discharge_kwh': values(d),
81     'curtailment_kwh': values(w), 'energy_start_kwh': ev[:-1], 'energy_end_kwh': ev[1:],
82     'charge_mode': values(z)}
83 result = {'status': status, 'objective_yuan': float(np.dot(p, x['grid_kwh'])),
84     'solver_objective': h.getObjectiveValue(), 'relaxed': relaxed,
85     'lower_bound_yuan': h.getObjectiveValue() if relaxed else info.mip_dual_bound,
86     'mip_gap': None if relaxed else info.mip_gap,
87     'nodes': None if relaxed else info.mip_node_count, 'runtime_seconds': h.getRunTime(),
88     'variables': h.getNumCol(), 'constraints': h.getNumRow(),
89     'max_primal_infeasibility': info.max_primal_infeasibility,
90     'max_integrality_violation': None if relaxed else info.max_integrality_violation,
91     'lower_bound_unit': 'secondary_weighted_charge_objective' if cost_cap is not None else '
92     yuan',
93     'primary_cost_cap_yuan': cost_cap}
94 if tag:
95     h.writeSolution(str(WORK / f'results/q1/{tag}.sol'), 0)
96 return x, result
97
98 def export_plan(data, x, cfg, name, evidence):
99     out = WORK / 'results/q1' / name
100     out.mkdir(parents=True, exist_ok=True)
101     frame = data.copy()
102     for key, val in x.items():
103         frame[key] = val
104     frame['cost_yuan'] = frame.price_yuan_per_kwh * frame.grid_kwh
105     frame.to_csv(out / 'schedule.csv', index=False)
106     pd.DataFrame({'minute': np.arange(145)*10,
107         'energy_kwh': np.r_[frame.energy_start_kwh.iloc[0], frame.energy_end_kwh]}).to_csv(
108         out / 'states.csv', index=False)
109     summary = {key: float(frame[key].sum()) for key in ['cost_yuan', 'grid_kwh', 'charge_kwh',
110         'discharge_kwh', 'curtailment_kwh']}
111     summary.update(initial_energy_kwh=float(frame.energy_start_kwh.iloc[0]),
112         terminal_energy_kwh=float(frame.energy_end_kwh.iloc[-1]),
113         status=cfg['status'], scenario=name)
114     table1 = frame.loc[frame.start_minute.isin([600,720,840,960,1080,1200]), ['start_minute',
115         'end_minute', 'grid_kwh']]
116     table1.to_csv(out / 'table1_intervals.csv', index=False)
117     table2 = frame.assign(block_start=frame.start_minute//240*240).groupby('block_start')[['charge_kwh',
118         'discharge_kwh']].sum().reset_index()
119     table2['block_end'] = table2.block_start+240
120     table2[['block_start', 'block_end', 'charge_kwh', 'discharge_kwh']].to_csv(out / 'table2_blocks.csv',
121         index=False)
122     save_json(out / 'summary.json', summary)
123     save_json(out / 'config_snapshot.json', cfg)
124     save_json(out / 'solver_status.json', evidence)

```

```

117     return frame, summary
118
119
120 def main():
121     parser = argparse.ArgumentParser(description=__doc__)
122     parser.add_argument('--config', type=Path, default=WORK / 'configs/model_baseline.json')
123     args = parser.parse_args()
124     cfg = json.loads(args.config.read_text())
125     for folder in ['logs/q1', 'results/q1']:
126         (WORK / folder).mkdir(parents=True, exist_ok=True)
127     source = WORK / cfg['input']
128     data = pd.read_csv(source, float_precision='round_trip')
129     if (len(data) != 144 or data.start_minute.tolist() != list(range(0,1440,10))
130         or data.end_minute.tolist() != list(range(10,1441,10))):
131         raise ValueError('Q1 must preserve the reviewed 144 interval time keys')
132     fields = ['price_yuan_per_kwh', 'load_kwh', 'pv_forecast_kwh']
133     if not np.isfinite(data[fields]).all().all() or (data[fields] < 0).any().any():
134         raise ValueError('Nonfinite or negative input')
135     if cfg['interval_minutes'] != 10:
136         raise ValueError('The reviewed Q1 CSV uses 10 minute intervals')
137     preserved = sorted((WORK / 'data/processed').glob('*.csv')) + sorted((WORK / 'raw').glob('*.xlsx'))
138     before = {str(p.relative_to(WORK)): digest(p) for p in preserved}
139     save_json(WORK / 'logs/q1/input_hashes_before.json', before)
140     save_json(WORK / 'logs/q1/environment.json', {'python':sys.version, 'executable':Path(sys.
141         executable).name,
142         'platform':platform.platform(), 'highs':highspy.Highs().version(),
143         'packages':{n:importlib.metadata.version(n) for n in ['numpy','pandas','highspy','
144             matplotlib','openpyxl']})
145     summaries, frames = {}, {}
146     for name in ['baseline', 'roundtrip_90']:
147         current = copy.deepcopy(cfg)
148         if name == 'roundtrip_90':
149             current['battery'].update(eta_charge=cfg['sensitivity']['eta_charge'], eta_discharge=cfg['
150                 sensitivity']['eta_discharge'],
151                 efficiency_basis='roundtrip_0.9
152                 _symmetric_working_assumption_not_official')
153             x, evidence = solve_model(data, current, tag=name+'_milp')
154             xl, lower = solve_model(data, current, relaxed=True, tag=name+'_lp')
155             evidence['lp_lower_bound_yuan'] = lower['objective_yuan']
156             evidence['milp_minus_lp_yuan'] = evidence['objective_yuan']-lower['objective_yuan']
157             frames[name], summaries[name] = export_plan(data, x, current, name, evidence)
158             export_plan(data, xl, current, name+'_lp', lower)
159         n = len(data)
160         net = data.load_kwh-data.pv_forecast_kwh
161         zero = np.zeros(n)
162         no_battery = {'grid_kwh': np.maximum(net,0), 'charge_kwh':zero, 'discharge_kwh':zero,
163             'curtailment_kwh':np.maximum(-net,0), 'energy_start_kwh':np.full(n,cfg['battery']['
164                 initial_energy_kwh']),
165             'energy_end_kwh':np.full(n,cfg['battery']['initial_energy_kwh']), 'charge_mode':zero}
166         _, summaries['no_storage'] = export_plan(data,no_battery,cfg,'no_storage',{'status':'Analytic
167             feasible baseline'})
168         # Cost-face probe supplies a concrete alternative when the optimum is non-unique.
169         cap = summaries['baseline']['cost_yuan'] + 1e-7
170         alternate, status = solve_model(data,cfg,tag='alternate_milp',cost_cap=cap)
171         alt_frame, _ = export_plan(data,alternate,cfg,'alternate',status)
172         keys = ['grid_kwh','charge_kwh','discharge_kwh','energy_end_kwh']
173         save_json(WORK / 'results/q1/nonuniqueness.json', {
174             'cost_cap_yuan':cap, 'alternate_cost_yuan':float(alt_frame.cost_yuan.sum()),
175             'cost_difference_yuan':float(alt_frame.cost_yuan.sum()-summaries['baseline']['cost_yuan']),
176             'max_plan_difference_kwh':float(np.max(np.abs(alt_frame[keys].to_numpy()-frames['baseline'][
177                 keys].to_numpy()))),
178             'interpretation':'cost-face witness within numerical tolerance; no claim of unique dispatch'})
179         comparison = pd.DataFrame(summaries.values())
180         comparison['saving_yuan_vs_no_storage'] = summaries['no_storage']['cost_yuan'] - comparison.
181             cost_yuan
182         comparison['saving_percent_vs_no_storage'] = comparison.saving_yuan_vs_no_storage / summaries['
183             no_storage']['cost_yuan']*100
184         comparison.to_csv(WORK / 'results/q1/comparison.csv', index=False)
185         sensitivity = {'eta_baseline':0.9, 'eta_alternative':cfg['sensitivity']['eta_charge'],

```

```

177         'cost_change_yuan': summaries['roundtrip_90']['cost_yuan'] - summaries['baseline']['cost_yuan']]
178     for key in keys:
179         delta = frames['roundtrip_90'][key] - frames['baseline'][key]
180         sensitivity[key+'_l1_change'] = float(delta.abs().sum())
181         sensitivity[key+'_max_change'] = float(delta.abs().max())
182     save_json(WORK / 'results/q1/efficiency_sensitivity.json', sensitivity)
183     after = {str(p.relative_to(WORK)): digest(p) for p in preserved}
184     save_json(WORK / 'logs/q1/input_hashes_after.json', after)
185     if before != after:
186         raise RuntimeError('A protected input changed during this run')
187     print(comparison[['scenario', 'cost_yuan', 'grid_kwh', 'saving_percent_vs_no_storage']].to_string(
188         index=False))
189
190 if __name__ == '__main__':
191     main()

```

Program 4. scripts/run_q2.py

```

1  """Causal Q2 baseline: historical forecasts, daily MILP, fixed commitments and actual replay."""
2  import copy
3  import hashlib
4  import json
5  import os
6  from pathlib import Path
7  import sys
8
9  WORK=Path(__file__).resolve().parents[1]
10 sys.dont_write_bytecode=True
11 import numpy as np
12 import pandas as pd
13 from solve_q1 import solve_model
14
15 for key in ['TMPDIR', 'TMP', 'TEMP']:
16     os.environ[key]=str(WORK/'data/interim/q2')
17
18
19 def write_json(path,obj):
20     path.parent.mkdir(parents=True,exist_ok=True)
21     path.write_text(json.dumps(obj,ensure_ascii=False,indent=2,allow_nan=False)+'\n')
22
23
24 def fourier(k):
25     time=np.arange(144)/144
26     return np.column_stack([np.ones(144)]+[f(2*np.pi*j*time) for j in range(1,k+1) for f in [np.sin,np.cos]])
27
28
29 def forecast_day(history,day,kind,cfg):
30     day=pd.Timestamp(day)
31     if len(history) and (pd.to_datetime(history.interval_end)>day).any():
32         raise ValueError('future observation in forecast history')
33     if len(history)==0:
34         return np.full(144,np.nan),np.full(144,np.nan),{'method':'cold_start_zero_commitment','history_end':None}
35     if len(history)%144:
36         raise ValueError('forecast history requires complete days')
37     dates=pd.to_datetime(history.date.unique())
38     loads=history.load_actual_kwh.to_numpy().reshape(-1,144)
39     pvs=history.pv_actual_kwh.to_numpy().reshape(-1,144)
40     n=len(loads)
41     load=loads[-7 if n>=7 else -1].copy()
42     pv=pvs[-1].copy()
43     meta={'method':kind,'history_end':str(pd.Timestamp(history.interval_end.iloc[-1])),
44         'load_method':'lag7' if n>=7 else 'lag1','pv_method':'lag1'}
45     if kind!='seasonal' and n>=7+cfg['min_load_training_days']:
46         f=fourier(cfg['load_harmonics'])
47         def design(i,date):
48             weekday=np.tile([float(date.weekday()==k) for k in range(1,7)],(144,1))

```



```

49         return np.column_stack([f, loads[i-1]/1000, loads[i-7]/1000, weekday])
50     first=max(7, n-cfg['load_training_days'])
51     x=np.vstack([design(i, dates[i]) for i in range(first, n)])
52     y=loads[first:].ravel()
53     beta, _, rank, s=np.linalg.lstsq(x, y, rcond=None)
54     load=np.maximum(design(n, day)@beta, 0)
55     meta.update(load_method='OLS_lag1_lag7_weekday_fourier', load_coefficients=beta.tolist(),
56                load_rank=int(rank), load_columns=x.shape[1], load_condition=float(s[0]/s[-1]),
57                load_training_first=str(dates[first].date()), load_training_last=str(dates[-1].date
    ()))
58     if kind=='linear_harmonic' and n>=2:
59         window=min(n, cfg['pv_training_days'])
60         f=fourier(cfg['pv_harmonics'])
61         x=np.tile(f, (window, 1))
62         y=pvs[-window:].ravel()
63         beta=np.linalg.lstsq(x, y, rcond=None)[0]
64         residual=y-x@beta
65         denom=float(residual[-1]@residual[-1])
66         phi=float(np.clip((residual[-1]@residual[1:])/denom if denom>1e-12 else 0., -cfg['ar1_bound'],
    cfg['ar1_bound']))
67         pv=np.maximum(f@beta+phi*np.arange(1, 145)*residual[-1], 0)
68         meta.update(pv_method='rolling_harmonic_AR1', pv_coefficients=beta.tolist(), pv_phi=phi,
69                   pv_last_residual=float(residual[-1]), pv_training_first=str(dates[-window].date()),
70                   pv_training_last=str(dates[-1].date()))
71     return load, pv, meta
72
73
74 def execute_interval(grid, load, pv, energy, eta_c, eta_d, storage=True):
75     net=grid+pv-load
76     c=d=emergency=surplus=0.
77     if net>=0:
78         c=min(net, 5000/6, max(0., (10800-energy)/eta_c)) if storage else 0.
79         surplus=net-c
80     else:
81         d=min(-net, 5000/6, max(0., eta_d*(energy-1200))) if storage else 0.
82         emergency=-net-d
83     # Bookkeeping convention: use PV first; disposal of already-paid grid energy is separate.
84     unused_grid=min(grid, surplus)
85     return {'charge_actual_kwh':c, 'discharge_actual_kwh':d, 'emergency_kwh':emergency,
86           'surplus_kwh':surplus, 'unused_grid_kwh':unused_grid, 'pv_curtailment_kwh':surplus-
    unused_grid,
87           'energy_start_actual_kwh':energy, 'energy_end_actual_kwh':energy+eta_c*c-d/eta_d}
88
89
90 def summarize(frame):
91     d=frame.copy()
92     d['load_abs_error_kwh']=(d.load_forecast_kwh-d.load_actual_kwh).abs()
93     d['pv_abs_error_kwh']=(d.pv_forecast_kwh-d.pv_actual_kwh).abs()
94     sums=['grid_plan_kwh', 'emergency_kwh', 'planned_cost_yuan', 'emergency_cost_yuan', 'total_cost_yuan',
95          'charge_actual_kwh', 'discharge_actual_kwh', 'surplus_kwh', 'unused_grid_kwh',
    'pv_curtailment_kwh']
96     daily=d.groupby('date')[sums].sum()
97     daily['energy_start_actual_kwh']=d.groupby('date').energy_start_actual_kwh.first()
98     daily['energy_end_actual_kwh']=d.groupby('date').energy_end_actual_kwh.last()
99     daily['load_mae_kwh']=d.groupby('date').load_abs_error_kwh.mean()
100    daily['pv_mae_kwh']=d.groupby('date').pv_abs_error_kwh.mean()
101    daily['emergency_intervals']=d.assign(count=d.emergency_kwh>1e-6).groupby('date')['count'].sum()
102    return daily.reset_index()
103
104
105 def save_run(folder, frame, meta):
106     folder.mkdir(parents=True, exist_ok=True)
107     frame.to_csv(folder/'ledger.csv', index=False)
108     daily=summarize(frame)
109     daily.to_csv(folder/'daily.csv', index=False)
110     write_json(folder/'models_and_solvers.json', meta)
111     totals={key:float(frame[key].sum()) for key in ['grid_plan_kwh', 'emergency_kwh', 'planned_cost_yuan',
    'emergency_cost_yuan', 'total_cost_yuan', 'surplus_kwh', 'unused_grid_kwh', 'pv_curtailment_kwh
    ']}

```

```

113 totals.update(days=int(frame.date.nunique()),intervals=len(frame),
114               initial_energy_kwh=float(frame.energy_start_actual_kwh.iloc[0]),
115               final_energy_kwh=float(frame.energy_end_actual_kwh.iloc[-1]))
116 for source in ['load','pv']:
117     error=frame[source+'_forecast_kwh']-frame[source+'_actual_kwh']
118     totals[source+'_mae_kwh']=float(error.abs().mean())
119     totals[source+'_rmse_kwh']=float(np.sqrt((error**2).mean()))
120 positive=frame.pv_actual_kwh>0
121 totals['pv_positive_actual_mae_kwh']=float((frame.loc[positive,'pv_forecast_kwh']-frame.loc[
122     positive,'pv_actual_kwh']).abs().mean())
122 write_json(folder/'summary.json',totals)
123 return totals
124
125
126 def main():
127     cfg=json.loads((WORK/'configs/q2_baseline.json').read_text())
128     physical=json.loads((WORK/'configs/model_baseline.json').read_text())
129     assert physical['battery']['min_energy_kwh']==1200 and physical['battery']['max_energy_kwh']==10800
130     assert physical['battery']['max_charge_kw']==physical['battery']['max_discharge_kw']==5000
131     out=WORK/'results/q2'
132     out.mkdir(parents=True,exist_ok=True)
133     files=[WORK/'data/processed/actual_10min.csv',WORK/'data/processed/fixed_price.csv',
134           WORK/'configs/q2_baseline.json',WORK/'configs/model_baseline.json',WORK/'scripts/solve_q1.py',
135           WORK/'scripts/run_q2.py']
136     before={str(f.relative_to(WORK)):hashlib.sha256(f.read_bytes()).hexdigest() for f in files}
137     write_json(out/'input_code_hashes.json',before)
138     write_json(out/'config_snapshot.json',cfg)
139     write_json(out/'physical_snapshot.json',physical)
140     fields=['date','slot_id','interval_start','interval_end','available_time','load_actual_kwh','
141     pv_actual_kwh']
142     actual=pd.read_csv(files[0],usecols=fields,float_precision='round_trip')
143     if len(actual)!=52560 or actual.groupby('date').size().ne(144).any(): raise ValueError('Incomplete
144     annual data')
145     price=pd.read_csv(files[1],float_precision='round_trip').price_yuan_per_kwh.to_numpy()
146     actual['interval_end']=pd.to_datetime(actual.interval_end)
147     actual['interval_start']=pd.to_datetime(actual.interval_start)
148     dates=pd.date_range('2025-01-01','2025-12-31')
149     cache={}
150     eta_c=physical['battery']['eta_charge']
151     eta_d=physical['battery']['eta_discharge']
152     def run(kind,start,end,initial,storage=True):
153         energy=float(initial)
154         frames=[]
155         models=[]
156         for day in pd.date_range(start,end):
157             index=(day-dates[0]).days
158             current=actual.iloc[index*144:(index+1)*144].reset_index(drop=True).copy()
159             key=(kind,str(day.date()))
160             if key not in cache:
161                 # Data boundary: only completed observations enter prediction.
162                 history=actual.iloc[:index*144].copy()
163                 cache[key]=forecast_day(history,day,kind,cfg)
164                 load,pv,model=cache[key]
165                 model=copy.deepcopy(model)
166                 params=copy.deepcopy(physical)
167                 params['battery']['initial_energy_kwh']=energy
168                 if index==0 or not storage:
169                     grid=np.zeros(144) if index==0 else np.maximum(load-pv,0)
170                     plan={'grid_kwh':grid,'charge_kwh':np.zeros(144),'discharge_kwh':np.zeros(144),
171                         'energy_start_kwh':np.full(144,energy),'energy_end_kwh':np.full(144,energy),
172                         'curtailment_kwh':np.zeros(144) if index==0 else np.maximum(pv-load,0),'
173                         charge_mode':np.zeros(144)}
174                     status={'status':'Cold start zero commitment' if index==0 else 'Analytic no storage',
175                             'runtime_seconds':0.}
176                 else:
177                     data=pd.DataFrame({'price_yuan_per_kwh':price,'load_kwh':load,'pv_forecast_kwh':pv})
178                     plan,status=solve_model(data,params)
179                     current['issue_time']=str(day)
180                     current['price_yuan_per_kwh']=price

```

```

177         current['load_forecast_kwh']=load
178         current['pv_forecast_kwh']=pv
179         for key,column in {'grid_kwh':'grid_plan_kwh','charge_kwh':'charge_plan_kwh','discharge_kwh':
180             'discharge_plan_kwh',
181             'energy_start_kwh':'energy_start_plan_kwh','energy_end_kwh':'
182             energy_end_plan_kwh',
183             'curtailment_kwh':'pv_curtailment_plan_kwh','charge_mode':'
184             charge_mode_plan'}.items():
185             current[column]=plan[key]
186             events=[]
187             for slot in range(144):
188                 event=execute_interval(max(0.,plan['grid_kwh'][slot]),current.load_actual_kwh.iloc[slot
189 ],
190                                     current.pv_actual_kwh.iloc[slot],energy,eta_c,eta_d,storage)
191                 energy=event['energy_end_actual_kwh']
192                 events.append(event)
193                 for key in events[0]: current[key]=[e[key] for e in events]
194                 current['planned_cost_yuan']=price*current.grid_plan_kwh
195                 current['emergency_cost_yuan']=cfg['emergency_price_multiplier']*price*current.
196                 emergency_kwh
197                 current['total_cost_yuan']=current.planned_cost_yuan+current.emergency_cost_yuan
198                 frames.append(current)
199                 models.append({'date':str(day.date()),'forecast':model,'solver':status,'initial_energy_kwh':
200                 :float(params['battery']['initial_energy_kwh'])})
201                 if day.day==1: print(kind,'storage' if storage else 'no_storage',day.date(), 'SOC',round(
202                 energy,3),flush=True)
203                 return pd.concat(frames,ignore_index=True),models
204             warm,meta=run('seasonal', '2025-01-01', '2025-01-31', 6000.)
205             save_run(out/'warmup', warm,meta)
206             jan15=warm.loc[warm.date=='2025-01-15', 'energy_start_actual_kwh'].iloc[0]
207             initial_feb=float(warm.energy_end_actual_kwh.iloc[-1])
208             unit_value=float(price.mean()*eta_d)
209             calibration=[]
210             for kind in cfg['candidates']:
211                 frame,meta=run(kind,cfg['calibration_start'],cfg['calibration_end'],jan15)
212                 total=save_run(out/'calibration'/kind,frame,meta)
213                 calibration.append({'candidate':kind,**total,'inventory_adjusted_score_yuan':total['
214                 total_cost_yuan']-unit_value*(total['final_energy_kwh']-total['initial_energy_kwh'])})
215             cal=pd.DataFrame(calibration)
216             cal.to_csv(out/'calibration/comparison.csv',index=False)
217             selected=str(cal.loc[cal.inventory_adjusted_score_yuan.idxmin(), 'candidate'])
218             write_json(out/'selection.json', {'selected':selected, 'frozen_at': '2025-02-01T00:00:00',
219                 'calibration_last_available_time': '2025-02-01T00:00:00', 'inventory_value_yuan_per_kwh':
220                 unit_value,
221                 'common_evaluation_initial_energy_kwh':initial_feb, 'selection_criterion':cfg['selection'
222                 ]})
223             print('Frozen candidate:',selected, 'common Feb1 SOC:',initial_feb,flush=True)
224             comparisons=[]
225             for policy,kind,storage in [('selected',selected,True),('seasonal','seasonal',True),('no_storage',
226                 selected,False)]:
227                 frame,meta=run(kind,cfg['evaluation_start'],cfg['evaluation_end'],initial_feb,storage)
228                 total=save_run(out/policy,frame,meta)
229                 total['inventory_adjusted_cost_yuan']=total['total_cost_yuan']-unit_value*(total['
230                 final_energy_kwh']-initial_feb)
231                 comparisons.append({'policy':policy,'forecast_model':kind,**total})
232                 # Full emergency-event export, plus the four dates and Q1-format tables required by Q2.
233                 mask=frame.emergency_kwh>1e-6
234                 events=frame.loc[mask, ['date', 'interval_start', 'interval_end', 'emergency_kwh']].copy()
235                 events.to_csv(out/policy/'emergency_intervals.csv',index=False)
236                 representative=frame.loc[frame.date.isin(['2025-03-20', '2025-06-21', '2025-09-23', '2025-12-21'])
237                 ].copy()
238                 representative.loc[representative.slot_id.isin([61,73,85,97,109,121]), ['date', 'interval_start',
239                 'interval_end', 'grid_plan_kwh']].to_csv(out/policy/'table1_representative.csv',index=False)
240                 blocks=representative.assign(block_start_minute=(representative.slot_id-1)//24*240).groupby(['
241                 date', 'block_start_minute'])[['charge_actual_kwh', 'discharge_actual_kwh']].sum().reset_index()
242                 blocks['block_end_minute']=blocks.block_start_minute+240
243                 blocks.to_csv(out/policy/'table2_representative.csv',index=False)
244                 events.loc[events.date.isin(representative.date.unique())].to_csv(out/policy/'
245                 table3_representative.csv',index=False)
246             pd.DataFrame(comparisons).to_csv(out/'comparison.csv',index=False)

```

```

231 after={str(f.relative_to(WORK)):hashlib.sha256(f.read_bytes()).hexdigest() for f in files}
232 assert before==after,'Input or implementation changed during the run'
233 write_json(out/'run_status.json',{ 'completed':True, 'inputs_unchanged':True, 'selected':selected,
234                                     'evaluation_days':334, 'q3_q4_executed':False})
235 print(pd.DataFrame(comparisons)[['policy', 'total_cost_yuan', 'emergency_kwh', 'final_energy_kwh']].
      to_string(index=False),flush=True)
236
237
238 if __name__=='__main__': main()

```

Program 5. scripts/storage_control_v6/compat.py

```

1  """Frozen function bodies; source provenance in SOURCE_PROVENANCE.json."""
2
3  import highspy
4  import numpy as np
5  import pandas as pd
6
7  def fourier(k):
8      time=np.arange(144)/144
9      return np.column_stack([np.ones(144)]+[f(2*np.pi*j*time) for j in range(1,k+1) for f in [np.sin,np.
      cos]])
10
11 def forecast_day(history,day,kind,cfg):
12     day=pd.Timestamp(day)
13     if len(history) and (pd.to_datetime(history.interval_end)>day).any():
14         raise ValueError('future observation in forecast history')
15     if len(history)==0:
16         return np.full(144,np.nan),np.full(144,np.nan),{'method':'cold_start_zero_commitment',
17     history_end':None}
18     if len(history)%144:
19         raise ValueError('forecast history requires complete days')
20     dates=pd.to_datetime(history.date.unique())
21     loads=history.load_actual_kwh.to_numpy().reshape(-1,144)
22     pvs=history.pv_actual_kwh.to_numpy().reshape(-1,144)
23     n=len(loads)
24     load=loads[-7 if n>=7 else -1].copy()
25     pv=pvs[-1].copy()
26     meta={'method':kind,'history_end':str(pd.Timestamp(history.interval_end.iloc[-1])),
27     'load_method':'lag7' if n>=7 else 'lag1','pv_method':'lag1'}
28     if kind!='seasonal' and n>=7+cfg['min_load_training_days']:
29         f=fourier(cfg['load_harmonics'])
30         def design(i,date):
31             weekday=np.tile([float(date.weekday()==k) for k in range(1,7)],(144,1))
32             return np.column_stack([f,loads[i-1]/1000,loads[i-7]/1000,weekday])
33         first=max(7,n-cfg['load_training_days'])
34         x=np.vstack([design(i,dates[i]) for i in range(first,n)])
35         y=loads[first:].ravel()
36         beta,_ ,rank,s=np.linalg.lstsq(x,y,rcond=None)
37         load=np.maximum(design(n,day)@beta,0)
38         meta.update(load_method='OLS_lag1_lag7_weekday_fourier',load_coefficients=beta.tolist(),
39     load_rank=int(rank),load_columns=x.shape[1],load_condition=float(s[0]/s[-1]),
40     load_training_first=str(dates[first].date()),load_training_last=str(dates[-1].date
41     ()))
42     if kind=='linear_harmonic' and n>=2:
43         window=min(n,cfg['pv_training_days'])
44         f=fourier(cfg['pv_harmonics'])
45         x=np.tile(f,(window,1))
46         y=pvs[-window:].ravel()
47         beta=np.linalg.lstsq(x,y,rcond=None)[0]
48         residual=y-x@beta
49         denom=float(residual[-1]@residual[-1])
50         phi=float(np.clip((residual[-1]@residual[1:])/denom if denom>1e-12 else 0.,-cfg['ar1_bound'],
51     cfg['ar1_bound']))
52         pv=np.maximum(f@beta+phi*np.arange(1,145)*residual[-1],0)
53         meta.update(pv_method='rolling_harmonic_AR1',pv_coefficients=beta.tolist(),pv_phi=phi,
54     pv_last_residual=float(residual[-1]),pv_training_first=str(dates[-window].date()),
55     pv_training_last=str(dates[-1].date()))
56     return load,pv,meta

```

```

55 def execute_interval(grid,load,pv,energy,eta_c,eta_d,storage=True):
56     net=grid+pv-load
57     c=d=emergency=surplus=0.
58     if net>=0:
59         c=min(net,5000/6,max(0.,(10800-energy)/eta_c)) if storage else 0.
60         surplus=net-c
61     else:
62         d=min(-net,5000/6,max(0.,eta_d*(energy-1200))) if storage else 0.
63         emergency=-net-d
64     # Bookkeeping convention: use PV first; disposal of already-paid grid energy is separate.
65     unused_grid=min(grid,surplus)
66     return {'charge_actual_kwh':c,'discharge_actual_kwh':d,'emergency_kwh':emergency,
67           'surplus_kwh':surplus,'unused_grid_kwh':unused_grid,'pv_curtailment_kwh':surplus-
        unused_grid,
68           'energy_start_actual_kwh':energy,'energy_end_actual_kwh':energy+eta_c*c-d/eta_d}
69
70 def contract_fee(original,final,price,rule):
71     delta=np.asarray(final)-np.asarray(original)
72     if rule not in ('A','B'):raise ValueError('Unknown settlement')
73     sign=-1 if rule=='A' else 1
74     return price*(original+1.5*np.maximum(delta,0)+sign*.5*np.maximum(-delta,0))
75
76 def solve_horizon(load,pv,price,initial,terminal,physical,original=None,rule='A',relaxed=False):
77     """Q1 physical MILP, with separately specified rolling initial and day-end target."""
78     n=len(load);b=physical['battery'];dt=physical['interval_minutes']/60
79     mc=b['max_charge_kw']*dt;md=b['max_discharge_kw']*dt
80     h=highspy.Highs();h.setOptionValue('output_flag',False)
81     for key,value in physical['solver'].items():
82         if key!='name':
83             if h.setOptionValue(key,value)!=highspy.HighsStatus.kOk:raise ValueError(key)
84     q=[h.addVariable(lb=float(original[t]) if original is not None and rule=='B' else 0,
85                   obj=float(price[t]) if original is None else 0,name=f'q_{t}') for t in range(n)]
86     c=[h.addVariable(ub=mc,name=f'c_{t}') for t in range(n)]
87     d=[h.addVariable(ub=md,name=f'b_{t}') for t in range(n)]
88     # Surplus includes PV and paid grid energy: required for the B no-refund case.
89     w=[h.addVariable(name=f'w_{t}') for t in range(n)]
90     e=[h.addVariable(lb=b['min_energy_kwh'],ub=b['max_energy_kwh'],name=f'E_{t}') for t in range(n+1)]
91     z=[h.addVariable(ub=1,type=highspy.HighsVarType.kContinuous if relaxed else highspy.HighsVarType.
92                   kInteger,name=f'z_{t}') for t in range(n)]
93     h.addConstr(e[0]==initial);h.addConstr(e[-1]==terminal)
94     for t in range(n):
95         h.addConstr(q[t]+d[t]-c[t]-w[t]==float(load[t]-pv[t]))
96         h.addConstr(e[t+1]==e[t]+b['eta_charge']*c[t]-(1/b['eta_discharge'])*d[t])
97         h.addConstr(c[t]<=mc*z[t]);h.addConstr(d[t]<=md*(1-z[t]))
98         if original is not None:
99             v=h.addVariable(obj=1,name=f'fee_{t}')
100             p=float(price[t]);base=float(original[t])
101             h.addConstr(v>=1.5*p*q[t]-.5*p*base)
102             if rule=='A':h.addConstr(v>=.5*p*q[t]+.5*p*base)
103             elif rule=='B':h.addConstr(v>=-.5*p*q[t]+1.5*p*base)
104             else:raise ValueError('Unknown settlement')
105     h.run()
106     if h.getModelStatus()!=highspy.HighsModelStatus.kOptimal:
107         raise RuntimeError(h.getModelStatusToString(h.getModelStatus()))
108     solution=h.getSolution();info=h.getInfo()
109     values=lambda variables:np.array([solution.col_value[int(v)] for v in variables])
110     state=values(e)
111     plan={'grid_kwh':values(q),'charge_kwh':values(c),'discharge_kwh':values(d),'surplus_kwh':values(w),
112          'energy_start_kwh':state[:-1],'energy_end_kwh':state[1:], 'charge_mode':values(z)}
113     obj=float(np.dot(price,plan['grid_kwh']) if original is None else contract_fee(original,plan['
114     grid_kwh'],price,rule).sum())
115     status={'status':'Optimal','objective_yuan':obj,'solver_objective_yuan':h.getObjectiveValue(),
116           'lower_bound_yuan':h.getObjectiveValue() if relaxed else info.mip_dual_bound,
117           'mip_gap':None if relaxed else info.mip_gap,'runtime_seconds':h.getRuntime(),
118           'max_primal_infeasibility':info.max_primal_infeasibility,'relaxed':relaxed,
119           'variables':h.getNumCol(),'constraints':h.getNumRow()}
120     return plan,status
121
122 def forecast_price(history,issue,cfg):

```

```

121     """Only completed observations are accepted; future actual prices cannot be passed."""
122     issue=pd.Timestamp(issue);h=history.copy()
123     h['interval_start']=pd.to_datetime(h.interval_start);h['interval_end']=pd.to_datetime(h.
124         interval_end)
125     if h.interval_end.gt(issue).any():raise ValueError('future actual price in history')
126     if h.empty or h.interval_end.iloc[-1]!=issue:raise ValueError('Incomplete price history')
127     series=h.set_index('interval_start').actual_price_yuan_per_kwh
128     if series.index.has_duplicates or not series.index.equals(pd.date_range(series.index[0],issue,freq=
129         '10min',inclusive='left')):
130         raise ValueError('Noncontinuous price history')
131     start=max(series.index[0]+pd.Timedelta(days=7),issue-pd.Timedelta(days=config['price_training_days']))
132     train=pd.date_range(start,issue,freq='10min',inclusive='left')
133     if len(train)<144:raise ValueError('Insufficient price training history')
134     target=pd.date_range(issue,issue.normalize()+pd.Timedelta(days=1),freq='10min',inclusive='left')
135     def design(times):
136         phase=(times.hour*60+times.minute).to_numpy()/1440
137         cols=[np.ones(len(times))]
138         for k in range(1,config['price_harmonics']+1):cols.extend([np.sin(2*np.pi*k*phase),np.cos(2*np.pi*
139             k*phase)])
140         cols.extend([series.loc[times-pd.Timedelta(days=1)].to_numpy(),series.loc[times-pd.Timedelta(
141             days=7)].to_numpy()])
142         cols.extend([(times.weekday==k).astype(float) for k in range(1,7)])
143         return np.column_stack(cols)
144     x=design(train);y=series.loc[train].to_numpy();beta,_=rank,s=np.linalg.lstsq(x,y,rcond=None)
145     raw=design(target)@beta;floor=config['price_floor_yuan_per_kwh'];values=np.maximum(raw,floor)
146     meta={'method':'rolling_OLS','issue_time':str(issue),'training_first_start':str(train[0]),
147         'training_last_end':str(train[-1]+pd.Timedelta(minutes=10)),'training_rows':len(train),
148         'feature_count':x.shape[1],'rank':int(rank),'condition':float(s[0]/s[-1]) if s[-1]>0 else None,
149         'coefficients':beta.tolist(),'floor_yuan_per_kwh':floor,'floor_activations':int((raw<floor).sum
150             ()),
151         'raw_forecast_min':float(raw.min()),'raw_forecast_max':float(raw.max())}
152     return values,meta
153
154 def solve_target_model(data, cfg, relaxed=False, tag=None, cost_cap=None):
155     """Return a dispatch plus solver evidence; LP relaxes only mode integrality."""
156     n = len(data)
157     b = cfg['battery']
158     dt = cfg['interval_minutes'] / 60
159     mc, md = b['max_charge_kw'] * dt, b['max_discharge_kw'] * dt
160     h = highspy.Highs()
161     h.setOptionValue('output_flag', bool(tag))
162     if tag:
163         h.setOptionValue('log_file', str(WORK / f'logs/q1/{tag}.log'))
164     for k, v in cfg['solver'].items():
165         if k != 'name':
166             if h.setOptionValue(k, v) != highspy.HighsStatus.kOk:
167                 raise ValueError(f'Invalid solver option: {k}')
168     p = data.price_yuan_per_kwh.to_numpy()
169     load = data.load_kwh.to_numpy()
170     pv = data.pv_forecast_kwh.to_numpy()
171     g = [h.addVariable(obj=float(p[t]) if cost_cap is None else 0., name=f'g_{t}') for t in range(n)]
172     c = [h.addVariable(ub=mc, obj=float(t + 1) if cost_cap is not None else 0., name=f'c_{t}') for t in range(n)]
173     d = [h.addVariable(ub=md, name=f'd_{t}') for t in range(n)]
174     w = [h.addVariable(ub=float(pv[t]), name=f'w_{t}') for t in range(n)]
175     e = [h.addVariable(lb=b['min_energy_kwh'], ub=b['max_energy_kwh'], name=f'E_{t}') for t in range(n+1)]
176     z = [h.addVariable(ub=1, type=highspy.HighsVarType.kContinuous if relaxed else highsps.HighsVarType.kInteger,
177         name=f'z_{t}') for t in range(n)]
178     h.addConstr(e[0] == b['initial_energy_kwh'], name='initial')
179     h.addConstr(e[n] == b['terminal_target_kwh'], name='terminal')
180     for t in range(n):
181         h.addConstr(g[t] - w[t] + d[t] - c[t] == float(load[t]-pv[t]), name=f'balance_{t}')
182         h.addConstr(e[t+1] - e[t] - b['eta_charge'] * c[t] + (1 / b['eta_discharge']) * d[t] == 0.,
183             name=f'state_{t}')
184         h.addConstr(c[t] <= mc * z[t], name=f'charge_mode_{t}')
185         h.addConstr(d[t] + md * z[t] <= md, name=f'discharge_mode_{t}')
186     if cost_cap is not None:
187         h.addConstr(sum(float(p[t]) * g[t] for t in range(n)) <= cost_cap, name='primary_cost_cap')

```

```

182     if tag:
183         h.writeModel(str(WORK / f'results/q1/{tag}.lp'))
184     h.run()
185     status = h.modelStatusToString(h.getModelStatus())
186     if h.getModelStatus() != highs.HighsModelStatus.kOptimal:
187         raise RuntimeError(f'{tag}: {status}; see solver log; no optimal result published')
188     sol, info = h.getSolution(), h.getInfo()
189     values = lambda vs: np.array([sol.col_value[int(v)] for v in vs])
190     ev = values(e)
191     x = {'grid_kwh': values(g), 'charge_kwh': values(c), 'discharge_kwh': values(d),
192         'curtailment_kwh': values(w), 'energy_start_kwh': ev[:-1], 'energy_end_kwh': ev[1:],
193         'charge_mode': values(z)}
194     result = {'status': status, 'objective_yuan': float(np.dot(p, x['grid_kwh'])),
195             'solver_objective': h.getObjectiveValue(), 'relaxed': relaxed,
196             'lower_bound_yuan': h.getObjectiveValue() if relaxed else info.mip_dual_bound,
197             'mip_gap': None if relaxed else info.mip_gap,
198             'nodes': None if relaxed else info.mip_node_count, 'runtime_seconds': h.getRunTime(),
199             'variables': h.getNumCol(), 'constraints': h.getNumRow(),
200             'max_primal_infeasibility': info.max_primal_infeasibility,
201             'max_integrality_violation': None if relaxed else info.max_integrality_violation,
202             'lower_bound_unit': 'secondary_weighted_charge_objective' if cost_cap is not None else '
    yuan',
203             'primary_cost_cap_yuan': cost_cap}
204     if tag:
205         h.writeSolution(str(WORK / f'results/q1/{tag}.sol'), 0)
206     return x, result

```

Program 6. scripts/storage_control_v6/inputs.py

```

1  """Causal forecast provenance; no legacy imports or writes."""
2  import json
3  from pathlib import Path
4  import numpy as np
5  import pandas as pd
6  from compat import forecast_price
7
8  WORK=Path(__file__).resolve().parents[2]
9  ROOT=WORK/'results/storage_control_v6'
10
11
12  def read(path): return pd.read_csv(path,float_precision='round_trip')
13
14
15  class Inputs:
16      def __init__(self,with_pv=True,actual=None,pv_records=None,price_cache=True):
17          self.actual=read(WORK/'data/processed/actual_10min.csv') if actual is None else actual.copy()
18          for c in ['interval_start','interval_end']:self.actual[c]=pd.to_datetime(self.actual[c])
19          self.days={d:f.reset_index(drop=True) for d,f in self.actual.groupby('date',sort=False)}
20          self.archive=read(WORK/'results/q2_direct_v4/forecasts/linear_harmonic.csv')
21          self.b0={d:f.reset_index(drop=True) for d,f in self.archive.groupby('date',sort=False)}
22          self.fixed=read(WORK/'data/processed/fixed_price.csv').price_yuan_per_kwh.to_numpy()
23          self.physical=json.loads((WORK/'configs/model_baseline.json').read_text())
24          self.price_config=json.loads((WORK/'configs/q4_baseline.json').read_text())
25          self.price_cache={};self.price_meta={};self.pv_cache={};self.with_pv=with_pv
26          if price_cache and (ROOT/'inputs/price_forecasts.csv').exists():
27              prices=read(ROOT/'inputs/price_forecasts.csv')
28              self.price_cache={pd.Timestamp(k):f.price_forecast_yuan_per_kwh.to_numpy() for k,f in
prices.groupby('issue_time',sort=False)}
29              self.price_meta={pd.Timestamp(k):v for k,v in json.loads((ROOT/'inputs/price_models.json').
read_text()).items()}
30              if with_pv:
31                  self.pv=read(WORK/'data/processed/pv_forecast_10min.csv') if pv_records is None else
pv_records.copy()
32                  for c in ['issue_time','interval_start','interval_end']:self.pv[c]=pd.to_datetime(self.pv[c]
])
33                  self.pv=self.pv.loc[self.pv.interval_end<=self.pv.issue_time.dt.normalize()+pd.Timedelta(
days=1)].copy()
34                  self.issues={k:f.sort_values('interval_start').reset_index(drop=True) for k,f in self.pv.
groupby('issue_time',sort=False)}

```

```

35         hist=self.pv.loc[self.pv.pv_forecast_kwh>0].merge(self.actual[['interval_start',
36         pv_actual_kwh']],on='interval_start',validate='many_to_one')
37         hist['residual_kwh']=hist.pv_actual_kwh-hist.pv_forecast_kwh
38         self.pv_history=hist
39
40     def pv_forecast(self,date,hour,corrected):
41         key=(date,hour,corrected)
42         if key in self.pv_cache:return self.pv_cache[key]
43         issue=pd.Timestamp(date)+pd.Timedelta(hours=hour);f=self.issues[issue]
44         wanted=pd.date_range(issue,issue.normalize()+pd.Timedelta(days=1),freq='10min',inclusive='left',
45         )
46         assert np.array_equal(f.interval_start.to_numpy(),wanted.to_numpy())
47         raw=f.pv_forecast_kwh.to_numpy();used=raw.copy();bias=0.;count=0;latest=None
48         active=corrected and date>='2025-04-01'
49         if active:
50             cut=issue.normalize();hist=self.pv_history
51             eligible=hist.loc[(hist.issue_time<issue)&(hist.issue_time.dt.hour==hour)&(hist.
52             interval_start>=cut-pd.Timedelta(days=28))&(hist.interval_end<=cut)]
53             count=len(eligible);bias=float(eligible.residual_kwh.mean()) if count else 0.
54             latest=str(eligible.interval_end.max()) if count else None
55             used=np.where(raw>0,np.maximum(0,raw+bias),0.)
56             meta={'issue_time':str(issue),'active':active,'bias_kwh':bias,'training_rows':count,'
57             training_cutoff':str(issue.normalize()),'latest_target_end':latest}
58             self.pv_cache[key]=(used,raw,meta)
59             return self.pv_cache[key]
60
61     def prices(self,issue,method):
62         issue=pd.Timestamp(issue);slot=issue.hour*6
63         if method=='fixed':return self.fixed[slot:],{'method':'fixed','issue_time':str(issue)}
64         if issue not in self.price_cache:
65             h=self.actual.loc[self.actual.interval_end<=issue,['interval_start','interval_end','
66             actual_price_yuan_per_kwh']]
67             self.price_cache[issue],self.price_meta[issue]=forecast_price(h,issue,self.price_config)
68             return self.price_cache[issue],self.price_meta[issue]
69
70     def scenarios(self,date,hour,source_hour,corrected):
71         day=pd.Timestamp(date);slot=hour*6;load=self.b0[date].load_forecast_kwh.to_numpy()[slot:]
72         if self.with_pv:
73             used,raw,meta=self.pv_forecast(date,source_hour,corrected)
74             used=used[slot-source_hour*6:];raw=raw[slot-source_hour*6:]
75         else:
76             used=self.b0[date].pv_forecast_kwh.to_numpy()[slot:];raw=used.copy();meta={'source':'B0','
77             issue_time':str(day)}
78         dates=[d for d in self.b0 if day-pd.Timedelta(days=112)<=pd.Timestamp(d)<day]
79         residuals=[]
80         for d in dates:
81             base=self.b0[d]
82             if self.with_pv:
83                 past_pv=self.pv_forecast(d,source_hour,corrected)[0][slot-source_hour*6:]
84                 past_load=base.load_forecast_kwh.to_numpy()[slot:]
85                 real=self.days[d]
86                 residual=(real.load_actual_kwh-real.pv_actual_kwh).to_numpy()[slot:]- (past_load-past_pv
87             )
88         else:residual=base.net_residual_kwh.to_numpy()[slot:]
89         residuals.append(residual)
90         residual=np.array(residuals)
91         assert len(dates)>=7 and np.all(np.isfinite(residual))
92         risk=residual[[pd.Timestamp(d)>=day-pd.Timedelta(days=28) for d in dates]]
93         return load,used,raw,load-used+residual,risk,dates,meta
94
95     def prepare():
96         target=ROOT/'inputs';target.mkdir(exist_ok=True)
97         data=Inputs(with_pv=False,price_cache=False);frames=[];metas={}
98         for day in pd.date_range('2025-02-01','2025-12-31'):
99             for hour in [0,6,12,18]:
100                 issue=day+pd.Timedelta(hours=hour);p,m=data.prices(issue,'ols');metas[str(issue)]=m
101                 frames.append(pd.DataFrame({'issue_time':str(issue),'slot_id':np.arange(hour*6+1,145),'
102                 price_forecast_yuan_per_kwh':p}))
103                 if day.day==1:print('price',day.date(),flush=True)

```



```

97     pd.concat(frames).to_csv(target/'price_forecasts.csv',index=False)
98     (target/'price_models.json').write_text(json.dumps(metas,indent=2)+'\n')
99
100
101 if __name__=='__main__':prepare()

```

Program 7. scripts/storage_control_v6/control.py

```

1  """Causal bus-side interval dispatch with a preissued storage reserve threshold."""
2
3
4  def execute_interval(grid, load, pv, energy, reserve, eta_c=.9, eta_d=.9):
5      """Balance this interval without charging from emergency electricity.
6
7      ‘‘reserve‘‘ limits deficit discharge; it is not an inventory target that must
8      be reached. An inventory already below reserve remains available for surplus
9      charging, but no energy is discharged until it exceeds reserve.
10     """
11     net = grid + pv - load
12     c = d = emergency = surplus = 0.
13     if net >= 0:
14         c = min(net, 5000 / 6, max(0., (10800 - energy) / eta_c))
15         surplus = net - c
16     else:
17         d = min(-net, 5000 / 6, max(0., eta_d * (energy - reserve)))
18         emergency = -net - d
19     unused_grid = min(grid, surplus)
20     return {'charge_actual_kwh': c, 'discharge_actual_kwh': d,
21           'emergency_kwh': emergency, 'surplus_kwh': surplus,
22           'unused_grid_kwh': unused_grid,
23           'pv_curtailment_kwh': surplus - unused_grid,
24           'energy_start_actual_kwh': energy,
25           'energy_end_actual_kwh': energy + eta_c * c - d / eta_d,
26           'reserve_threshold_kwh': reserve}

```

Program 8. scripts/storage_control_v6/kernel.py

```

1  """Strict float64 greedy regression and reserve-policy scenario objectives.
2
3  The compiled artifact and compiler temporary files are local to v6. The legacy
4  module is never imported, and no v5 runtime is read or written.
5  """
6  import ctypes
7  import hashlib
8  import json
9  import os
10 from pathlib import Path
11 import platform
12 import subprocess
13
14 import numpy as np
15
16 SOURCE = Path(__file__).with_suffix('.c')
17 WORK = SOURCE.parents[2]
18 RUNTIME = WORK / 'results/storage_control_v6/runtime'
19 RUNTIME.mkdir(parents=True, exist_ok=True)
20 digest = hashlib.sha256(SOURCE.read_bytes()).hexdigest()
21 build = RUNTIME / digest
22 build.mkdir(exist_ok=True)
23 binary = build / ('kernel.dylib' if platform.system() == 'Darwin' else 'kernel.so')
24 command = ['cc', '-O3', '-std=c99', '-dynamiclib' if platform.system() == 'Darwin' else '-shared',
25           '-fPIC', str(SOURCE), '-o', str(binary)]
26 if not binary.exists():
27     environment = dict(os.environ, TMPDIR=str(RUNTIME))
28     subprocess.run(command, check=True, capture_output=True, text=True, env=environment)
29     (build / 'build.json').write_text(json.dumps({
30         'command': command, 'source_sha256': digest,
31         'binary_sha256': hashlib.sha256(binary.read_bytes()).hexdigest(),
32         'compiler': subprocess.run(['cc', '--version'], check=True, capture_output=True,

```

```

33         text=True, env=environment).stdout}, indent=2) + '\n')
34 lib = ctypes.CDLL(str(binary))
35 vec = np.ctypeslib.ndpointer(dtype=np.float64, flags='C_CONTIGUOUS')
36 lib.legacy_objective.argtypes = [ctypes.c_int, ctypes.c_int, vec, vec, vec,
37     ctypes.c_double, ctypes.c_double, ctypes.c_double, vec, vec, vec]
38 lib.legacy_objective.restype = ctypes.c_double
39 lib.reserve_objective.argtypes = [ctypes.c_int, ctypes.c_int, vec, vec, vec, vec,
40     ctypes.c_double, ctypes.c_double, ctypes.c_double, vec, vec, vec, vec]
41 lib.reserve_objective.restype = ctypes.c_double
42
43
44 def _inputs(q, net, price, initial, kappa, mu):
45     q, net, price = [np.ascontiguousarray(x, dtype=np.float64) for x in (q, net, price)]
46     if (q.ndim != 1 or price.shape != q.shape or len(q) == 0 or net.ndim != 2
47         or len(net) == 0 or net.shape[1] != len(q)):
48         raise ValueError('q and price must have length T; net must have nonempty shape (S,T)')
49     if not all(np.all(np.isfinite(x)) for x in (q, net, price, initial, kappa, mu)):
50         raise ValueError('objective inputs must be finite')
51     return q, net, price
52
53
54 def legacy_evaluate(q, net, price, initial, kappa=5., mu=.38232):
55     """The old C arithmetic retained for regression and matched greedy searches."""
56     q, net, price = _inputs(q, net, price, initial, kappa, mu)
57     grad, fees, end = np.empty(len(q)), np.empty(len(net)), np.empty(len(net))
58     score = lib.legacy_objective(len(net), len(q), q, net, price, initial, kappa, mu,
59         grad, fees, end)
60     if not np.isfinite(score) or not np.all(np.isfinite(grad)):
61         raise FloatingPointError('legacy objective returned nonfinite score or gradient')
62     return {'score': score, 'gradient': grad, 'emergency_fee': fees, 'end_energy': end}
63
64
65 def evaluate(q, reserve, net, price, initial, kappa=5., mu=.38232):
66     """Evaluate a shared causal (q,R) policy and its piecewise derivatives.
67
68     At an E == R tie, the active reserve branch is used: the R derivative is
69     one-sided from decreasing R, and the E derivative from increasing E. Other
70     demand/power/space ties use the legacy branch ordering. These local choices
71     are not global optimality certificates for the nonsmooth objective.
72     """
73     q, net, price = _inputs(q, net, price, initial, kappa, mu)
74     reserve = np.ascontiguousarray(reserve, dtype=np.float64)
75     if (reserve.shape != q.shape or not np.all(np.isfinite(reserve))
76         or np.any(reserve < 1200.) or np.any(reserve > 10800.)):
77         raise ValueError('reserve must have length T and finite values in [1200,10800]')
78     grad, reserve_grad = np.empty(len(q)), np.empty(len(q))
79     fees, end = np.empty(len(net)), np.empty(len(net))
80     score = lib.reserve_objective(len(net), len(q), q, reserve, net, price, initial,
81         kappa, mu, grad, reserve_grad, fees, end)
82     if not np.isfinite(score) or not all(np.all(np.isfinite(x)) for x in (grad, reserve_grad)):
83         raise FloatingPointError('reserve objective returned nonfinite score or gradient')
84     return {'score': score, 'gradient': grad, 'reserve_gradient': reserve_grad,
85         'emergency_fee': fees, 'end_energy': end}

```

Program 9. scripts/storage_control_v6/kernel.c

```

1 #include <math.h>
2 #include <stdlib.h>
3
4 /* The function body is deliberately identical to the frozen v5 objective. */
5 double legacy_objective(int S, int T, const double *q, const double *net, const double *p,
6     double initial, double kappa, double mu, double *grad,
7     double *fees, double *end) {
8     double *buf = malloc(4 * (size_t)T * sizeof(double));
9     if (!buf) return NAN;
10    double *a=buf, *b=buf+T, *ue=buf+2*T, *uq=buf+3*T;
11    const double eta=.9, power=5000./6.;
12    double result=0.;
13    for (int t=0;t<T;t++) { grad[t]=p[t]; result+=q[t]*p[t]; }
14    for (int s=0;s<S;s++) {

```

```

15     double e=initial, fee=0.;
16     for (int t=0;t<T;t++) {
17         double v=q[t]-net[s*T+t];
18         ue[t]=0.; uq[t]=0.;
19         if (v>=0.) {
20             double space=fmax((10800.-e)/eta,0.);
21             double c=fmin(fmin(v,power),space);
22             if (v<=power && v<=space) {a[t]=1.; b[t]=eta;}
23             else if (power<=space) {a[t]=1.; b[t]=0.;}
24             else {a[t]=0.; b[t]=0.;}
25             e+=eta*c;
26         } else {
27             double deficit=-v, avail=fmax(eta*(e-1200.),0.);
28             double d=fmin(fmin(deficit,power),avail);
29             fee+=5.*p[t]*fmax(deficit-d,0.);
30             if (deficit<=power && deficit<=avail) {a[t]=1.; b[t]=1./eta;}
31             else if (power<=avail) {a[t]=1.; b[t]=0.; uq[t]=-1.;}
32             else {a[t]=0.; b[t]=0.; ue[t]=-eta; uq[t]=-1.;}
33             e-=d/eta;
34         }
35     }
36     fees[s]=fee; end[s]=e;
37     result+=(kappa/5.*fee-mu*(e-initial))/S;
38     double adjoint=-mu;
39     for (int t=T-1;t>=0;t--) {
40         grad[t]+=(kappa*p[t]*uq[t]+adjoint*b[t])/S;
41         adjoint=kappa*p[t]*ue[t]+adjoint*a[t];
42     }
43 }
44 free(buf);
45 return result;
46 }
47
48 double reserve_objective(int S, int T, const double *q, const double *reserve,
49                         const double *net, const double *p, double initial,
50                         double kappa, double mu, double *grad, double *rgrad,
51                         double *fees, double *end) {
52     double *buf = malloc(6 * (size_t)T * sizeof(double));
53     if (!buf) return NAN;
54     double *a=buf, *b=buf+T, *c=buf+2*T;
55     double *ue=buf+3*T, *uq=buf+4*T, *ur=buf+5*T;
56     const double eta=.9, power=5000./6.;
57     double result=0.;
58     for (int t=0;t<T;t++) {
59         grad[t]=p[t]; rgrad[t]=0.; result+=q[t]*p[t];
60     }
61     for (int s=0;s<S;s++) {
62         double e=initial, fee=0.;
63         for (int t=0;t<T;t++) {
64             double v=q[t]-net[s*T+t];
65             c[t]=0.; ue[t]=0.; uq[t]=0.; ur[t]=0.;
66             if (v>=0.) {
67                 double space=fmax((10800.-e)/eta,0.);
68                 double charge=fmin(fmin(v,power),space);
69                 if (v<=power && v<=space) {a[t]=1.; b[t]=eta;}
70                 else if (power<=space) {a[t]=1.; b[t]=0.;}
71                 else {a[t]=0.; b[t]=0.;}
72                 e+=eta*charge;
73             } else {
74                 double deficit=-v, avail=fmax(eta*(e-reserve[t]),0.);
75                 double discharge=fmin(fmin(deficit,power),avail);
76                 fee+=5.*p[t]*fmax(deficit-discharge,0.);
77                 if (deficit<=power && deficit<=avail) {
78                     a[t]=1.; b[t]=1./eta;
79                 } else if (power<=avail) {
80                     a[t]=1.; b[t]=0.; uq[t]=-1.;
81                 } else if (e>=reserve[t]) {
82                     /* Active reserve: E_next=R; left R / right E derivative. */
83                     a[t]=0.; b[t]=0.; c[t]=1.;
84                     ue[t]=-eta; uq[t]=-1.; ur[t]=eta;

```

```

85         } else {
86             /* Below R is feasible. No discharge or emergency charging. */
87             a[t]=1.; b[t]=0.; uq[t]=-1.;
88         }
89         e-=discharge/eta;
90     }
91 }
92 fees[s]=fee; end[s]=e;
93 result+=(kappa/5.*fee-mu*(e-initial))/S;
94 double adjoint=-mu;
95 for (int t=T-1;t>=0;t--) {
96     grad[t]+=(kappa*p[t]*uq[t]+adjoint*b[t])/S;
97     rgrad[t]+=(kappa*p[t]*ur[t]+adjoint*c[t])/S;
98     adjoint=kappa*p[t]*ue[t]+adjoint*a[t];
99 }
100 }
101 free(buf);
102 return result;
103 }

```

Program 10. scripts/storage_control_v6/policy.py

```

1  """Shared causal procurement/reserve candidates with matched finite search budgets."""
2  import json
3  import time
4  from pathlib import Path
5
6  import numpy as np
7  import pandas as pd
8  from scipy.optimize import minimize
9
10 from compat import solve_horizon, solve_target_model
11 from kernel import evaluate as core_evaluate, legacy_evaluate
12
13 WORK = Path(__file__).resolve().parents[2]
14 LABELS = ['profile_start', 'profile_incumbent', 'profile_final',
15           'point_start', 'point_incumbent', 'point_final',
16           'extra_profile_start', 'extra_profile_incumbent', 'extra_profile_final',
17           'extra_greedy_best_start', 'extra_greedy_best_incumbent', 'extra_greedy_best_final',
18           'keep_current']
19 LEGACY_LABELS = LABELS[:6] + ['keep_current']
20 OPTIONS = {'maxiter':180, 'maxfun':1600, 'maxls':30, 'maxcor':10, 'ftol':1e-10, 'gtol':1e-5}
21 FLOOR, CEILING = 1200., 10800.
22
23
24 def evaluate(q, reserve, net, price, initial, kappa=5., mu=.38232, q0=None, legacy=False):
25     """Rule A changes procurement cash/gradient, never the reserve gradient."""
26     if legacy:
27         if not np.all(np.asarray(reserve) == FLOOR):
28             raise ValueError('legacy scoring requires reserve=1200')
29         result = legacy_evaluate(q, net, price, initial, kappa, mu)
30         result['reserve_gradient'] = np.zeros_like(q, dtype=float)
31     else:
32         result = core_evaluate(q, reserve, net, price, initial, kappa, mu)
33         ordinary = float(np.dot(q, price))
34         if q0 is not None:
35             delta = np.asarray(q) - q0
36             contract = float(np.sum(price * (q0 + 1.5 * np.maximum(delta, 0) - .5 * np.maximum(-delta, 0)))
37         )
38         result['score'] += contract - ordinary
39         result['gradient'] += price * np.where(delta > 0, .5, np.where(delta < 0, -.5, 0.))
40         ordinary = contract
41         result['ordinary'] = ordinary
42     return result
43
44 def _selected(scores):
45     scores = np.asarray(scores)
46     if not np.isfinite(scores).all():
47         raise FloatingPointError('nonfinite retained candidate score')

```

```

48     return int(np.flatnonzero(scores <= scores.min() + 1e-8)[0])
49
50
51 def decision(load, pv, net, residual28, price, energy, slot, physical,
52             q0=None, current=None, current_reserve=None, control='reserve'):
53     if control not in ('legacy', 'greedy', 'reserve'):
54         raise ValueError('unknown storage control')
55     n = 144 - slot
56     if current is not None and current_reserve is None:
57         if control == 'reserve':
58             raise ValueError('current_reserve is required with a held reserve policy')
59             current_reserve = np.full(n, FLOOR)
60     if (current is None) != (current_reserve is None):
61         raise ValueError('current and current_reserve must be supplied together')
62     if current is not None and (np.shape(current) != (n,) or np.shape(current_reserve) != (n,)):
63         raise ValueError('held q/reserve horizon mismatch')
64     if control != 'reserve' and current_reserve is not None and not np.all(current_reserve == FLOOR):
65         raise ValueError('greedy/legacy held reserve must be 1200')
66     for value in (load, pv, net, residual28, price, q0, current, current_reserve):
67         if value is not None and not np.isfinite(value).all():
68             raise ValueError('nonfinite decision input')
69     if current is not None and (np.any(current < 0) or np.any(current_reserve < FLOOR) or np.any(
70         current_reserve > CEILING)):
71         raise ValueError('infeasible held q/reserve pair')
72
73     # Hourly risk quantiles initialize procurement; they do not constrain final q/R.
74     hours = np.arange(slot, 144) // 6
75     delta = np.empty(n)
76     for hour in np.unique(hours):
77         tau = .85 if 9 <= hour < 11 else .65 if 11 <= hour < 18 else .95 if 18 <= hour < 22 else .8
78         cols = hours == hour
79         delta[cols] = np.quantile(residual28[:, cols].ravel(), tau, method='linear')
80     risk_load = np.maximum(load + delta, 0.)
81     risk_pv = pv + np.maximum(-load - delta, 0.)
82     physical = json.loads(json.dumps(physical))
83     physical['battery'].update(initial_energy_kwh=float(energy), terminal_target_kwh=FLOOR)
84     initializer_failure = None
85     try:
86         if slot == 0:
87             profile, milp = solve_target_model(pd.DataFrame({'price_yuan_per_kwh':price,
88                 'load_kwh':risk_load, 'pv_forecast_kwh':risk_pv}), physical)
89         else:
90             profile, milp = solve_horizon(risk_load, risk_pv, price, energy, FLOOR,
91                 physical, original=q0, rule='A')
92     except RuntimeError as exc:
93         initializer_failure = str(exc)
94         start0 = np.maximum(load - pv, 0.)
95         profile = {}
96         milp = {'status':'failed', 'runtime_seconds':0., 'exception':str(exc)}
97
98     plans, reserves, logs = [], [], []
99     floor = np.full(n, FLOOR)
100
101 def search(start, reserve_start, joint_reserve, label):
102     incumbent = {'score':float('inf'), 'q':start.copy(), 'reserve':reserve_start.copy()}
103     trace = []
104     greedy_score = not joint_reserve
105     def unpack(x):
106         return x[:n] * 1000., FLOOR + x[n:] * 1000. if joint_reserve else floor.copy()
107     def objective(x):
108         q, reserve = unpack(x)
109         if not np.isfinite(q).all() or not np.isfinite(reserve).all():
110             raise FloatingPointError('nonfinite optimizer iterate')
111         result = evaluate(q, reserve, net, price, energy, q0=q0, legacy=greedy_score)
112         gradient = np.r_[result['gradient'], result['reserve_gradient']] if joint_reserve else
113         result['gradient']
114         if not np.isfinite(result['score']) or not np.isfinite(gradient).all():
115             raise FloatingPointError('nonfinite optimizer score/gradient')
116         trace.append(float(result['score']))

```

```

116         if result['score'] < incumbent['score']:
117             incumbent.update(score=float(result['score']), q=q.copy(), reserve=reserve.copy())
118             return result['score'] / 10000., gradient * .1
119         x0 = np.r_[start / 1000., (reserve_start - FLOOR) / 1000.] if joint_reserve else start / 1000.
120         bounds = [(0., None)] * n + [(0., (CEILING - FLOOR) / 1000.)] * n if joint_reserve else []
121         # Evaluate the start before entering scipy so exceptional exits always have a finite fallback.
122         initial_value = evaluate(start, reserve_start, net, price, energy, q0=q0, legacy=greedy_score)
123         if not np.isfinite(initial_value['score']):
124             raise FloatingPointError('nonfinite search start')
125         tic = time.perf_counter()
126         returned_problem = None
127         try:
128             opt = minimize(objective, x0, method='L-BFGS-B', jac=True, bounds=bounds, options=OPTIONS)
129             returned_q, returned_reserve = unpack(opt.x)
130             if not np.isfinite(returned_q).all() or not np.isfinite(returned_reserve).all():
131                 returned_problem = 'nonfinite optimizer returned candidate'
132             elif np.any(returned_q < -1e-7) or np.any(returned_reserve < FLOOR - 1e-7) or np.any(
133                 returned_reserve > CEILING + 1e-7):
134                 returned_problem = 'infeasible optimizer returned candidate'
135             else:
136                 returned_q = np.maximum(returned_q, 0.)
137                 returned_reserve = np.clip(returned_reserve, FLOOR, CEILING)
138                 returned_value = evaluate(returned_q, returned_reserve, net, price, energy, q0=q0,
139                 legacy=greedy_score)
140                 if not np.isfinite(returned_value['score']):
141                     returned_problem = 'nonfinite optimizer returned score'
142                 log = {'success':bool(opt.success), 'status':int(opt.status), 'message':str(opt.message),
143                     'nit':int(opt.nit), 'nfev':int(opt.nfev), 'njev':int(opt.njev), 'exception':None}
144                 except (ValueError, FloatingPointError, AssertionError) as exc:
145                     returned_problem = 'optimizer exception: ' + str(exc)
146                     log = {'success':False, 'status':-99, 'message':str(exc), 'nit':0,
147                         'nfev':len(trace), 'njev':len(trace), 'exception':str(exc)}
148                 if not np.isfinite(incumbent['score']):
149                     incumbent['score'] = float(initial_value['score'])
150                 if returned_problem is not None:
151                     returned_q, returned_reserve = incumbent['q'].copy(), incumbent['reserve'].copy()
152                     plans.extend([start.copy(), incumbent['q'].copy(), returned_q.copy()])
153                     reserves.extend([reserve_start.copy(), incumbent['reserve'].copy(), returned_reserve.copy()])
154                     log.update(label=label, joint_reserve=joint_reserve, options=OPTIONS.copy(),
155                         wall_seconds=time.perf_counter() - tic, evaluation_score_yuan=trace,
156                         incumbent_score_yuan=float(incumbent['score']), returned_candidate_replaced=
157                         returned_problem is not None,
158                         returned_candidate_problem=returned_problem)
159                     logs.append(log)
160
161         search(start0, floor, False, 'profile')
162         search(np.maximum(load - pv, 0.), floor, False, 'point')
163         legacy_scores = [evaluate(q, reserve, net, price, energy, q0=q0, legacy=True)['score']
164             for q, reserve in zip(plans, reserves)]
165         best_greedy = plans[_selected(legacy_scores)].copy()
166         if control != 'legacy':
167             nominal = np.asarray(profile.get('energy_end_kwh', floor))
168             reserve_seed = np.clip(nominal, FLOOR, CEILING) if control == 'reserve' else floor
169             search(start0, reserve_seed, control == 'reserve', 'extra_profile')
170             search(best_greedy, floor, control == 'reserve', 'extra_greedy_best')
171         if current is not None:
172             plans.append(current.copy())
173             reserves.append(current_reserve.copy())
174         values = [evaluate(q, reserve, net, price, energy, q0=q0, legacy=control == 'legacy')
175             for q, reserve in zip(plans, reserves)]
176         scores = np.array([v['score'] for v in values])
177         labels = (LEGACY_LABELS if control == 'legacy' else LABELS)[:len(plans)]
178         return {'q':np.array(plans), 'reserve':np.array(reserves), 'risk_delta':delta,
179             'score':scores, 'ordinary':np.array([v['ordinary'] for v in values]),
180             'emergency_fee':np.array([v['emergency_fee'] for v in values]),
181             'end_energy':np.array([v['end_energy'] for v in values]), 'candidate_labels':labels,
182             'selected_index':_selected(scores), 'optimizer':logs, 'milp_initializer':milp,
183             'initializer_failure':initializer_failure, 'initializer_plan':profile}

```

Program 11. scripts/storage_control_v6/run.py

```

1  """Twenty-two causal q/reserve policies; all output stays in v6, never overwritten."""
2  import argparse
3  import json
4  import os
5  import time
6  from pathlib import Path
7
8  import numpy as np
9  import pandas as pd
10
11  from inputs import Inputs, ROOT, WORK
12  from policy import decision, OPTIONS
13  from control import execute_interval
14
15  CONFIG = json.loads((WORK / 'configs/storage_control_v6/experiments.json').read_text())
16  POLICIES = CONFIG['policies']
17  SUMS = ['grid_original_kwh', 'grid_effective_kwh', 'emergency_kwh', 'original_cost_yuan',
18         'increase_cost_yuan', 'decrease_adjustment_yuan', 'contract_cost_yuan', 'emergency_cost_yuan',
19         'total_cost_yuan', 'charge_actual_kwh', 'discharge_actual_kwh', 'surplus_kwh',
20         'unused_grid_kwh', 'pv_curtailment_kwh']
21
22
23  def write(path, obj):
24      path.write_text(json.dumps(obj, ensure_ascii=False, indent=2, allow_nan=False) + '\n')
25
26
27  def run_one(name, stage='runs', start='2025-02-01', end='2025-12-31',
28             initial=7268.4231640740745, data=None, control=None):
29      cfg = dict(POLICIES[name])
30      if control is not None:
31          if control != 'legacy':
32              raise ValueError('only legacy regression can override the named control')
33          cfg['control'] = control
34      folder = ROOT / stage / name
35      folder.mkdir(parents=True, exist_ok=False)
36      (folder / 'evidence').mkdir()
37      runtime = ROOT / 'runtime'
38      runtime.mkdir(parents=True, exist_ok=True)
39      for key in ['TMPDIR', 'TMP', 'TEMP']:
40          os.environ[key] = str(runtime)
41      write(folder / 'config.json', dict(policy=name, **cfg, start=start, end=end,
42          initial_energy_kwh=initial, scenario_window=112, kappa=5, mu=.38232,
43          optimizer=OPTIONS, searches_per_issue=2 if cfg['control']=='legacy' else 4,
44          protocol=CONFIG['protocol'], reserve_bounds_kwh=[1200., 10800.],
45          settlement='A', time_horizon='remaining current day', prefix_freeze='both q and reserve'))
46      data = Inputs(with_pv=cfg['with_pv']) if data is None else data
47      if getattr(data, 'with_pv', cfg['with_pv']) != cfg['with_pv']:
48          raise ValueError('Inputs PV permission does not match policy')
49      energy = initial
50      frames, versions, metadata = [], [], []
51      tic = time.perf_counter()
52      for date in pd.date_range(start, end).strftime('%Y-%m-%d'):
53          rows = data.days[date]
54          q0 = current = reserve0 = current_reserve = None
55          events = []
56          issuehours = [0, 6, 12, 18] if cfg['updates'] else [0]
57          for slot in range(144):
58              if slot in [6 * h for h in issuehours]:
59                  hour = slot // 6
60                  source_hour = hour if cfg['pv_update'] else 0
61                  issue = pd.Timestamp(date) + pd.Timedelta(hours=hour)
62                  load, pv, raw, net, risk, dates, pvmeta = data.scenarios(date, hour, source_hour, cfg['corrected'])
63                  price, price_meta = data.prices(issue, cfg['price'])
64                  base = None if q0 is None else q0[slot:].copy()
65                  held = None if current is None else current[slot:].copy()
66                  held_reserve = None if current_reserve is None else current_reserve[slot:].copy()
67                  prefix_q = np.array([]) if current is None else current[:slot].copy()

```

```

68         prefix_reserve = np.array([]) if current_reserve is None else current_reserve[:slot].
copy()
69         d = decision(load, pv, net, risk, price, energy, slot, data.physical,
70                     q0=base, current=held, current_reserve=held_reserve, control=cfg['control'])
71         selected = int(d['selected_index'])
72         q, reserve = d['q'][selected], d['reserve'][selected]
73         if q0 is None:
74             q0, current = q.copy(), q.copy()
75             reserve0, current_reserve = reserve.copy(), reserve.copy()
76         else:
77             current[slot:] = q
78             current_reserve[slot:] = reserve
79             if not np.array_equal(current[:slot], prefix_q) or not np.array_equal(
current_reserve[:slot], prefix_reserve):
80                 raise AssertionError('executed q/reserve prefix changed')
81             relative = 'evidence/' + date + f'_{hour:02d}.npz'
82             arrays = {key:d[key] for key in ['q', 'reserve', 'risk_delta', 'score', 'ordinary', '
emergency_fee', 'end_energy', 'selected_index']}
83             arrays.update(scenario_net=net, price=price, load_hat=load, pv_hat=pv, raw_pv=raw,
84                          q0=np.array([]) if base is None else base,
85                          current=np.array([]) if held is None else held,
86                          current_reserve=np.array([]) if held_reserve is None else held_reserve,
87                          executed_q=prefix_q, executed_reserve=prefix_reserve)
88             arrays.update({'initializer_' + key:value for key, value in d['initializer_plan'].items
()})
89             np.savez_compressed(folder / relative, **arrays)
90             metadata.append({'date':date, 'issue_hour':hour, 'source_hour':source_hour,
91                            'correction':cfg['corrected'], 'price_method':cfg['price'], 'control':cfg['control']
],
92                            'initial_energy_kwh':float(energy), 'scenario_source_dates':dates,
93                            'scenario_count':len(dates), 'selected_index':selected,
94                            'candidate_labels':d['candidate_labels'], 'evidence_file':relative,
95                            'optimizer':d['optimizer'], 'milp_initializer':d['milp_initializer'],
96                            'initializer_failure':d['initializer_failure'], 'pv_metadata':pvmeta,
97                            'price_metadata':price_meta, 'issue_time':str(issue)})
98             versions.append(pd.DataFrame({'date':date, 'issue_hour':hour, 'issue_time':str(issue),
99                                          'slot_id':np.arange(slot + 1, 145),
100                                          'interval_start':rows.interval_start.iloc[slot:].to_numpy(),
101                                          'interval_end':rows.interval_end.iloc[slot:].to_numpy(),
102                                          'grid_original_kwh':q0[slot:], 'grid_kwh':q,
103                                          'reserve_original_kwh':reserve0[slot:], 'reserve_threshold_kwh':reserve,
104                                          'load_forecast_kwh':load, 'pv_forecast_kwh':pv, 'pv_raw_forecast_kwh':raw,
105                                          'price_forecast_yuan_per_kwh':price, 'selected_index':selected}))
106             active_slot, active_hour = slot, hour
107             row = rows.iloc[slot]
108             event = execute_interval(float(current[slot]), float(row.load_actual_kwh),
109                                    float(row.pv_actual_kwh), energy, float(current_reserve[slot]), .9, .9)
110             energy = event['energy_end_actual_kwh']
111             event.update(date=date, slot_id=slot + 1, interval_start=str(row.interval_start),
112                         interval_end=str(row.interval_end), active_issue_hour=active_hour,
113                         load_actual_kwh=float(row.load_actual_kwh), pv_actual_kwh=float(row.pv_actual_kwh),
114                         load_forecast_kwh=float(load[slot - active_slot]), pv_forecast_kwh=float(pv[slot -
active_slot]),
115                         grid_original_kwh=float(q0[slot]), grid_effective_kwh=float(current[slot]),
116                         reserve_original_kwh=float(reserve0[slot]),
117                         price_yuan_per_kwh=float(data.fixed[slot] if cfg['billing']=='fixed' else row.
actual_price_yuan_per_kwh))
118             events.append(event)
119             frame = pd.DataFrame(events)
120             price = frame.price_yuan_per_kwh
121             delta = frame.grid_effective_kwh - frame.grid_original_kwh
122             frame['original_cost_yuan'] = price * frame.grid_original_kwh
123             frame['increase_cost_yuan'] = 1.5 * price * np.maximum(delta, 0)
124             frame['decrease_adjustment_yuan'] = -.5 * price * np.maximum(-delta, 0)
125             frame['contract_cost_yuan'] = frame.original_cost_yuan + frame.increase_cost_yuan + frame.
decrease_adjustment_yuan
126             frame['emergency_cost_yuan'] = 5 * price * frame.emergency_kwh
127             frame['total_cost_yuan'] = frame.contract_cost_yuan + frame.emergency_cost_yuan
128             frames.append(frame)
129             if date.endswith('-01'):

```



```

130         print(name, date, 'seconds', round(time.perf_counter() - tic, 2), flush=True)
131     ledger = pd.concat(frames, ignore_index=True)
132     ledger.to_csv(folder / 'ledger.csv', index=False)
133     pd.concat(versions, ignore_index=True).to_csv(folder / 'plan_versions.csv', index=False)
134     write(folder / 'decisions.json', metadata)
135     daily = ledger.groupby('date')[SUMS].sum()
136     daily['energy_start_actual_kwh'] = ledger.groupby('date').energy_start_actual_kwh.first()
137     daily['energy_end_actual_kwh'] = ledger.groupby('date').energy_end_actual_kwh.last()
138     daily.to_csv(folder / 'daily.csv')
139     optimizers = [o for d in metadata for o in d['optimizer']]
140     summary = {key:float(ledger[key].sum()) for key in SUMS}
141     summary.update(policy=name, branch=cfg['branch'], price_method=cfg['price'], control=cfg['control']
142     ],
143     billing=cfg['billing'], days=len(frames), intervals=len(ledger), initial_energy_kwh=initial,
144     final_energy_kwh=float(energy), optimizer_count=len(optimizers),
145     optimizer_success_count=sum(o['success'] for o in optimizers),
146     optimizer_non_success_count=sum(not o['success'] for o in optimizers),
147     optimizer_exception_count=sum(o['exception'] is not None for o in optimizers),
148     invalid_return_fallback_count=sum(o.get('returned_candidate_replaced',False) for o in
149     optimizers),
150     initializer_fallback_count=sum(d['initializer_failure'] is not None for d in metadata),
151     retained_start_selected_count=sum(d['candidate_labels'][d['selected_index']].endswith('_start')
152     for d in metadata),
153     keep_current_selected_count=sum(d['candidate_labels'][d['selected_index']]=='keep_current' for
154     d in metadata),
155     optimizer_nfev=sum(o['nfev'] for o in optimizers),
156     optimizer_total_wall_seconds=sum(o['wall_seconds'] for o in optimizers),
157     milp_total_runtime_seconds=sum(d['milp_initializer']['runtime_seconds'] for d in metadata),
158     wall_seconds=time.perf_counter() - tic,
159     inventory_adjusted_cost_yuan=summary['total_cost_yuan'] - .6895775 * (energy - initial),
160     emergency_margin_yuan=1000000. - summary['emergency_cost_yuan'],
161     passes_emergency_reference=summary['emergency_cost_yuan'] <= 1000000.)
162     write(folder / 'summary.json', summary)
163     print('DONE', name, summary['total_cost_yuan'], summary['emergency_cost_yuan'], flush=True)
164     return ledger
165
166 if __name__ == '__main__':
167     parser = argparse.ArgumentParser()
168     parser.add_argument('--policies', nargs='+', choices=list(POLICIES), default=list(POLICIES))
169     parser.add_argument('--stage', default='runs')
170     parser.add_argument('--start', default='2025-02-01')
171     parser.add_argument('--end', default='2025-12-31')
172     parser.add_argument('--initial', type=float, default=7268.4231640740745)
173     parser.add_argument('--control', choices=['legacy'], default=None, help='two-search regression only')
174
175     args = parser.parse_args()
176     for name in args.policies:
177         run_one(name, args.stage, args.start, args.end, args.initial, control=args.control)

```

Program 12. scripts/storage_control_v6/analyze.py

```

1  """Matched historical comparisons, with no production model imports or tuning."""
2  from pathlib import Path
3  import hashlib,json
4  import numpy as np
5  import pandas as pd
6
7  WORK=Path(__file__).resolve().parents[2]
8  ROOT=WORK/'results/storage_control_v6'
9  REPORT=WORK/'reports/storage_control_v6'
10 NU=.6895775
11
12 def read(path):return pd.read_csv(path,float_precision='round_trip')
13 def write(path,value):path.write_text(json.dumps(value,ensure_ascii=False,indent=2,allow_nan=False)+'\n')
14
15 def paired_record(candidate,reference,purpose,c,r,c_end,r_end):
16     if not c.index.equals(r.index):raise ValueError('paired dates differ')
17     values=r.total_cost_yuan-c.total_cost_yuan
18     month=values.groupby(values.index.str[:7]).sum()

```

```

18 total=float(values.sum());reference_total=float(r.total_cost_yuan.sum())
19 row=dict(candidate=candidate,reference=reference,purpose=purpose,total_saving_yuan=total,
20         saving_percent=100*total/reference_total if reference_total else None,
21         contract_saving_yuan=float((r.contract_cost_yuan-c.contract_cost_yuan).sum()),
22         emergency_saving_yuan=float((r.emergency_cost_yuan-c.emergency_cost_yuan).sum()),
23         winning_days=int((values>0).sum()),losing_days=int((values<0).sum()),tied_days=int((values==0).
sum()),
24         losing_months=', '.join(month.index[month<0]),winning_months=int((month>0).sum()),
25         inventory_adjusted_saving_yuan=total+NU*(c_end-r_end))
26 # Identical resampling draws for every comparison: fixed 20260915 seed.
27 rng=np.random.default_rng(20260915)
28 for block in [7,14]:
29     start=rng.integers(0,len(values),size=(2000,int(np.ceil(len(values)/block))))
30     indices=((start[:, :, None]+np.arange(block))%len(values)).reshape(2000,-1)[ :, :len(values)]
31     low,high=np.quantile(values.to_numpy()[indices].sum(axis=1),[.025,.975])
32     row.update({f'block{block}_lower_yuan':float(low),f'block{block}_upper_yuan':float(high),f'
block{block}_crosses_zero':bool(low<=0<=high)})
33 day_rows=[dict(candidate=candidate,reference=reference,date=k,saving_yuan=float(v)) for k,v in
values.items()]
34 month_rows=[dict(candidate=candidate,reference=reference,month=k,saving_yuan=float(v)) for k,v in
month.items()]
35 return row,day_rows,month_rows
36
37 def main():
38     config=json.loads((WORK/'configs/storage_control_v6/experiments.json').read_text())
39     rows=[];daily={};ends={};sources={}
40     for name,cfg in config['policies'].items():
41         folder=ROOT/'runs'/name
42         summary=json.loads((folder/'summary.json').read_text());ledger=read(folder/'ledger.csv')
43         if len(ledger)!=48096 or ledger.date.nunique()!=334:raise ValueError('Incomplete annual path '+
name)
44         row=dict(summary,base_policy=name.rsplit('_',1)[0],stage='matched_v6',source_ledger=str((folder
/'ledger.csv').relative_to(WORK)))
45         reserve=ledger.reserve_threshold_kwh.to_numpy();energy=ledger.energy_start_actual_kwh.to_numpy
()
46         deficit=np.maximum(ledger.load_actual_kwh-ledger.pv_actual_kwh-ledger.grid_effective_kwh,0)
47         greedy=np.minimum(np.minimum(deficit,5000/6),np.maximum(.9*(energy-1200),0))
48         held=np.maximum(greedy-ledger.discharge_actual_kwh,0)
49         row.update(reserve_above_floor_intervals=int((reserve>1200+1e-6).sum()),
50                 deliberate_retention_intervals=int((held>1e-6).sum()),
51                 withheld_discharge_kwh=float(held.sum()),
52                 emergency_while_retaining_intervals=int(((held>1e-6)&(ledger.emergency_kwh>1e-6)).sum()),
53                 emergency_intervals=int((ledger.emergency_kwh>1e-7).sum()))
54         rows.append(row);daily[name]=ledger.groupby('date')[['total_cost_yuan','contract_cost_yuan','
emergency_cost_yuan']].sum()
55         ends[name]=float(ledger.energy_end_actual_kwh.iloc[-1]);sources[name]={'path':row['
source_ledger'],'sha256':hashlib.sha256((folder/'ledger.csv').read_bytes()).hexdigest()}
56     comparison=pd.DataFrame(rows);comparison.to_csv(ROOT/'comparison_all.csv',index=False)
57     selected={'q1':'baseline'}
58     for branch in ['q2','q3','q42','q43']:
59         eligible=comparison.loc[(comparison.branch==branch)&(comparison.control=='reserve')]
60         selected[branch]=eligible.sort_values(['total_cost_yuan','emergency_cost_yuan'],kind='stable').
iloc[0].policy
61     write(ROOT/'selection.json',dict(rule='minimum unrounded reserve-arm historical total, emergency
fee, then frozen matrix order; greedy benchmarks retained separately',selected=selected,
historical_exploration=True))
62     pairs=[]
63     for name in config['policies']:
64         if name.endswith('_reserve'):pairs.append((name,name.removesuffix('_reserve')+'_greedy',
storage-control freedom; matched four-search protocol'))
65     for arm in ['greedy','reserve']:
66         pairs.extend([(f'q3_soc_{arm}',f'q3_no_update_{arm}','state feedback and paid revisions'),
67                     (f'q3_raw_{arm}',f'q3_soc_{arm}','incremental raw PV updates'),
68                     (f'q3_w28_{arm}',f'q3_raw_{arm}','Q3 PV correction'),
69                     (f'q42_ols_{arm}',f'q42_fixed_{arm}','Q42 planning-price input'),
70                     (f'q43_raw_ols_{arm}',f'q43_raw_fixed_{arm}','Q43 planning-price input raw PV'),
71                     (f'q43_w28_ols_{arm}',f'q43_raw_ols_{arm}','Q43 PV correction OLS price'),
72                     (f'q43_w28_fixed_{arm}',f'q43_raw_fixed_{arm}','Q43 PV correction fixed price')])
73     # Frozen v5 is a named historical reference with its original two-search budget.

```

```

74 frozen={'q2':'results/q2_direct_v4/runs/D112','q3':'results/unified_direct_v5/runs/q3_raw','q42':'
    results/unified_direct_v5/runs/q42_fixed','q43':'results/unified_direct_v5/runs/q43_raw_ols'}
75 for branch,rel in frozen.items():
76     name='frozen_v5_'+branch;path=WORK/rel/'ledger.csv';f=read(path)
77     if branch=='q2':f['contract_cost_yuan']=f.planned_cost_yuan
78     daily[name]=f.groupby('date')[['total_cost_yuan','contract_cost_yuan','emergency_cost_yuan']].
    sum();ends[name]=float(f.energy_end_actual_kwh.iloc[-1])
79     sources[name]={'path':str(path.relative_to(WORK)),'sha256':hashlib.sha256(path.read_bytes()).
    hexdigest()}
80     pairs.append((selected[branch],name,'selected reserve versus frozen two-search v5; combined
    controller/search/selection change'))
81 records=[];day_rows=[];month_rows=[]
82 for c,r,purpose in pairs:
83     row,days,months=paired_record(c,r,purpose,daily[c],daily[r],ends[c],ends[r]);records.append(row
    );day_rows.extend(days);month_rows.extend(months)
84 pd.DataFrame(records).to_csv(ROOT/'paired_summary.csv',index=False)
85 pd.DataFrame(day_rows).to_csv(ROOT/'paired_daily.csv',index=False);pd.DataFrame(month_rows).to_csv(
    ROOT/'paired_monthly.csv',index=False)
86 months=[]
87 for name,day in daily.items():
88     frame=day.groupby(day.index.str[:7]).sum().reset_index(names='month');frame['policy']=name;
    months.append(frame)
89 pd.concat(months).to_csv(ROOT/'monthly_all.csv',index=False)
90 write(REPORT/'analysis_sources.json',dict(sources=sources,bootstrap=dict(seed=20260915,replicates
    =2000,blocks=[7,14],identical_draws_across_pairs=True,scope='descriptive fixed historical paths,
    not selection-adjusted or new physical simulations'))))
91 write(REPORT/'analysis_summary.json',dict(policies=len(rows),pairs=len(records),selected=selected,
92     monetary_identity_max_error=max(abs(r['total_saving_yuan']-r['contract_saving_yuan']-r['
    emergency_saving_yuan']) for r in records),
93     original_results_modified=False))
94 print(json.dumps(selected,ensure_ascii=False,indent=2));print(comparison[['policy','total_cost_yuan
    ','emergency_cost_yuan','deliberate_retention_intervals']].to_string(index=False))
95 if __name__=='__main__':main()

```

Program 13. scripts/check_saved.py

```

1  """Lightweight independent checks of saved plans, costs and workbook identities.
2
3  This file does not import a production optimizer. The optional full replay uses
4  prepared original input data but never searches for a new procurement strategy.
5  """
6  from pathlib import Path
7  from collections import Counter
8  import argparse
9  import hashlib
10 import json
11 import math
12 import numpy as np
13 import pandas as pd
14 import openpyxl
15
16 WORK=Path(__file__).resolve().parents[1]
17 counts=Counter();errors=[];maximum={}
18 def read(path):return pd.read_csv(path,float_precision='round_trip')
19 def check(name,actual,expected,tol=1e-6):
20     a,b=np.asarray(actual),np.asarray(expected)
21     counts[name]+=int(a.size)
22     if a.shape!=b.shape:
23         errors.append({'check':name,'shape_actual':list(a.shape),'shape_expected':list(b.shape)});
24         return
25     if a.dtype.kind in 'iu' and b.dtype.kind in 'iu':
26         diff=float(np.max(np.abs(a-b))) if a.size else 0.
27         maximum[name]=max(maximum.get(name,0.),diff)
28         ok=bool(np.all(np.isfinite(a)) and np.all(np.isfinite(b)) and diff<=tol)
29     else:ok=bool(np.array_equal(a,b))
30     if not ok:errors.append({'check':name,'max_error':maximum.get(name)})
31
32 def replay(frame,initial):
33     """Independent sequential balance, reserve discharge and Rule-A cash replay."""
34     energy=float(initial);records=[]

```

```

34 for row in frame.itertuples():
35     q=float(row.grid_effective_kwh);q0=float(row.grid_original_kwh)
36     reserve=float(row.reserve_threshold_kwh)
37     net=q+float(row.pv_actual_kwh)-float(row.load_actual_kwh)
38     start=energy
39     if net>=0:
40         charge=min(net,5000./6,max(0.,(10800.-energy)/.9));discharge=emergency=0.
41         surplus=net-charge;energy+=.9*charge
42     else:
43         charge=surplus=0.
44         discharge=min(-net,5000./6,max(0.,.9*(energy-reserve)))
45         emergency=-net-discharge;energy-=discharge/.9
46     price=float(row.price_yuan_per_kwh)
47     original=price*q0;increase=1.5*price*max(q-q0,0.)
48     decrease=-.5*price*max(q0-q,0.)
49     contract=original+increase+decrease;urgent=5*price*emergency
50     records.append({'charge_actual_kwh':charge,'discharge_actual_kwh':discharge,
51                    'emergency_kwh':emergency,'surplus_kwh':surplus,
52                    'unused_grid_kwh':min(q,surplus),'pv_curtailment_kwh':surplus-min(q,surplus),
53                    'energy_start_actual_kwh':start,'energy_end_actual_kwh':energy,
54                    'original_cost_yuan':original,'increase_cost_yuan':increase,
55                    'decrease_adjustment_yuan':decrease,'contract_cost_yuan':contract,
56                    'emergency_cost_yuan':urgent,'total_cost_yuan':contract+urgent})
57 return pd.DataFrame(records,index=frame.index)
58
59 def main():
60     parser=argparse.ArgumentParser(description=__doc__)
61     parser.add_argument('--full-inputs',action='store_true',help='Replay all selected annual paths
        using data prepared from the original attachments')
62     parser.add_argument('--output',type=Path,default=WORK/'check_results.json')
63     args=parser.parse_args()
64     mapping=json.loads((WORK/'workbooks/policy_mapping.json').read_text())
65     workbook_records=[]
66     for item in mapping['workbooks']:
67         path=WORK/'workbooks'/item['file']
68         check('workbook_binary_identity',hashlib.sha256(path.read_bytes()).hexdigest(),item['
        english_workbook_sha256'])
69         wb=openpyxl.load_workbook(path,data_only=False)
70         check('sheet_names',wb.sheetnames,[s['english_sheet_name'] for s in item['sheets']])
71         cell_count=formulas=dates=0
72         for sheet,expected in zip(wb,item['sheets']):
73             check('worksheet_dimensions',[sheet.max_row,sheet.max_column],[expected['rows'],expected['
        columns']])
74             for row in sheet:
75                 for cell in row:
76                     cell_count+=1;formulas+=cell.data_type=='f';dates+=cell.data_type=='d'
77                     if cell.data_type=='e':errors.append({'check':'spreadsheet_error','file':item['file
        '], 'cell':cell.coordinate})
78             workbook_records.append({'file':item['file'],'cells_read':cell_count,'formulas':formulas,'dates
        ':dates})
79         wb.close()
80     q1=read(WORK/'saved/q1/baseline/schedule.csv')
81     check('q1_slot_keys',q1.slot_id,np.arange(1,145))
82     check('q1_balance',q1.grid_kwh+q1.pv_forecast_kwh+q1.discharge_kwh-q1.load_kwh-q1.charge_kwh-q1.
        curtailment_kwh,np.zeros(144))
83     check('q1_state',q1.energy_start_kwh+.9*q1.charge_kwh-q1.discharge_kwh/.9,q1.energy_end_kwh)
84     check('q1_state_continuity',q1.energy_start_kwh.iloc[1:],q1.energy_end_kwh.iloc[:-1])
85     check('q1_endpoints',[q1.energy_start_kwh.iloc[0],q1.energy_end_kwh.iloc[-1]],[6000.,6000.])
86     check('q1_cash',q1.grid_kwh*q1.price_yuan_per_kwh,q1.cost_yuan)
87     check('q1_modes',np.minimum(q1.charge_kwh,q1.discharge_kwh),np.zeros(144))
88     summary=json.loads((WORK/'saved/q1/baseline/summary.json').read_text())
89     check('q1_total',math.fsum(q1.cost_yuan),summary['cost_yuan'])
90     fixed=read(WORK/'saved/fixed_price.csv').price_yuan_per_kwh.to_numpy()
91     check('diagnostic_inventory_value',float(fixed.mean()*0.9),.6895775,1e-12)
92     warm=read(WORK/'saved/warmup/ledger.csv')
93     check('warmup_start',float(warm.energy_start_actual_kwh.iloc[0]),6000.)
94     check('warmup_continuity',warm.energy_start_actual_kwh.iloc[1:],warm.energy_end_actual_kwh.iloc
        [:-1])
95     check('warmup_final',float(warm.energy_end_actual_kwh.iloc[-1]),7268.4231640740745)
96

```

```

97 config=json.loads((WORK/'configs/storage_control_v6/experiments.json').read_text())['policies']
98 selection=json.loads((WORK/'saved/comparisons/selection.json').read_text())['selected']
99 full_source=read(WORK/'data/processed/actual_10min.csv') if args.full_inputs else None
100 for name in [value for key,value in selection.items() if key!='q1']:
101     folder=WORK/'saved/selected'/name
102     with np.load(folder/'execution_plans.npz',allow_pickle=False) as packed:
103         plans=pd.DataFrame({key:packed[key] for key in packed.files})
104     with np.load(folder/'plan_versions.npz',allow_pickle=False) as packed:
105         versions=pd.DataFrame({key:packed[key] for key in packed.files})
106     check('annual_row_count',len(plans),48096,0)
107     check('annual_slot_keys',plans.slot_id,np.tile(np.arange(1,145),334),0)
108     check('annual_dates',plans.date,np.repeat(pd.date_range('2025-02-01','2025-12-31').strftime('%Y
-%m-%d'),144))
109     check('q_nonnegative',bool((plans[['grid_original_kwh','grid_effective_kwh']]>=0).all().all()),
True)
110     check('reserve_bounds',bool(plans.reserve_threshold_kwh.between(1200.,10800.).all()),True)
111     hours=[0,6,12,18] if config[name]['updates'] else [0]
112     for date,day in plans.groupby('date',sort=False):
113         current_q=current_r=None
114         for hour in hours:
115             issued=versions.loc[(versions.date==date)&(versions.issue_hour==hour)]
116             start=hour*6
117             check('version_slot_keys',issued.slot_id,np.arange(start+1,145),0)
118             check('version_q0',issued.grid_original_kwh,day.grid_original_kwh.to_numpy()[start:],0)
119             if hour==0:
120                 current_q=issued.grid_kwh.to_numpy().copy();current_r=issued.reserve_threshold_kwh.
to_numpy().copy()
121             else:
122                 current_q[start:]=issued.grid_kwh.to_numpy();current_r[start:]=issued.
reserve_threshold_kwh.to_numpy()
123                 end=min(start+36,144) if config[name]['updates'] else 144
124                 check('executed_q_from_issued_version',day.grid_effective_kwh.to_numpy()[start:end],
current_q[start:end],0)
125                 check('executed_R_from_issued_version',day.reserve_threshold_kwh.to_numpy()[start:end],
current_r[start:end],0)
126                 four=read(folder/'ledger_four_dates.csv')
127                 for date,frame in four.groupby('date',sort=False):
128                     rebuilt=replay(frame,float(frame.energy_start_actual_kwh.iloc[0]))
129                     for column in rebuilt:check('four_date_'+column,rebuilt[column],frame[column])
130                     check('four_date_energy_bounds',bool(rebuilt.energy_end_actual_kwh.between(1200.-1e
-6,10800.+1e-6).all()),True)
131                     check('four_date_modes',np.minimum(rebuilt.charge_actual_kwh,rebuilt.discharge_actual_kwh),
np.zeros(144))
132                     if args.full_inputs:
133                         source=full_source.loc[full_source.date>='2025-02-01'].reset_index(drop=True)
134                         frame=plans.copy()
135                         check('raw_actual_keys',source[['date','slot_id']].astype(str),frame[['date','slot_id']].
astype(str))
136                         frame['load_actual_kwh']=source.load_actual_kwh;frame['pv_actual_kwh']=source.pv_actual_kwh
137                         frame['price_yuan_per_kwh']=np.tile(fixed,334) if config[name]['billing']=='fixed' else
source.actual_price_yuan_per_kwh
138                         rebuilt=replay(frame,7268.4231640740745)
139                         saved=read(WORK/f'saved/all_policies/{name}/daily.csv')
140                         rebuilt['date']=frame.date
141                         sums=rebuilt.groupby('date').sum(numeric_only=True)
142                         for column in ['original_cost_yuan','increase_cost_yuan','decrease_adjustment_yuan','
contract_cost_yuan','emergency_cost_yuan','total_cost_yuan','charge_actual_kwh','
discharge_actual_kwh','emergency_kwh']:
143                             check('annual_'+column,sums[column],saved[column],1e-5)
144                             check('annual_end_energy',rebuilt.groupby('date').energy_end_actual_kwh.last(),saved.
energy_end_actual_kwh)
145                     for name in config:
146                         folder=WORK/'saved/all_policies'/name
147                         daily=read(folder/'daily.csv');summary=json.loads((folder/'summary.json').read_text())
148                         check('all_policy_days',len(daily),334,0)
149                         for field in ['original_cost_yuan','increase_cost_yuan','decrease_adjustment_yuan','
contract_cost_yuan','emergency_cost_yuan','total_cost_yuan']:
150                             check('all_policy_'+field,math.fsum(daily[field]),summary[field],1e-5)
151                         check('all_policy_cash_components',daily.contract_cost_yuan+daily.emergency_cost_yuan,daily.
total_cost_yuan,1e-6)

```

```

152 report={'passed':not errors,'full_input_replay':args.full_inputs,'annual_optimizations_run':0,
153         'checks':dict(counts),'maximum_absolute_errors':maximum,'errors':errors,'workbooks':
        workbook_records,
154         'limitations':'Default mode checks four specified dates, all annual issued q/R versions,
        all saved daily summaries and exact workbook identities. It does not certify optimality or rerun
        historical policy searches.')}
155 args.output.parent.mkdir(parents=True,exist_ok=True)
156 args.output.write_text(json.dumps(report,indent=2)+'\n')
157 print(json.dumps({'passed':report['passed'],'checks':sum(counts.values()),'errors':len(errors),'
        full_input_replay':args.full_inputs}))
158 raise SystemExit(0 if report['passed'] else 1)
159
160 if __name__=='__main__':main()

```

Program 14. scripts/smoke_test.py

```

1 """Compile/import the scientific core and solve two bounded diagnostic problems."""
2 from pathlib import Path
3 import importlib.metadata
4 import json
5 import sys
6 import numpy as np
7 import pandas as pd
8
9 WORK=Path(__file__).resolve().parents[1]
10 sys.path.insert(0,str(WORK/'scripts/storage_control_v6'))
11 import kernel
12 from policy import decision
13 from solve_q1 import solve_model
14
15 physical=json.loads((WORK/'configs/model_baseline.json').read_text())
16 data=pd.read_csv(WORK/'saved/q1/input_day.csv',float_precision='round_trip')
17 _,status=solve_model(data,physical)
18 expected=json.loads((WORK/'saved/q1/baseline/summary.json').read_text())['cost_yuan']
19 assert abs(status['objective_yuan']-expected)<1e-5
20 # A short synthetic remaining-day problem exercises the real finite q/R search.
21 load=np.full(6,500.);pv=np.zeros(6);price=np.array([.4,.5,.6,.7,.8,.9])
22 net=np.vstack([load-100.,load+100.]);residual=np.vstack([np.full(6,-100.),np.full(6,100.)])
23 result=decision(load,pv,net,residual,price,6000.,138,physical,control='reserve')
24 i=int(result['selected_index']);q=result['q'][i];reserve=result['reserve'][i]
25 assert len(result['q'])==12 and np.all(q>=0) and np.all((reserve>=1200)&(reserve<=10800))
26 assert np.isfinite(result['score']).all()
27 out={'passed':True,'q1_objective_yuan':status['objective_yuan'],
28      'short_diagnostic_intervals':6,'short_diagnostic_candidates':len(result['q']),
29      'annual_optimizations_run':0,'selected_diagnostic_candidate':i,
30      'packages':{name:importlib.metadata.version(name) for name in ['numpy','pandas','scipy','highspy',
        'openpyxl']}}
31 (WORK/'smoke_results.json').write_text(json.dumps(out,indent=2)+'\n')
32 print(json.dumps(out))

```

Complete configuration files

Configuration. configs/data_processing.json

```

1 {
2   "source_directory": "raw",
3   "time_zone": "naive_problem_local_calendar",
4   "raw_time_label": "interval_end_assumption",
5   "power_interpretation": "interval_average_assumption",
6   "interval_minutes": 10,
7   "actual_available_at": "interval_end_inclusive_assumption",
8   "forecast_conversion": "piecewise_linear_point_power_trapezoid",
9   "first_hour_endpoint": "latest_strictly_earlier_issue_forecast_targeting_current_issue",
10  "missing_endpoint": "omit_first_hour_record_gap_no_zero_fill",
11  "future_actual_price_known": false,
12  "forecast_evaluation": "complete_hour_energy_six_10min_intervals",
13  "assumption_status": "pending_team_confirmation"
14 }

```

Configuration. configs/model_baseline.json

```

1 {
2   "status": "internal_working_assumptions_not_submission",
3   "input": "data/processed/q1_day.csv",
4   "interval_minutes": 10,
5   "time_mapping": "raw label denotes interval end; 00:10 -> [00:00,00:10); no shift or circular wrap",
6   "power_interpretation": "interval_average",
7   "template_export": "blocked_pending_time_mapping_confirmation",
8   "battery": {
9     "rated_capacity_kwh": 12000,
10    "min_energy_kwh": 1200,
11    "max_energy_kwh": 10800,
12    "max_charge_kw": 5000,
13    "max_discharge_kw": 5000,
14    "power_boundary": "AC_bus_side_working_assumption",
15    "initial_energy_kwh": 6000,
16    "initial_basis": "Chinese Appendix 1 fixes Jan 1 at 6000; English says start of operation; adopted
17    for undated Q1, not a source date",
18    "terminal": "equal_to_fixed_initial",
19    "eta_charge": 0.9,
20    "eta_discharge": 0.9,
21    "efficiency_basis": "each_direction_0.9_working_assumption_not_official"
22  },
23  "sensitivity": {"name": "roundtrip_90", "eta_charge": 0.9486832980505138, "eta_discharge":
24    0.9486832980505138},
25  "curtailment": "PV_only_0_to_given_PV; no_grid_sale; no_disposal_of_purchased_energy",
26  "solver": {
27    "name": "HiGHS",
28    "threads": 1,
29    "random_seed": 0,
30    "time_limit": 120,
31    "mip_rel_gap": 1e-9,
32    "mip_abs_gap": 1e-7,
33    "primal_feasibility_tolerance": 1e-8,
34    "dual_feasibility_tolerance": 1e-8,
35    "mip_feasibility_tolerance": 1e-8
36  },
37  "validation": {"energy_atol_kwh": 1e-6, "cost_atol_yuan": 1e-6, "integer_atol": 1e-7}
38 }

```

Configuration. configs/q2_baseline.json

```

1 {
2   "status": "internal_baseline_not_submission",
3   "q1_parameter_source": "configs/model_baseline.json",
4   "initialization_start": "2025-01-01",
5   "evaluation_start": "2025-02-01",
6   "evaluation_end": "2025-12-31",
7   "calibration_start": "2025-01-15",
8   "calibration_end": "2025-01-31",
9   "candidates": ["seasonal", "linear_persistence", "linear_harmonic"],
10  "load_training_days": 28,
11  "load_harmonics": 2,
12  "min_load_training_days": 7,
13  "pv_training_days": 7,
14  "pv_harmonics": 3,
15  "ar1_bound": 0.99,
16  "cold_start": "Jan1_no_history_zero_day_ahead_commitment; forecasts_NA_not_imputed;
17  greedy_actual_execution",
18  "common_warmup": "Jan1-Jan31 seasonal baseline continuous actual SOC; all evaluation policies share
19  Feb1 state",
20  "selection": "Jan15-Jan31 continuous shadow replay from same Jan15 SOC; minimize actual_cost minus
21  mean_fixed_price*eta_discharge*(end_energy-start_energy); freeze Feb1",
22  "terminal_inventory_value": "mean_known_fixed_price_times_eta_discharge;
23  comparison_proxy_only_not_grid_bill",
24  "plan_terminal": "equal_to_actual_start_of_each_day; baseline_assumption_only",
25  "actual_terminal": "carry_actual_state_forward_without_reset",
26  "executor": "fixed_grid_commitment; surplus_charge_first; deficit_discharge_first;
27  remaining_deficit_emergency",

```

```

23 "surplus_accounting": "PV_priority_convention; unused_grid=min(total_surplus,planned_grid);
    pv_curtailment=remaining_surplus",
24 "emergency_price_multiplier": 5,
25 "time_mapping": "inherit_Q1_interval_end_assumption; formal_template_pending",
26 "scope": "Q2_base_closed_loop; no_Q3_Q4_or_optional_risk_models"
27 }

```

Configuration. configs/q4_baseline.json

```

1 {
2   "status": "internal_working_assumptions_not_submission",
3   "evaluation_start": "2025-02-01",
4   "evaluation_end": "2025-12-31",
5   "price_information": "Unknown future price by default; observed only at interval_end <= decision_time
    . Perfect-price branches are explicitly hypothetical.",
6   "settlement": "All contracted and emergency energy settled at target interval actual price; final
    effective quantity billed once relative to midnight original. No advance price locking.",
7   "price_model": "OLS intercept, daily Fourier K=2, price lag1day and lag7day, six weekday indicators;
    last 28 completed days/intervals, re-fit at each allowed decision",
8   "price_training_days": 28,
9   "price_harmonics": 2,
10  "price_floor_yuan_per_kwh": 0.000001,
11  "price_floor_basis": "Predeclared numerical positivity floor, never estimated from future minima;
    record every activation",
12  "forecasts": "Q4-2 inherits frozen Q2 selected load and PV forecasts. Q4-3 inherits same load and
    exact issue PV from Appendix 3. No load or PV re-selection.",
13  "initial_state": "Common Q2 January warmup February 1 state; actual state continues independently per
    policy",
14  "terminal_state": "Each forecast horizon ends at today's midnight actual state; extra baseline
    assumption, actual state never reset",
15  "surplus": "Allow free disposal of paid grid and PV, no sale; Q3 planner/executor unchanged",
16  "inventory_proxy": "Known Appendix1 mean fixed price times discharge efficiency, not realized future
    average price",
17  "policies": {
18    "q42_fixed": {"branch": "4-2", "price": "fixed", "rule": "A", "update_hours": []},
19    "q42_ols": {"branch": "4-2", "price": "ols", "rule": "A", "update_hours": []},
20    "q42_perfect": {"branch": "4-2", "price": "perfect", "rule": "A", "update_hours": []},
21    "q43_no_update_ols": {"branch": "4-3", "price": "ols", "rule": "A", "update_hours": []},
22    "q43_all_A_ols": {"branch": "4-3", "price": "ols", "rule": "A", "update_hours": [6, 12, 18]},
23    "q43_all_B_ols": {"branch": "4-3", "price": "ols", "rule": "B", "update_hours": [6, 12, 18]},
24    "q43_all_A_hold0": {"branch": "4-3", "price": "hold0", "rule": "A", "update_hours": [6, 12, 18]},
25    "q43_all_A_fixed": {"branch": "4-3", "price": "fixed", "rule": "A", "update_hours": [6, 12, 18]},
26    "q43_all_A_perfect": {"branch": "4-3", "price": "perfect", "rule": "A", "update_hours": [6, 12,
    18]}
27  },
28  "design_disclosure": "Policy families fixed before this Q4 calculation, after Q2/Q3 annual results
    were inspected; retrospective evaluation, no full-year retuning or claim of untouched test
    selection",
29  "formal_excel_exported": false
30 }

```

Configuration. configs/storage_control_v6/experiments.json

```

1 {
2   "policies": {
3     "q2_greedy": {
4       "branch": "q2",
5       "price": "fixed",
6       "updates": false,
7       "pv_update": false,
8       "corrected": false,
9       "with_pv": false,
10      "billing": "fixed",
11      "control": "greedy"
12    },
13    "q2_reserve": {
14      "branch": "q2",
15      "price": "fixed",
16      "updates": false,

```



```

17     "pv_update": false,
18     "corrected": false,
19     "with_pv": false,
20     "billing": "fixed",
21     "control": "reserve"
22 },
23 "q3_no_update_greedy": {
24     "branch": "q3",
25     "price": "fixed",
26     "updates": false,
27     "pv_update": false,
28     "corrected": false,
29     "with_pv": true,
30     "billing": "fixed",
31     "control": "greedy"
32 },
33 "q3_no_update_reserve": {
34     "branch": "q3",
35     "price": "fixed",
36     "updates": false,
37     "pv_update": false,
38     "corrected": false,
39     "with_pv": true,
40     "billing": "fixed",
41     "control": "reserve"
42 },
43 "q3_soc_greedy": {
44     "branch": "q3",
45     "price": "fixed",
46     "updates": true,
47     "pv_update": false,
48     "corrected": false,
49     "with_pv": true,
50     "billing": "fixed",
51     "control": "greedy"
52 },
53 "q3_soc_reserve": {
54     "branch": "q3",
55     "price": "fixed",
56     "updates": true,
57     "pv_update": false,
58     "corrected": false,
59     "with_pv": true,
60     "billing": "fixed",
61     "control": "reserve"
62 },
63 "q3_raw_greedy": {
64     "branch": "q3",
65     "price": "fixed",
66     "updates": true,
67     "pv_update": true,
68     "corrected": false,
69     "with_pv": true,
70     "billing": "fixed",
71     "control": "greedy"
72 },
73 "q3_raw_reserve": {
74     "branch": "q3",
75     "price": "fixed",
76     "updates": true,
77     "pv_update": true,
78     "corrected": false,
79     "with_pv": true,
80     "billing": "fixed",
81     "control": "reserve"
82 },
83 "q3_w28_greedy": {
84     "branch": "q3",
85     "price": "fixed",
86     "updates": true,

```

```

87     "pv_update": true,
88     "corrected": true,
89     "with_pv": true,
90     "billing": "fixed",
91     "control": "greedy"
92 },
93 "q3_w28_reserve": {
94     "branch": "q3",
95     "price": "fixed",
96     "updates": true,
97     "pv_update": true,
98     "corrected": true,
99     "with_pv": true,
100    "billing": "fixed",
101    "control": "reserve"
102 },
103 "q42_ols_greedy": {
104     "branch": "q42",
105     "price": "ols",
106     "updates": false,
107     "pv_update": false,
108     "corrected": false,
109     "with_pv": false,
110     "billing": "actual",
111     "control": "greedy"
112 },
113 "q42_ols_reserve": {
114     "branch": "q42",
115     "price": "ols",
116     "updates": false,
117     "pv_update": false,
118     "corrected": false,
119     "with_pv": false,
120     "billing": "actual",
121     "control": "reserve"
122 },
123 "q42_fixed_greedy": {
124     "branch": "q42",
125     "price": "fixed",
126     "updates": false,
127     "pv_update": false,
128     "corrected": false,
129     "with_pv": false,
130     "billing": "actual",
131     "control": "greedy"
132 },
133 "q42_fixed_reserve": {
134     "branch": "q42",
135     "price": "fixed",
136     "updates": false,
137     "pv_update": false,
138     "corrected": false,
139     "with_pv": false,
140     "billing": "actual",
141     "control": "reserve"
142 },
143 "q43_raw_ols_greedy": {
144     "branch": "q43",
145     "price": "ols",
146     "updates": true,
147     "pv_update": true,
148     "corrected": false,
149     "with_pv": true,
150     "billing": "actual",
151     "control": "greedy"
152 },
153 "q43_raw_ols_reserve": {
154     "branch": "q43",
155     "price": "ols",
156     "updates": true,

```

```

157     "pv_update": true,
158     "corrected": false,
159     "with_pv": true,
160     "billing": "actual",
161     "control": "reserve"
162 },
163 "q43_w28_ols_greedy": {
164     "branch": "q43",
165     "price": "ols",
166     "updates": true,
167     "pv_update": true,
168     "corrected": true,
169     "with_pv": true,
170     "billing": "actual",
171     "control": "greedy"
172 },
173 "q43_w28_ols_reserve": {
174     "branch": "q43",
175     "price": "ols",
176     "updates": true,
177     "pv_update": true,
178     "corrected": true,
179     "with_pv": true,
180     "billing": "actual",
181     "control": "reserve"
182 },
183 "q43_raw_fixed_greedy": {
184     "branch": "q43",
185     "price": "fixed",
186     "updates": true,
187     "pv_update": true,
188     "corrected": false,
189     "with_pv": true,
190     "billing": "actual",
191     "control": "greedy"
192 },
193 "q43_raw_fixed_reserve": {
194     "branch": "q43",
195     "price": "fixed",
196     "updates": true,
197     "pv_update": true,
198     "corrected": false,
199     "with_pv": true,
200     "billing": "actual",
201     "control": "reserve"
202 },
203 "q43_w28_fixed_greedy": {
204     "branch": "q43",
205     "price": "fixed",
206     "updates": true,
207     "pv_update": true,
208     "corrected": true,
209     "with_pv": true,
210     "billing": "actual",
211     "control": "greedy"
212 },
213 "q43_w28_fixed_reserve": {
214     "branch": "q43",
215     "price": "fixed",
216     "updates": true,
217     "pv_update": true,
218     "corrected": true,
219     "with_pv": true,
220     "billing": "actual",
221     "control": "reserve"
222 }
223 },
224 "optimizer": {
225     "maxiter": 180,
226     "maxfun": 1600,

```

```

227     "maxls": 30,
228     "maxcor": 10,
229     "ftol": 1e-10,
230     "gtol": 1e-05
231 },
232 "scenario_window": 112,
233 "kappa": 5,
234 "mu": 0.38232,
235 "initial_energy_kwh": 7268.4231640740745,
236 "protocol": "causal_q_reserve_v6",
237 "control_bounds_kwh": [
238     1200,
239     10800
240 ],
241 "searches_per_issue": 4,
242 "midnight_candidate_count": 12,
243 "paid_update_candidate_count": 13,
244 "legacy_regression_searches_per_issue": 2,
245 "procurement_variable_scale": 1000,
246 "reserve_variable_scale": 1000,
247 "objective_scale": 10000,
248 "stable_selection_tolerance_yuan": 1e-08,
249 "candidate_retention": [
250     "start",
251     "best_evaluated",
252     "returned"
253 ],
254 "reserve_seed": [
255     "nominal_MILP_energy_end",
256     "constant_1200"
257 ],
258 "selection": "minimum unrounded actual cash per reserve branch; emergency fee then declared order
    breaks exact ties",
259 "billing_rule": "A",
260 "warmup": "shared frozen January seasonal baseline; separate continuous state from February 1",
261 "actual_price_access": "realized billing only; planning as declared price input",
262 "q1": "unchanged frozen verified MILP"
263 }

```

Exact Python dependencies

```

1 numpy==2.3.5
2 pandas==2.2.3
3 scipy==1.18.1
4 highspy==1.14.0
5 openpyxl==3.1.5
6 matplotlib==3.11.1

```